



D8 G2B Processing External API Technical Specification

Version 1.80

2026

1 Content

1	Content.....	2
2	Revision History.....	5
3	About document.....	17
4	Protocol.....	17
4.1	Connection.....	17
4.2	Http method	17
4.3	Http headers.....	17
4.4	Data format.....	17
4.5	URL format.....	17
4.6	Labels and formats.....	17
4.7	Field Restriction Control	18
4.8	Types description.....	18
5	Request and Response formats	18
5.1	Request	18
5.2	Response.....	21
5.3	Common Objects.....	24
5.1	Data encryption - encryptedData in response.....	27
5.2	Data encryption - PIN data in request	29
6	Kernel Services	30
6.1	xkernel/getLkey.....	30
6.2	xkernel/getCompanyList	31
6.3	xkernel/getMerchant	32
6.4	xkernel/getMerchantList.....	33
7	Issuer Services.....	34
7.1	xmiss/getCardInfo	34
7.2	xmiss/getCardBasicInfo	36
7.3	xmiss/getCardAccounts	37
7.4	xmiss/getCardNotifications	38
7.5	xmiss/setCardStatus	39
7.6	xmiss/activateCard	39
7.7	xmiss/generatePIN.....	40
7.8	xmiss/setPIN.....	40
7.9	xmiss/getCVV2.....	41

7.10	xmiss/setCVV2Type.....	41
7.11	xmiss/getLimits.....	42
7.12	xmiss/setLimits.....	43
7.13	xmiss/removeLimits.....	44
7.14	xmiss/resetCardPINtries	45
7.15	xmiss/setCardAccountPriority.....	45
7.16	xmiss/getAuthRestrictions.....	46
7.17	xmiss/addAuthRestrictions.....	47
7.18	xmiss/removeAuthRestrictions.....	47
7.19	xmiss/updateAccountBalance.....	48
7.20	xmiss/batchUpdateAccountBalances	49
7.21	xmiss/resetCardCVV2Counter	51
7.22	xmiss/verifyPIN.....	52
7.23	xmiss/getCustomerBasicInfo.....	53
7.24	xmiss/getCustomerAccountList.....	54
7.25	xmiss/getCustomerCardList	55
7.26	xmiss/getCustomerCardAccountLinkageList	55
7.27	xmiss/getAccountBasicInfo	56
7.28	xmiss/getAccountCardList	56
7.29	xmiss/getAccountLimitList	56
7.30	xmiss/getAccountStatement.....	57
8	Transaction Services	60
8.1	Overview.....	60
8.2	xkernel/initiateTransaction	60
8.3	xkernel/getCardTransactionHistory.....	60
8.4	xkernel/getTransactionDetails.....	62
8.5	xkernel/authorizeTransaction.....	66
8.6	xkernel/reverseTransaction.....	76
8.7	xkernel/getTransactionStatus.....	77
8.8	xkernel/calculateAcqCommission	78
9	POS services	78
9.1	xpos/getPosTypeList.....	78
9.2	xpos/getPos	79
9.3	xpos/getPosList.....	80

10	Data import services	81
10.1	xkernel/mdi.....	81
10.2	xmiss/mdi.....	82
10.3	xpos/mdi.....	82
11	Proxy services	83
11.1	xmiss/g2bHceProxy	83
12	Examples	84
	Appendix A - Response Codes	125
	API Codes	128
	Appendix B - Transaction Status Codes.....	128
	Appendix C - Message Reason Codes.....	130
	Appendix D Money Transfer Data - Business Application Identifier	131
	Appendix E - Money Transfer Data - Funding source	132
	Appendix F – Sub-transaction Code.....	133
	Appendix G - PIN block formats.....	133
13	Appendix H – Encryption Examples.....	134
13.1	List of example files	134
13.2	Example of encryptedData in response (getLKey).....	135
13.3	Example of PIN data in request (setPIN).....	136
13.4	Example of PIN data in request (setPIN) with OAEP encryption (April 2025)	137
	Appendix I – Recurring transaction types.	139
	Appendix J – Transaction codes.....	139
	Appendix K – Message Class.....	140
	Appendix L – Message Function	140
	Appendix M – Function Code.....	140

2 Revision History

Date	Document version	Reason	Author
03.12.2020	1.0	First version	Arina Poļāčonoka, Rinalds Vizulis
22.12.2020	1.1	Service descriptions added for all services. Services separated into 2 chapters - Issuer Services and Transaction Services. New service added: miss/resetCardPINtries. New attributes added in services: miss/generatePIN, miss/getCardBasicInfo, miss/getCardAccounts. Field "acqCommission" removed from kernel/authorizeTransaction.	Rinalds Vizulis
12.02.2021	1.2	Added "xapi/" for endpoints. Updated kernel/authorizeTransaction description. Added new response code – D03 - Card not linked to Account. Added http header description and new type – Timestamp. Added new call - miss/setCardAccountPriority	Arina Poļāčonoka, Rinalds Vizulis
14.03.2021	1.3	Removed merchantid from kernel/authorizeTransaction. Services miss/getCardLimits and miss/setCardLimits replaced with miss/getLimits and miss/setLimits. New service miss/removeLimits added. Improved description of services: kernel/authorizeTransaction, miss/getBasicCardInfo, miss/getCardAccounts.	Arina Poļāčonoka, Rinalds Vizulis
25.10.2021	1.4	Added field new field in /getTransactionDetails ectxrefno Added new API code D04-Transaction not found	Arina Poļāčonoka
01.11.2021	1.5	Changed request for kernel/reverseTransaction, kernel/getTransactionStatus, kernel/authorizeTransaction Renamed id to tlid in responses and request fields. Updated URL format and endpoints. Updated getTransactionDetails response fields	Arina Poļāčonoka Rinalds Vizulis
05.11.2021	1.6	Added lkey object in kernel/getTransactionDetails, replaced senderKey,recipietkey to Object. Usage of fields tlid and ecTxRefno changed in transaction related services.	Arina Poļāčonoka Rinalds Vizulis
02.12.2021	1.7	Added new services miss/activateCard, kernel/calculatelssCommission Added new fields and transaction types in kernel/authorizeTransaction Added classificatory values for reasonCode, note	Arina Poļāčonoka Rinalds Vizulis

		about configurable fields and additional changes in Card services	
22.12.2021	1.8	Service xkernel/calculatelssCommission removed. New field "purchrefno" added in - xkernel/authorizeTransaction request - xkernel/getTransactionDetails response	Rinalds Vizulis
29.12.2021	1.9	Following response fields removed from Service xmiss/generatePIN: - lkey - encryptedData - pin Response attribute "encryptedData" converted into object with 2 attributes - sessionDESKey and data. Chapter Data encryption added.	Rinalds Vizulis
10.01.2022	1.9.1	Changed encoding in xmiss/setPIN call: - PIN block is HEX encoded. - Cryptogram is HEX encoded. Changed encoding in common fields rsaKeyBlock and encryptedData - now attributes of these fields are HEX encoded. Added new response code D/05 - Lkey not found	Arina Poļáčonoka Rinalds Vizulis
17.01.2022	1.9.2	Added examples for some calls: - xkernel/getLkey - xmiss/setCardStatus - xmiss/generatePIN - xmiss/setPIN - xmiss/setCardAccountPriority - xmiss/getCVV2 Changed txnType length and type code New fields added in xkernel/authorizeTransaction request: - senderfirstName - senderlastName - senderAccount - recipientCity - recipientStreet - recipientfirstName - recipientlastName - recipientAccount - senderFundsSrc - businessAppld	Arina Poļáčonoka
09.02.2022	1.9.3	Added/updated examples for some calls: - xmiss/getCardInfo - xmiss/resetCardPINTries - xmiss/activateCard - xmiss/addAuthRestrictions - xmiss/removeAuthRestrictions - xmiss/getAuthRestrictions Added new search fields - xkernel/getTransactionDetails request	Arina Poļáčonoka

28.02.2022	1.9.4	Added new Request fields in xkernel/getCardTransactionHistory Change cardAccounts and cardAuthRestrictions in xmiss/getCardInfo Response to Array	Arina Poļáčonoka
02.03.2022	1.9.5	Added new field accountCurrCode in such calls: - xmiss/getLimits - xmiss/setLimits - xmiss/removeLimits Provided example for - xmiss/setCVV2Type Changed length for field ecTxRefno from 64 to 150	Arina Poļáčonoka
08.04.2022	1.9.6	Provided example for - xkernel/initiateTransaction - xkernel/authorizeTransaction - xkernel/reverseTransaction	Arina Poļáčonoka
13.04.2022	1.9.7	Updated examples for: - xkernel/getLkey - xmiss/getCVV2 - xkernel/getTransactionDetails - xkernel/getCardTransactionHistory - xmiss/getCardInfo	Arina Poļáčonoka
06.05.2022	1.9.8	Make fields cardkey, senderAccount, recipientCardKey conditional in xkernel/authorizeTransaction call Updated examples for: - xkernel/GetTransactionStatus - xkernel/authorizeTransaction	Arina Poļáčonoka
25.05.2022	1.9.9	Changed request fields optionality to mandatory for: - xmiss/setLimits - xmiss/removeLimits - xmiss/addAuthRestrictions - xmiss/removeAuthRestrictions	Arina Poļáčonoka
11.10.2022	1.10	Added new Transaction status code	Arina Poļáčonoka
19.10.2022	1.10.1	Renamed object name in xkernel/calculateAcqCommission call	Arina Poļáčonoka
26.10.2022	1.10.2	Make cvv2 field optional in xkernel/authorizeTransaction Added Appendix D – Money Transfer Data – Business Application Identifier Added Appendix E – Money Transfer Data – Funding source	Arina Poļáčonoka
22.11.2022	1.11	Improved description of encryption processes. Added description of setPIN. Added new chapter "Appendix G – Encryption Examples" New API codes added: C/42 - Clear PAN not allowed in request	Arina Poļáčonoka Rinalds Vizulis

		C/43 - CVV2 not allowed in request	
28.12.2022	1.12	For service xkernel/calculateAcqCommission: - Corrected fields: totalCommAmt, commCurrency - Added field: ectxrefno Added example in chapter 8	Rinalds Vizulis
10.02.2023	1.13	Updated request/response examples	Sabīne Ciemiņa
14.04.2023	1.14	In getLkey service added mark that expiryDate is not validated. Added Reason codes. Added field enabled to authRestriction services Changed xmiss/removeAuthRestrictions fields priority, actFrom and actTo to optional. Added new Business Application Identifier values to Appendix 12 – OGG, GPO Added new condition for Business Application Identifier and values to Appendix 12 – OGG, GPO	Sabīne Ciemiņa
28.02.2023	1.14	Added Reason codes.	Sabīne Ciemiņa
16.03.2023	1.14	Added field enabled to authRestriction services	Sabīne Ciemiņa
21.03.2023	1.14	Changed xmiss/removeAuthRestrictions fields priority, actFrom and actTo to optional.	Sabīne Ciemiņa
30.03.2023	1.14	Added new Business Application Identifier values to Appendix 12 – OGG, GPO	Sabīne Ciemiņa
14.04.2023	1.14	Added new condition for Business Application Identifier and values to Appendix 12 – OGG, GPO	Sabīne Ciemiņa
22.05.2023	1.15	Added balance fields to xkernel/getCardTransactionHistory response. Added companyRegNumber to getLimits response Added to removeLimits request mandatory field currCode. Added field replace to xmiss/addAuthRestrictions request. Added field deletedRecord to xmiss/removeLimits, xmiss/removeAuthRestrictions response.	Sabīne Ciemiņa
24.05.2023	1.16	Added field replace to xmiss/setLimits request. Updated replace description for xmiss/addAuthRestrictions. Updated xmiss/removeLimits, xmiss/removeAuthRestrictions, xmiss/setLimits, xmiss/removeLimits examples	Sabīne Ciemiņa
27.03.2023	1.17	Added total count to paging response. Added new services: - xmiss/getCustomerCards; - xmiss/getCardNotifications; - xkernel/mdi; - xmiss/mdi. Added new to: - xmiss/getCardInfo request field reqCardNotifications and block cardNotifications in response;	Sabīne Ciemiņa

		- xmiss/getCardAccounts response fields blkAmtCr and blkAmtDr; - xmiss/setPin request field pinBlockType. Updated examples in section Examples. Added Appendix G - PIN block formats. Added balance fields to xkernel/getCardTransactionHistory response. Added curbill, amtbill, amtbillcb and ratebill fields to xkernel/getCardTransactionHistory response. Added field originalEcTxRefno to xkernel/authorizeTransaction. Updated field tstamp_insert, when_created format specifications to GetCardTransactionHistory.	
11.07.2023	1.18	Added hceCardRef to xmiss/getCardBasicInfo.	Sabīne Ciemiņa
19.07.2023	1.19	New service added: xmiss/updateAccountBalance	Sabīne Ciemiņa
22.08.2023	1.20	Added to xmiss/getLimits value -1 description for statistics fields. Added fields amtBillBal and amtBillCred to xkernel/getTransactionDetails response. Added new error code 1/30 Invalid account. Updated xkernel/authorizeTransaction notes.	Sabīne Ciemiņa
8.09.2023	1.21	Changed Invalid account code from 1/30 to D/07. Changed expiryDate in xkernel/initiateTransaction request to optional. Moved encryptData object under Data object in general response structure description. Added Response field top object descriptions. Added DELTA request example for miss/updateAccountBalance.	Sabīne Ciemiņa
9.10.2023	1.22	In xkernel/authorizeTransaction added note that expiryDate for REFUND is optional.	Sabīne Ciemiņa
1.11.2023	1.23	Added API Action code description: 32-41	Natalja Vasijeva
23.11.2023	1.24	Added fields to : - xmiss/activateCard new request field deliveryApprovalCode. - xkernel/reverseTransaction new request field txnCurrency. - xmiss/setCardStatus new request field forced.	Sabīne Ciemiņa
30.11.2023	1.25	Changed in xmiss/getCardBasicInfo response field expiryDate format to YYYYMMDD	Sabīne Ciemiņa
6.12.2023	1.26	Changed field originalEcTxRefno to mandatory for SALES_COM transactions in kernel/authorizeTransaction. Preauthorization and completion request/response examples added to Section 10 Examples. Added 8.5.3 Preauthorization flow.	Sabīne Ciemiņa
18.12.2023	1.27	Added to xkernel/getLkey request field mode.	Sabīne Ciemiņa

10.01.2024	1.28	Added additional 3DS fields to authorizeTransaction: - 3dsProgramProtocol; - 3dsDsTransactionId. Added service xmiss/g2bHceProxy and example. Added to service xkernel/getTransactionStatus new request parameter tllid.	Sabīne Ciemiņa
19.01.2024	1.29	Updated idorg description in xkernel/getTransactionDetails response.	Sabīne Ciemiņa
06.02.2024	1.30	Added to xkernel/getTransactionDetails response field isscountry. Changed naming for fields in xmiss/updateAccountBalance. Added field currCode under in limits Object[]. New request attribute replace in xmiss/setLimits, xmiss/addAuthRestrictions. New response attribute deletedRecords in xmiss/removeLimits, xmiss/removeAuthRestrictions. Reverted changes from v1.29. Added to xkernel/authorizeTransaction new request parameter tavv.	Sabīne Ciemiņa
07.02.2024	1.31	Added to Request and response formats new response field status.sessionId. Increased tavv field length to symbols.	Sabīne Ciemiņa
26.02.2024	1.32	Adden new fields to authorizeTransaction request: senderMiddleName, senderlocState, senderlocPostcode, senderPhone, senderDateOfBirth, senderIdType, senderIdNum, senderIdCountry, senderIdExpdate, senderNationality, senderCountryOfBirth, transferAdditionalMessage, senderParticipationId, transferPurpose, transferClaimCode, recipientMiddleName, recipientState, recipientPostcode, recipientPhone, recipientDateOfBirth, recipientIdType, recipientIdNum, recipientIdCountry, recipientIdExpdate, recipientNationality, recipientCountryOfBirth. Changed txnCurrency to Optional in xkernel/reverseTransaction. Created new common objects TransactionGroup, TransactionBasic. Added to transactionResponse object attribute transactionGroups.	Sabīne Ciemiņa
08.03.2024	1.33	Added attribute transactionGroups to kernel/getTransactionDetails response.	Sabīne Ciemiņa
26.03.2024	1.34	Changed Transacton group object groupType 1 to RFU.	Sabīne Ciemiņa
8.04.2024	1.35	Added to TransactionBasic object fields amtTxnFee and curTxnFee.	Sabīne Ciemiņa
04.12.2024	1.36	Added to authorizeTransaction request: new field mcPaymentGatewayId	Natalja Vasiļjeva

26.04.2024	1.37	Some corrections: "Data encryption - encryptedData in response" numeration change to 5.1, and "Data encryption - PIN data in request* numeration change to 5.2.	Natalja Vasiljeva
03.07.2024	1.38	Appendix A – Response Codes: Two missing Response codes added: 1/75, 1/76	Mikhail Dashuk
10.07.2024	1.39	Added more detailed status code description in "Appendix B - Transaction Status Codes", including list of final status codes. Added more detailed description of action and response codes under chapter "5.2. Response".	Rinalds Vizulis
17.07.2024	1.40	New field merchantComment in AuthorizeTransaction and GetTransactionDetails	Natalja Vasiljeva
29.07.2024	1.41	Restored missing appendix "Appendix H – Encryption Examples"	Rinalds Vizulis
16.08.2024	1.42	Changes in xmiss/updateAccountBalance: - Bugfix in response - fixed attribute name "newBalance" (previously there were 2 attributes "oldBalance"). - Attributes "crlimit, balincr, balincrexp" changed from optional to mandatory. Actually these attributes are always present in response. - Introduced new service mode for multiple account update.	Rinalds Vizulis
23.10.2024	1.43	- Changes in xmiss/getCardAccounts Added new field tstampBalance - Addev new Transaction group type -6	Natalja Vasiljeva
23.10.2024	1.44	Updated "Appendix B - Transaction Status Codes"	Rinalds Vizulis
31.10.2024	1.45	Added field "txStatus" in response of services: - xkernel/authorizeTransaction - xkernel/reverseTransaction	Rinalds Vizulis
11.11.2024	1.46	- Added new service xmiss/verifyPIN - Changed pinBlock field in Requist in xmiss/setPIN from O to M	Natalja Vasiljeva
21.11.2024	1.47	renamed xkernel/initiateTransaction response field to "ectxrefno"	Natalja Vasiljeva
10.12.2024	1.48	Service /xmiss/updateAccountBalance split into 2 services: /xmiss/updateAccountBalance (removing multiple account update) /xmiss/batchUpdateAccountBalances (new service for multiple account update) Changes in both services comparing to previous service version: - In Response object added new conditions under which response fields may be empty. Changes in /xmiss/batchUpdateAccountBalances comparing to multiple account update mode in	Rinalds Vizulis

		single service version: - New fields added in response array of accounts: - code - rspcode - message	
17.12.2024	1.48	Added new values and description for xkernel/authorizeTransaction fields: senderIdType, recipientIdType	Natalja Vasiljeva
14.01.2025	1.49	Added new fields for Service: xmiss/getCardBasicInfo: design, priv1, priv2, priv3, priv4, priv5	Natalja Vasiljeva
21.01.2025	1.50	Description of transaction status codes 4 and 5 updated in "Appendix B - Transaction Status Codes".	Rinalds Vizulis
28.01.2025	1.51	Type of Terminal Sequence Number field extended to support max value 999999	Victor Makarov
02.02.2025	1.52	Definition for data type Number (precision, scale) changed to Number (integer, fraction).	Natalja Vasiljeva
11.02.2025	1.53	Added field Approval Code for xkernel/authorizeTransaction	Natalja Vasiljeva
12.03.2025	1.54	Updated misleading description of service xmiss/generatePIN. In current implementation generated PIN is not returned in response of this service!	Rinalds Vizulis
02.04.2025	1.55	Updated xmiss/resetCardPINtries – added card status handling and EMV script.	Alex Butikov
10.04.2025	1.56	- New field in CardBasicInfo Object, service "xmiss/getCardBasicInfo" - New service "xmiss/resetCardCVV2Counter". - Additional examples for setPIN XAPI testing with OAEP RSA encryption	Natalja Vasiljeva Igors Geletko
12.05.2025	1.57	New field 'deliveryMethod' in 'xmiss/getCardBasicInfo' service	Eriks Kruze
12.05.2025	1.57	Condition C ¹ of 'xkernel/authorizeTransaction' extended with 'senderfirstName' and 'senderlastName'	Mikhail Dashuk
13.06.2025	1.58	Removed misleading example of non-existing service xmiss/getCustomerCards. Updated merchant commission workflow description in service xkernel/authorizeTransaction.	Rinalds Vizulis
16.06.2025	1.59	Fixed field formats in order to match implementation of xkernel/authorizeTransaction: - senderIdType - recipientState Added more detailed description for fields of xkernel/authorizeTransaction: - senderCountryOfBirth - transferPurpose	Rinalds Vizulis

		recipientCountryOfBirth	
18.07.2025	1.60	Added "amtTip" and "curTip" fields for "xkernel/getTransactionDetails" response	Madars Vitolins
31.07.2025	1.61	'Appendix D Money Transfer Data - Business Application Identifier' updated	Mikhail Dashuk
05.08.2025	1.62	'Appendix D Money Transfer Data - Business Application Identifier' updated: TCR, TCP, TFD, TVA	Mikhail Dashuk
01.09.2025	1.63	Added 'additionalData' field to xkernel/authorizeTransaction request	Alex Butikov
05.09.2025	1.64	Services: - xkernel/authorizeTransaction, - xkernel/getTransactionDetails Added field Field: "mcMac", string, 2, Mastercard Merchant Advice Code (DE47.84).	Natalja Vasiljeva
09.09.2025	1.65	For service xkernel/getTransactionDetails additional key attribute defined in request: ecTxRefno and added new field in response trnOrg.	Natalja Vasiljeva
16.09.2025	1.66	Added new fields for acquirer commission in service xkernel/authorizeTransaction: - acqCommSource - acqCommDefAmtFix - acqCommDefPercents - acqCommDefAmtMin - acqCommDefAmtMax Added chapter "8.5.1 Acquirer commission" describing Acquirer commission usage in service xkernel/authorizeTransaction.	Rinalds Vizulis
18.09.2025	1.67	Appendix A. Response code - added 165 and 085 response code.	Natalja Vasiljeva
13.10.2025	1.68	Updated description and C2 condition for field 'businessAppld'.	Mikhail Dashuk
28.10.2025	1.69	New fields in xkernel/authorizeTransaction: - recurrInd - recurrType New Appendix I - "Recurring transaction types".	Natalja Vasiljeva

05.11.2025	1.70	New fields in xkernel/authorizeTransaction and xkernel/getTransactionDetails: - sourceAccountType - destinationAccountType	Mikhail Dashuk
12.11.2025	1.71	Added new fields for issuer commission in service xkernel/authorizeTransaction: - issCommSource - issCommAmount - issCommDefAmtFix - issCommDefPercents - issCommDefAmtMin - issCommDefAmtMax	Rinalds Vizulis
12.11.2025	1.72	Some format error corrected (Appendix name)	Natalja Vasljeva
11.12.2025	1.73	Added new xkernel functions: - getMerchant - getMerchantList Added new functions to xkernel MDI: - Add merchant - Update merchant - Delete merchant - Set merchant address - Delete merchant address	Alex Butikov
15.12.2025	1.74	Added new field (DeliveryTrackingNumber) for the xmiss/getCardBasicInfo.	Eriks Kruze
17.12.2025	1.75	Added new xpos functions: - getPos - getPosList Added new functions to xpos MDI: - Add POS - Update POS - Delete POS - Set POS address - Delete POS address	Alex Butikov
19.01.2026	1.76	Added new xkernel function: - getCompanyList Added new xpos function: - getPosTypeList Added new fields in getMerchant response:	Alex Butikov

		• tipMerchantId • tipTerminalId • schemes Added new functions to xkernel MDI: • Set merchant scheme • Delete merchant scheme	
17.03.2026	1.77	Corrected getCardAccounts: field tstampBalance must be inside object cardAccounts. Different values for each account.	Natalja Vasljeva
10.04.2026	1.77	Changes in response of following services required: • xkernel/initiateTransaction • xkernel/calculateAcqCommission • xkernel/getCardTransactionHistory Changes: • Attribute "ectxrefno" renamed to "ecTxRefno" Note. For backward compatibility reasons attribute name may be changed to "ectxrefno" at configuration level. Please contact system administrators for more information	Natalja Vasljeva
10.06.2026	1.78	Defined new global objects: • CustomerKey • AccountKey. New services: • xmiss/getCustomerBasicInfo • xmiss/getCustomerAccountList • xmiss/getCustomerCardList • xmiss/getCustomerCardAccountLinkageList • xmiss/getAccountBasicInfo • xmiss/getAccountCardList • xmiss/getAccountLimitList • xmiss/getAccountStatement Changes in document formatting, without having actual changes in API: • xmiss/getCardAccounts: CardAccount object moved to separate chapter.	Rinalds Vizulis

		- xmiss/getCardBasicInfo: CardBasicInfo object specification moved to separate chapter. - xmiss/getCardInfo: Specified format "Array" for response attributes which are arrays, previously there was format "Object" used. - xmiss/getLimits: LimitGet object specification moved to separate chapter.	
03.07.2026	1.79	Added new TTI values: P72, P76	Natalja Vasiljeva
09.07.2026	1.80	Service: xkernel/authorizeTransaction changed TRANSF_CASH2C to TRANSF_H2C	Natalja Vasiljeva

3 About document

The purpose of this document is to define initial design of G2B Processing External API.

This document should be read by D8 G2B development team members and G2B customer team members.

4 Protocol

4.1 Connection

Https protocol used for connections between client and server.

Client certificate is used for authentication.

4.2 Http method

POST method used always!

4.3 Http headers

Following Http headers must be sent in requests:

- Pragma: no-cache
- Cache-Control: no-cache
- Content-Type: application/json

4.4 Data format

JSON format used for Requests and Responses.

4.5 URL format

API URLs are built using following format:

`https://<host>[:<port>][<context-root>]/<module>/<version>/<endpoint>`

where:

- host, port - hostname and port of the server (nginx)
- context-root - Optional prefix which will be used for routing purposes by nginx and will not be sent to backend services. For example "/xapi"
- <module> - module name. For example "xmiss" or "xkernel"
- <version> - API version, for example "1.0"
- <endpoint> - Service name value as specified in this design document.

Example with context root "myBank":

`http://myhost/myBank/xapi/xmiss/1.0/getCardInfo`

4.6 Labels and formats

Following labels will be used in this document:

- Column "Mand/Opt" or "M/O":
 - M - Mandatory;

- o O - Optional.
- o C - Conditional
- Column "Rest" (Field restrictions):
 - o empty - field is not restricted;
 - o R - field may be restricted.

4.7 Field Restriction Control

It is possible to restrict response fields at Client configuration level. Therefore, even if Response field is marked as Mandatory it does not mean that it will be always present in all responses. If Mandatory field is restricted, it will not be present in Response.

4.8 Types description

4.8.1 Date

The date data type is used to specify a date; values are represented as JSON Strings in following format – "YYYYMMDD".

4.8.2 Time

The time data type is used to specify a time; values are represented as JSON Strings in following format – "HHMMSS".

4.8.3 Timestamp

Timestamp data type is used to specify data and time; values are represented as JSON Strings in following format – "YYYYMMDDHHMMSS".

4.8.4 String

JSON String value.

4.8.5 Numeric

Numeric value. Include integers and floating-point numbers. Negative and positive numbers are supported.

Parameters:

- integer - The maximum number of digits allowed before the decimal separator. In case of negative number "-" sign is included in length of integer part as digit!
- fraction - The maximum number of digits allowed after the decimal separator.

Decimal separator: "." (dot).

4.8.6 Boolean

JSON Boolean value (true/false).

5 Request and Response formats

5.1 Request

Common structure of request objects:

```
{
  <service-attributes>,
  "paging":
```

```
{
  "size":n
  "lastRetrievedId":n  },
  "rsaKeyBlock":
  {
    "publicKey":"...",
    "publicKeyDigest":"..."
  },
  "ordering": [
    {
      "column": "...",
      "order_function": n
    }
  ],
  "filters": [
    {
      "column": "...",
      "values": [
        "...",
      ],
      "negation_operator": n,
      "logical_operator": n,
      "comparison_operator": n
    },
  ],
}
```

or:

```
{
  <service-attributes>,
  "pagingAdvanced":
  {
    "size":n
    "page":n
  },
  "rsaKeyBlock":
  {
    "publicKey":"...",
    "publicKeyDigest":"..."
  },
  "ordering": [
    {
      "column": "...",
      "order_function": n
    }
  ],
  "filters": [
    {
      "column": "...",
      "values": [
        "...",
      ],
      "negation_operator": n,
      "logical_operator": n,
      "comparison_operator": n
    },
  ],
}
```

where:

- <service-attributes> - service specific attributes, described in specification of each service.

- **paging** - Paging information. May be used only if it is specified that service supports paging option:
 - **size** - number of records to return in response. Maximum allowed page size is limited at system configuration. If size value in request exceeds maximum allowed size, then only maximum allowed number of records will be returned in this page.
 - **lastRetrievedId** - ID of last retrieved record. Value -1 must be used in case of 1st page.
- **pagingAdvanced** – Another version of paging information. May be used only if it is specified that service supports advanced paging option. Cannot be used together with “paging” option.
 - **size** - number of records to return in response. Maximum allowed page size is limited at system configuration. If size value in request exceeds maximum allowed size, then only maximum allowed number of records will be returned in this page.
 - **page** – number of page to return in response.
- **rsaKeyBlock** - Optional RSA key information. May be used in services that support encrypted data in response.
 - **publicKey** - Public key value. KLK 2048-bit Public key in PKCS#1 format. Encoding: HEX.
 - **publicKeyDigest** - Public key digest. PKCS#1 digest, generated with SHA-256 algorithm. Encoding: HEX
- **ordering** – Ordering instructions for functions that return list of objects. Must be supported by operation and number of ordering fields is limited. Element is array of ordering objects. Each ordering object corresponds to one ordering field and has following data elements:
 - **column** – name of the ordering column.
 - **order_function** – ordering direction, where 0 – ASC, 1 – DESC
- **filtering** – Filtering instructions for functions that return list of objects. Must be supported by operation and number of filtering fields is limited. Element is array of filtering objects. Each filtering object corresponds to one filtering field and has following data elements:
 - **column** – name of the filtering column.
 - **values** – array of one or more values required for filtering operations. Number and type of values depends on type of comparison operation and type of the filtering column. E.g. if column must be equal to some value then this value must be present in this array. If filtering operation uses range then array must contain first and last value of the range.
 - **negation_operator** – flag that can negate the condition, where 0 – do not apply negation, 1 – use negation for this column. Negation means that NOT logical operator is applied for this condition.
 - **logical_operator** – logical operator that will be used to combine this condition with the rest of conditions, where 0 – AND operator, 1 – OR operator.
 - **comparison_operator** – comparison operator for this column and value. Following table contains all supported comparison operators:

Value	Operator	Description
0	=	Column is equal a single value. Values array must contain exactly one value for comparison. If value contains wildcard characters '%' or '*' then LIKE operator is forced instead of equals.
1	RANGE	Column is in the range. Values array must contain two values for start and end of the range.
2	IN	Column is in the set of values. Values array must contain all set values.
3	LIKE	Column is like the value. Values array must contain exactly one value for comparison. Value must contain wildcards.
4	LESS	Column is less than a single value in values array.
5	GREATER	Column is greater than a single value in values array.
6	IS NULL	Column is NULL.
7	LESS EQUAL	Column is less or equal than a single value in values array.
8	GREATER EQUAL	Column is greater or equal than a single value in values array.

5.2 Response

Common structure of response objects:

```
{
  "data":
  {
    <service-attributes>,
    "encryptedData":
      "sessionDESKey":"...",
      "data":"..."
  },
  "paging":
  {
    "nextPagePresent":true/false
    "totalCount":n
  }
  "status":
  {
    "code":"x",
    "rspcode":"xx",
    "message":"...",
    "reasonClass":n,
    "reason":n,
    "sessionId":"x"
  }
}
```

Advanced paging response will use following paging section:

```
"pagingAdvanced": {
  "page": 1,
  "size": 5,
  "totalRecordCount": 11
},
```

where:

- data – generic data object:

- <service-attributes> - service specific response attributes, described in specification of each service.
- encryptedData - optional encrypted data block for services which supports encrypted data in response (see more information in chapter "**Error! Reference source not found.**"):
 - sessionDESKey - Session DES key encrypted under RSA key:
 - Data is encrypted with RSA key provided in request attribute rsaKeyBlock. PKCS#1 v1.5 padding is used.
 - Attribute contains HEX encoded cryptogram.
 - data - Sensitive data encrypted with Session DES key:
 - Attribute contains HEX encoded cryptogram.
 - DES3 EDE in ECB mode (no IV) using Double length key (16 bytes) with Odd parity and NULL byte padding.
 - Decrypted data is JSON object. Structure of JSON is described in each particular service where it is supported.
- paging - Optional paging information. Present if Service supports paging information:
 - nextPagePresent - flag indicating if more data is present.
 - totalCount – total record quantity retrieved according to requested parameters. This field is optional and can be skipped in few services.
- pagingAdvanced - Optional paging information. Present if Service supports paging information:
 - page – Response data page number.
 - size – Response data page size.
 - totalRecordCount – total number of database records matched search criteria and company permissions filter.
- status - Mandatory status information:
 - code/rsrcode - Service response code indicating whether request was executed usefully or failed and type of failure. Value 0/00 means successfully executed request. Possible error values must be provided in appendix of API specification.
 - message - Optional human readable error message.
 - reasonClass/reason - Optional reason code indicating more detailed reason of failure. Values must be provided in appendix of API specification.
 - sessionId – Optional XAPI session ID.

5.2.1 Code and Rsrcode

5.2.1.1 Correlation between "status" and Transaction response objects

In every response there is object "status" which contains attributes "code" and "rsrcode". These attributes contain result of web service call. However, there are few cases when these attributes correlate with similar attributes (actionCode, rspCode) in transaction object, these are transaction-related services:

- xkernel/getTransactionDetails
- xkernel/authorizeTransaction
- xkernel/reverseTransaction
- xkernel/getTransactionStatus

In case of transaction-related services main attributes for checking transaction, result is "actionCode", "rspCode" in transaction-related object (transactionResponse, transactionStatus)!

However, there are cases when "actionCode" and "rspCode" may be not present! These are cases when transaction did not reach point where it is either approved or rejected. For example: failure of request validation, transaction is still in the progress or transaction processing crashed. With "crash" here we describe situations when system was not able to finish transaction processing normally because of system error (no disk space, DB server error, etc.).

Table below describes correlation between "status" and transaction-related response objects.

status: code/rspcode	Transaction: actionCode/rspCode	Description
0/00	Not present	Web service call successful, but transaction still in the progress or crashed. Such case is possible when calling getTransactionStatus on transaction in the progress or crashed transaction, which stays in the progress forever.
Not 0/00	Not present	Transaction not created or system error occurred, including such cases • XAPI request validation error. In this case transaction is not created. • System error. In this case transaction might be created or not created depending on the stage in which error occurred.
Present	Present	In case when transaction exist and processing reached stage where transaction is either approved or rejected both pairs of attributes will be present and will contain equal values!

5.2.1.2 Advice processing

It is important to understand that there are 2 different transaction workflows in G2B Processing:

- Request (ISO8583 message function 0) - Real-time transaction request processing. Attributes actionCode/rspCode contains final result of transaction.
- Advice (ISO8583 message function 2) - Asynchronous transaction processing. This means that in real-time transaction is enqueued and it is processed later in the background. In this case real-time response attributes " " contains information regarding enqueue process:
 - 0/00 means that advice is successfully enqueued and will be processed later. This status is not final yet, values may change in case if background processing fails!
 - Not 0/00 means that advice failed before enqueueing for background process. In this case status is final and will not change!

Table below represents processing work depending on XAPI transaction type.

XAPI Transaction type	Processing workflow
Sales Completion, xkernel/authorizeTransaction, txnType = "SALES_COM"	Advice
Reversal, xkernel/reverseTransaction	Advice
All other transactions	Request

5.3 Common Objects

Common Objects may be used in Requests and Responses and will be referenced in service descriptions in API specification.

5.3.1 CardKey

CardKey Object represents key for finding one unique Card in system.

Card may be referenced using different set of attributes. One set must be present always and only one set may be used at one time. Following sets are supported:

- Clear PAN and optional Sequence Number. If Sequence Number not provided then default value 0 is used.

```
{
  "pan": "...",
  "sequence": n,
  "expiryDate": "d"
}
```

- LKEY id. LKEY id is unique id of encrypted PAN value in system.

```
{
  "lkeyId": n
  "expiryDate": "d"
}
```

- LKEY Alias. Unique PAN alias defined in system.

```
{
  "lkeyAlias": "...",
  "expiryDate": "d"
}
```

Expiry is present in all attribute sets. By default field is mandatory. In some cases it is possible to have it optional, such cases will be defined in API specification. If not specified that field expiryDate is optional, then it is mandatory!

Where:

- pan, string(19) - Clear PAN.
- sequence, numeric(5) - Card Sequence number
- expiryDate, string(4) - Card expiry date. Format "YYMM"
- lkeyId, numeric(20) - LKEY Id of PAN.
- lkeyAlias, string(64) - PAN Alias of Card.

5.3.2 Lkey

Lkey Object represents Card PAN in response objects.

```
{
  "maskedPan": "...",
  "pan": "...",
  "sequence": n,
  "lkeyId": n,
  "lkeyAlias": "...",
}
```

}

where:

- maskedPan, string(11) - Masked PAN 6+4.
- pan, string(19) - Clear PAN.
- sequence, numeric(5) - Card Sequence number
- lkeyId, numeric(20) - LKEY Id of PAN.
- lkeyAlias, string(64) - PAN Alias of Card.

Presence of field "pan" must be enabled at User level configuration. This means that field will be present only if Clear PAN is allowed for Customer.

5.3.3 TransactionGroup

TransactionGroup object represents transaction groups related to current Transaction object:

```
"transactionGroups":[
  {
    "groupId":6677
    "groupType":1,
    "transactions":[
      {"tId":16988, ...},
      {"tId":16997, ...}
    ]
  },
  {
    "groupId":6680
    "groupType":4,
    "transactions":[
      {"tId":16988, ...},
      {"tId":16907, ...}
    ]
  }
]
```

where:

- groupId, numeric(20,0) - Transaction Group ID.
- groupType, numeric(5,0) - Transaction Group Type:
 - 1 - Reserved for future use.
 - 2 - Original transaction linkage for reversal.
 - 3 - Original transaction linkage for e-comm refund.
 - 4 - Preauth, incremental, completion.
 - 5 - P2P Acquirer request transaction.
 - 6 - Matched authorisation and clearing transaction.
- Transactions, TransactionBasic[] - Array of transactions in the group.

5.3.4 TransactionBasic

Transaction basic object represents transaction general details in request objects. Used in cases if is needed only basic transaction information.

- tId, numeric(20,0) - Transaction ID in G2B.

- ecTxRefno, string(150) - E-commerce Transaction Reference Number.
- Rrn, string(12) - Retrieval reference number.
- Lkey, LKey - Lkey object.
- dateLocal, date - Actual Local Date of Txn.
- timeLocal, time - Actual Local Time of Txn.
- msgCls, string(1) - Message Class (see Appendix K – Message Class).
- msgFn, string(1) - Message Function (see Appendix L – Message Function). Can contain value 0.
- fnCode, numeric(5,0) - Function Code (see Appendix M – Function Code).
- txnCode, numeric(5,0) - Transaction Code (see Appendix J – Transaction Code).
- subTxnCode, string(2) - Sub-Transaction Code(See - Appendix F– Sub-transaction Code).
- txStatus, numeric(5,0) - Transaction status (see Appendix B - Transaction Status Codes).
- txnAmount, numeric(10,6) - Txn Amount in Acquirer Currency.
- txnCurrency, string(3) - Currency Code of Txn Amount. ISO 4217:2001 numeric codes.
- actionCode, string(1) - Response Action (see Appendix A – Action/Response codes). Can contain value 0.
- rspCode, string(2) - Response Code (see Appendix A – Action/Response codes).
- reasonCode, numeric(5,0) - Reason Code.
- rejReason, numeric(5,0) - Reject Reason Code (see Appendix F - Reject reason).
- aprvlCode, string(9) - Approval Code.
- Afe, string(8) - Acquiring FE.
- lfe, string(8) - Issuing FE.
- curBill, string(3) - Currency Code of Billing Amount. ISO 4217:2001 numeric codes.
- amtBill, numeric(10,6) - Card Holder Billing Amount.
- idOrg, numeric(20,0) - Original transaction ID. Filled with original transaction ID for reversals and PGI refunds.
- purchRefno, string(150) - E-Commerce purchase reference number.
- tstampInsert, string(20) - Record's insertion timestamp (UTC).
- amtTxnFee, numeric(10,6) - Acquirer Commission Amount (value is already included in txnAmount).
- curTxnFee, numeric(10,6) - Acquirer Commission Currency.

5.3.5 CustomerKey

CustomerKey Object represents key for finding one unique Customer in system.

Customer may be referenced using two different set of attributes:

- When using CustomerKey in requests one set must be present and only one set may be used at the time.
- When CustomerKey is used in response messages all sets (all fields) will be present.

Following sets are supported:

- Customer id

```
{
  "id":n
}
```

- Namespace company registration number and Customer code. Here it is important to pass correct company registration number - company that has Name Space Provider defined (Section "Name space provider" under Company screen, table NSPROVIDER).

```
{
  "nsCompanyRegNum": "...",
  "customerCode": "...",
}
```

Where:

- id, numeric(20) - Customer unique id (CUSTOMER.id).
- nsCompanyRegNum, string(64) - Namespace company registration number (COMPANY.reg_number)
- customerCode, string(64) - Customer Code (CUSTOMER.custocode).

5.3.6 AccountKey

AccountKey Object represents key for finding one unique Account in system.

Account may be referenced using two different set of attributes:

- When using AccountKey in requests one set must be present and only one set may be used at the time.
- When AccountKey is used in response messages all sets (all fields) will be present.

Following sets are supported:

- Account id

```
{
  "id": "n
}
```

- Account number and currency.

```
{
  "accountNumber": "...",
  "currency": "...",
}
```

Where:

- id, numeric(20) - Account unique id (ACCOUNT.id).
- accountNumber, string(64) - Account number (ACCOUNT.accnum)
- currency, string(3) - 3 digit ISO 4217:2001 numeric currency code (ACCOUNT.currcode).

5.1 Data encryption - encryptedData in response

5.1.1 Overview

There are services where sensitive data must be returned in response. In this case field "encryptedData" is used.

Encryption solution is based on following approach:

- Data is encrypted under Session DES key *SESS-des*
- Session DES Key *SESS-des* is encrypted under client generated RSA public key *SESS-pub*
- Digest of Client generated RSA public key *SESS-pub* is generated with client generated RSA private key *SIG-priv*.
- Client generated public key *SIG-pub* is loaded into G2B system at configuration stage and is used for *SESS-pub* validation at the time of each request.

See examples in chapter "Appendix H – Encryption Examples".

5.1.2 Configuration

Configuration is required once per WS client initial setup and may be repeated in case when key changes are required:

1. Client generates pair of RSA keys: *SIG-pub*, *SIG-priv*

Due to limitations of hardware security module, length of public key must be no longer than 2048 bits!

2. Security officer of Client sends *SIG-pub* to G2B using trusted channel:

Format: DER encoded in ASN.1 format (sequence of modulus, exponent)

3. Security officer of G2B imports *SIG-pub* into G2B database.

5.1.3 Online WS call

Procedure below describes required action for each Web Service call:

1. Client (preparing request):

- a. Generates new pair of RSA keys: *SESS-pub*, *SESS-priv*

Due to limitations of hardware security module, length of public key must be no longer than 2048 bits!

- b. Generates digest of *SESS-pub* digitally signed with *SIG-priv*

Identifier of the pad mode used in certificate signature generation - PKCS#1 v1.5 method (EMSA-PKCS1-v1_5) with SHA-256 algorithm.

- c. Sends *SESS-pub* and digest in request field "rsaKeyBlock"

HEX encoded values

2. G2B Processing server:

- a. Validates digest of *SESS-pub* using *SIG-pub* from database.
- b. Generates new DES key - *SESS-des*.

Double length 3DES key (16 bytes) with Odd parity.

- c. Encrypts *SESS-des* with *SESS-pub* and provides cryptogram in response field "encryptedData.sessionDESKey"
- d. Encrypts sensitive data with *SESS-des* and provides cryptogram in response field "encryptedData.data".

DES3 EDE in ECB mode (no IV).
Double length key (16 bytes) with Odd parity.
Null byte padding.

3. Client (parsing response):

- a. Decrypts value of "encryptedData.sessionDESKey" using RSA key *SESS-priv*. Result is clear DES key *SESS-des*.
- b. Decrypts value of " encryptedData.data" using DES key *SESS-des*. Result is clear JSON content of encrypted data. For more details see specification of particular Web Service.

Value is HEX encoded.
DES3 EDE in ECB mode (no IV).
Double length key (16 bytes) with Odd parity.
Null byte padding.

5.2 Data encryption - PIN data in request

5.2.1 Overview

There is service xmiss/setPIN where PIN value must be sent in request.

Encryption solution is based on following approach:

- PIN data is formatted as PIN block *PIN-block*.
- *PIN-block* is encrypted under session DES key *SESS-des*.
- *SESS-des* is encrypted under public RSA key of G2B Processing *XAPI-pub*.

See examples in chapter "Appendix H – Encryption Examples".

5.2.2 Configuration

Configuration is required once per XAPI module setup and may be repeated in case when key changes are required:

1. Security officer of G2B generates pair of RSA keys: *XAPI-pub*, *XAPI-priv*

Due to limitations of hardware security module, length of public key is restricted to 2048 bits!

2. Security officer of G2B sends *XAPI-pub* to Client using trusted channel:

Format: DER encoded in ASN.1 format (sequence of modulus, exponent)

3. Security officer of Client imports *XAPI-pub* into Client's infrastructure.

5.2.3 Online WS call

Procedure below describes required action for each Web Service call:

1. Client (preparing request):
 - a. Generates PIN block *PIN-block*:

Format: ANSI X9.8, also known as ISO 9564-1 Format 0
 - b. Generates new DES key *SESS-des*:

Double length 3DES key (16 bytes) with Odd parity.
 - c. Encrypts *PIN-block* with *SESS-des* and sends cryptogram in request field "pinBlock":

DES3 EDE in ECB mode (no IV).
Double length key (16 bytes) with Odd parity.
No padding!!!
"pinBlock" contains HEX encoded value of cryptogram.

- d. Encrypts *SESS-des* with *XAPI-pub* and sends cryptogram in request field "pinKeyUnderRSA".

"pinKeyUnderRSA" contains HEX encoded value of cryptogram.

2. G2B Processing server:
 - i. Decrypts "pinKeyUnderRSA" using *XAPI-priv* in order to get *SESS-des*.
 - ii. Uses "pinBlock" together with *SESS-des* in order to set new PIN value for the Card.

6 Kernel Services

6.1 xkernel/getLkey

Retrieve Clear PAN of Card. PAN is returned encrypted under RSA key.

See chapter "**Error! Reference source not found.**".

6.1.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object. ExpiryDate in cardkey block is not validated.
rsaKeyBlock	Object		O	RsaKeyBlock Object. If present then encryptedData will be present in Response.
mode	Numeric	1,0	O	Lkey service processing mode: • 0 - Standard mode (Default). ○ If PAN found then existing lkeyld value is returned. ○ If PAN not found then new LKey record is created and new value of lkeyld is returned. ○ It is important to understand that requesting service in standard mode with unknown or wrong PAN will always create new record in database! • 1 - Search mode. ○ If PAN found then existing lkeyld value is returned. ○ If PAN not found then lkeyld value -1 is returned. This mode is recommended in all cases when you are looking for existing PAN and do not want to fill database with unnecessary data in case when wrong value is passed in the request.

6.1.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
lkey	Object		M	R	Lkey object

encryptedData	string	350	O	R	Encrypted Data. Format: { "pan": "... " }, where: pan, string(19) - Clear PAN
---------------	--------	-----	---	---	---

6.2 xkernel/getCompanyList

Retrieve companies list based on search criteria.

6.2.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
pagingAdvanced	Object		M	Advanced paging object.
ordering	Array		O	Array of ordering objects. Supported ordering columns: "id" "regNumber" "name" "parentRegNumber"
filters	Array		O	Array of filtering objects. Supported filtering columns: "regNumber" "name" "parentRegNumber"
data	Object		M	Empty object.

6.2.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
companyList	Array		M		Array of CompanyListItem objects

CompanyListItem object:

Field	Format	Length	M/O	Restr	Description
regNumber	String	64	M		Company reg. number
name	String	200	M		Company name
parentRegNumber	String	64	O		Parent company reg. number
parentName	String	200	O		Parent company name

6.3 xkernel/getMerchant

Retrieve merchant information.

6.3.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
merchantId	String	16	M	Merchant identification code.

6.3.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
merchant	Object		M	R	Merchant object

Merchant object:

Field	Format	Length	M/O	Restr	Description
merchantId	String	16	M		Merchant ID code
merchantName	String	25	M		Merchant name
currencyCode	String	3	M		Merchant currency
mcc	Numeric		M		Merchant category code
statusCode	Numeric		M		Merchant status
notes	String	255	O		Notes
accountNum	String	40	O		Account number
parentCompany	String	64	M		Parent company reg.number
geolp4	String	16	O		IP address
merchantType	String	32	M		Merchant type code
dsReqName1	String	40	O		3DS Requestor Name (MC)
dsReqId1	String	35	O		3DS Requestor ID (MC)
dsReqUrl1	String	2048	O		3DS Requestor URL (MC)
dsReqName2	String	40	O		3DS Requestor Name (VISA)
dsReqId2	String	35	O		3DS Requestor ID (VISA)
dsReqUrl2	String	2048	O		3DS Requestor URL (VISA)
tstampInsert	String	20	O		Inserting timestamp
tstampClose	String	20	O		Closing timestamp
countryOrg	String	3	O		Country of Origin
taxCode	String	64	O		Tax code or ID
partnerCode	String	64	O		Partner code
tipMerchantId	String	16	O		Tip merchant ID
tipTerminalId	String	16	O		Tip terminal ID
govControl	Char	1	M		Government control flag
addresses	Array		M		Array of Address objects
schemes	Array		M		Array of Scheme objects

Merchant address object:

Field	Format	Length	M/O	Restr	Description
-------	--------	--------	-----	-------	-------------

addressType	Numeric		M		Address type: 2 - Legal address 3 - Physical address 4 - Postal address
locationName	String	25	O		Location name
address1	String	128	O		Address line 1
address2	String	128	O		Address line 2
city	String	40	O		City name
region	String	3	O		Region code
postalCode	String	40	O		ZIP/postal Code
country	String	3	O		Country code (numeric)
contactPerson1	String	128	O		Contact person 1
contactPerson2	String	128	O		Contact person 2
contactPerson3	String	128	O		Contact person 3
contactRole1	String	64	O		Contact person 1 role
contactRole2	String	64	O		Contact person 2 role
contactRole3	String	64	O		Contact person 3 role
contactPhone1	String	64	O		Contact person 1 phone
contactPhone2	String	64	O		Contact person 2 phone
contactPhone3	String	64	O		Contact person 3 phone
contactEmail1	String	254	O		Contact person 1 email
contactEmail2	String	254	O		Contact person 2 email
contactEmail3	String	254	O		Contact person 3 email
url	String	255	O		URL address
additionalInfo	String	255	O		Additional info

Merchant scheme object:

Field	Format	Length	M/O	Restr	Description
scheme	String	12	M		Scheme
field1	String	64	O		Scheme field 1
value1	String	16	O		Field value 1
field2	String	64	O		Scheme field 2
value2	String	16	O		Field value 2
field3	String	64	O		Scheme field 3
value3	String	16	O		Field value 3
field4	String	64	O		Scheme field 4
value4	String	16	O		Field value 4
field5	String	64	O		Scheme field 5
value5	String	16	O		Field value 5
field6	String	64	O		Scheme field 6
value6	String	16	O		Field value 6

6.4 xkernel/getMerchantList

Retrieve merchant list based on search criteria.

6.4.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
pagingAdvanced	Object		M	Advanced paging object.
ordering	Array		O	Array of ordering objects. Supported ordering columns: - "id"
filters	Array		O	Array of filtering objects. Supported filtering columns: - "merchantId" - "merchantName" - "statusCode"
data	Object		M	Empty object.

6.4.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
merchantList	Array		M		Array of MerchantListItem objects

MerchantListItem object:

Field	Format	Length	M/O	Restr	Description
merchantId	String	16	M		Merchant ID code
merchantName	String	25	M		Merchant name
statusCode	Numeric		M		Merchant status
taxCode	String	64	O		Tax code or ID

7 Issuer Services

7.1 xmiss/getCardInfo

Retrieve full Card information. This is composite service returning 7 data blocks. Each data block is described in its own separate service which may be called directly if one particular data block is required.

In request it is possible to indicate which data block must be returned in response by setting appropriate boolean variables to *true*.

7.1.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
reqCardBasicInfo	Boolean	-	O	If set to true then cardBasicInfo object is included in response.
reqCardAccounts	Boolean	-	O	If set to true then cardAccounts object is included in response.
reqCardLimits	Boolean	-	O	If set to true then cardLimits object is included in response.

reqCardAccountLimits	Boolean	-	0	If set to true then cardAccountLimits object is included in response.
reqCardAuthRestrictions	Boolean	-	0	If set to true then cardAuthRestrictions object is included in response.
reqCardTransactions	Boolean	-	0	If set to true then cardTransactions object is included in response.
cardTransactionCount	numeric	5,0	0	Number of transactions to return. Field used in case when reqCardTransactions=true. If field not present then configurable number of transactions is returned by default (for example, last 10)
reqCardNotifications	Boolean	-	0	If set to true then cardNotifications object is included in response.

7.1.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description		
cardBasicInfo	Object		O	R	CardBasicInfo Object. See details in "xmiss/getCardBasicInfo".		
cardAccounts	Array		O	R	Array of CardAccount Object. See details in "CardBasicInfo Object"		
					Field	Format	Len
					lkey	Object	
					expiryDate	date	
					title	string	10
					firstName	string	50
					lastName	string	50
					embossName	string	51
					deliveryPoint	string	30
					deliveryMethod	numeric	5,0

					statExternalUser	string	64
					statRequester	string	64
					statChangeTime	timestamp	-
					effDate	date	
					cdproduct	string	16
					svccode	string	3
					pvki	string	1
					pvv	string	4
					dcec	numeric	10,0
					miscInfo	string	64
					invalidPinTries	numeric	5,0
					cvv2Type	numeric	5,0
					commissionCategory	string	64
					feeCategory	string	64
					limitCategory	string	64
					productType	numeric	1
					hceCardRef	numeric	20,0
					designImage	string	64
					priv1	string	64
					priv2	string	64
					priv3	string	64
					priv4	string	64
					priv5	string	64
					cvv2tries	number	5,0
					xmiss/getCardAccounts"		
cardLimits	Array		O	R	Array of LimitGet Object for limitTypes values 'CCAT' and 'CCRD'. See details in "xmiss/getLimits"		
cardAccountLimits	Array		O	R	Array of LimitGet Object for limitTypes values 'AACN'. Array contains limits for all accounts linked to card. Account number may be found in field objectID. See details in "xmiss/getLimits"		
cardAuthRestrictions	Array		O	R	Array of RestrGet Object		

					See details in "xmiss/getCardAuthRestrictions"
cardTransactions	Array		O	R	Array of Transaction objects, containing last N transaction. Number of transactions is defined by request field cardTransactionCount or configurable value used if field cardTransactionCount not present. See details in "xkernel/getCardTransactionHistory"
cardNotifications	Array		O	R	Array of cardNotification Objects. See details in "xmiss/getCardNotifications"

7.2 xmiss/getCardBasicInfo

Retrieve Card's basic information.

7.2.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object

7.2.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
cardBasicInfo	Object		M	R	CardBasicInfo Object

7.2.2.1 CardBasicInfo Object

Field	Format	Length	M/O	Restr	Description
lkey	Object		M	R	Lkey object
expiryDate	date		O	R	Expiry date in format YYYYMMDD
title	string	10	O	R	Title
firstName	string	50	M	R	First name of the customer
lastName	string	50	M	R	Last name of the customer
embossName	string	51	O	R	Embossed name
deliveryPoint	string	30	O	R	Delivery point
deliveryMethod	numeric	5,0	O	R	Method for dispatching card. Supported values: 0 - Not specified 1 - Home address 2 - Branch 3 - Direct delivery 4 - Mail
deliveryTrackingNum	string	50	O	R	Delivery tracking number

ber					
customerCode	string	64	M	R	Customer code
fatherName	string	50	O	R	Father's name
maidenName	string	50	O	R	Maiden name
currCode	string	3	O	R	Default card currency code ISO 4217:2001 numeric codes
companyRegNumber	string	64	M	R	Company registration number
statCode	string	2	M	R	Status code (configurable value)
statReason	string	128	O	R	Reason for last status change
statExternalUser	string	64	O	R	Last status change user in external system.
statRequester	string	64	O	R	Last status change requestor (system or process name).
statChangeTime	timestamp	-	O	R	Timestamp of last status change.
effDate	date		O	R	Card effective date
cdproduct	string	16	M	R	Card product
svccode	string	3	O	R	Card service code
pvki	string	1	O	R	PIN Verification Key index
pvv	string	4	O	R	PIN verification value
dcec	numeric	10,0	O	R	Digitized Card Error Counter.
miscInfo	string	64	O	R	MISC data
invalidPinTries	numeric	5,0	O	R	Number of invalid PIN tries.
cvv2Type	numeric	5,0	O		CVV2 Type: 0 - Static 1 - Dynamic
commissionCategory	string	64	O	R	Commission category code
feeCategory	string	64	O	R	Fee category code
limitCategory	string	64	O	R	Limit category code
productType	numeric	1	O	R	Card type flag indicating whether card is virtual or physical. Possible values: 0 - Physical 1 - Virtual
hceCardRef	numeric	20,0	M	R	Card unique identifier used in HCE.
designImage	string	64	O	R	Card personalised image name
priv1	string	64	O	R	Private data 1
priv2	string	64	O	R	Private data 2
priv3	string	64	O	R	Private data 3
priv4	string	64	O	R	Private data 4
priv5	string	64	O	R	Private data 5
cvv2tries	number	5,0	O	R	Number of consecutive CVV2 failures.

7.3 xmiss/getCardAccounts

Retrieve list of Accounts linked to the Card.

7.3.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object

7.3.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Rest r	Description
cardAccounts	Object[]		M	R	Array of CardAccounts Object.

7.3.2.1 CardAccounts Object

Field	Format	Length	M/O	Rest r	Description
accountNumber	string	64	M		Account number
currency	string	3	M	R	Currency code of the account ISO 4217:2001 numeric codes
typeCode	string	2	M	R	Account type code (configurable value)
typeCodeDescr	string	20	M	R	Account type description
customerCode	string	64	M	R	Customer code
avlBal	numeric	10,6	M	R	Available balance.
blkAmt	numeric	10,6	M	R	Blocked amount.
openDate	date		M	R	Account open date
lastUsage	date		M	R	Account last usage date
statCode	string	2	M	R	Account status (configurable value)
priority	number	5	M	R	Card-Account linkage priority
crLimit	number	10,6	O	R	Credit Limit
balincr	number	10,6	O	R	Temporary balance increase
balincrExp	date		O	R	Expiry date of temporary balance increase
blkAmtCr	number	10,6	M	R	Total credit holds
blkAmtDr	number	10,6	M	R	Total debit holds
tstampBalance	string	20	M	R	UTC timestamp of last balance update. Note! Different values for each account.

7.4 xmiss/getCardNotifications

Retrieve list of customer contact data linked to the Card. Contains one record for each contact data.

7.4.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object

7.4.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
cardNotifications	Object[]		M	R	Array of cardNotification Object.
notificationTarget	String	64	O	R	Contact information data – mobile phone, email.
serviceType	string	32	O	R	Notification service type: SMSGEN - Generic SMS notifications SMSTXN - Transaction notifications by SMS

7.5 xmiss/setCardStatus

Set new status for Card. System will check is status change is allowed based on rules defined in Processing System. Restricted status changes will be rejected with code D/02 (Status change transition not allowed).

7.5.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
newStatCode	string	2	M	Card status code (configurable value)
reason	string	128	M	Reason for change
force	Boolean		O	Forced card status. If set to true then status transition check is ignored which means that any transition is allowed. Field is available only in case when special access item (/xapi/miss/forceCardStatus) is granted to the user!
externalUser	string	64	O	Username in external system. <i>For information purposes only. May be used, for example, in case when WS is called from some external application and username of external application user may be sent in this field.</i>

7.5.2 Response

Standard response, no service specific fields.

7.6 xmiss/activateCard

Card activation

7.6.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
reason	string	128	M	Reason for activation
externalUser	string	64	O	Username in external system. <i>For information purposes only. May be used, for example, in case when WS is called from some external application and username of external application user may be sent in this field.</i>
deliveryApprovalCode	string	32	O	Delivery Approval Code

7.6.2 Response

Standard response, no service specific fields.

7.7 xmiss/generatePIN

Generate new PIN value for Card.

Due to security reasons PIN is not returned in the response. If it is required to send new PIN value to some external system or cardholder G2B Processing Notification mechanism should be used.

7.7.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object

7.7.2 Response

Standard response, no service specific fields.

7.8 xmiss/setPIN

Set new PIN value. New value of PIN must be provided in encrypted PIN block format together with encrypted DES encryption key.

7.8.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
pinKeyUnderRSA	string	512	M	DES key used for PIN block. Key value should be encrypted under Public RSA key of processing. PKCS#1 v1.5 padding is used. Cryptogram is HEX encoded.
pinBlock	string	16	O	PIN Block format is defined in field

				pinBlockType. PIN Block must be encrypted using DES Encryption key specified in field pinKeyUnderRSA. Cryptogram must be provided in HEX encoded format.
pinBlockType	numeric	5,0	M	Pin block type: 0 - ANSI X9.8 (Thales Format 01) 1 - ISO 9564-1 Format 1 (Thales Format 05) Default value 0. Please see format description in Appendix G - PIN block formats.

7.8.2 Response

Standard response, no service specific fields.

7.9 xmiss/getCVV2

Retrieve Card's CVV2 value. Dynamic or static CVV2 value may be returned based on Card's settings.

7.9.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
rsaKeyBlock	Object		O	RsaKeyBlock Object. If present then encryptedData will be present in Response.

7.9.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
lkey	Object		M		Lkey Object
cvv2	string	3	O	R	Clear value of CVV2. Present only if clear CVV2 is allowed for Client.
cvv2Type	numeric	5,0	M	R	CVV2 Type: 0 - Static 1 - Dynamic
cvv2Expiry	timestamp		M	R	CVV2 expiry time. <i>For static CVV2 it will be equal to Card Expiry time.</i>
encryptedData	string	350	O	R	Encrypted Data. Format: {"cvv2": "..."}

					where: cvv2, string(3) - Clear value of CVV2
--	--	--	--	--	---

7.10 xmiss/setCVV2Type

Set Card's CVV2 type - dynamic or static.

7.10.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
cvv2Type	numeric	5,0	M	CVV2 Type: 0 - Static 1 - Dynamic

7.10.2 Response

Standard response, no service specific fields.

7.11 xmiss/getLimits

Retrieve Card and Account Limit configuration, including Card group limits.

7.11.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		C ¹	CardKey Object
accountNumber	string	64	C ¹	Account Number
accountCurrCode	string	3	C ¹	Limit currency. ISO 4217:2001 numeric codes
limitCategoryCode	string	32	C ¹	Limit Category Code
limitTypes	string[]		M	Array of Limit Category codes to retrieve: 'CCAT' - Card limit for Limit Category 'CCRD' - Individual Card Limit 'AACN' - Individual Account limit

¹ Request parameters depends on values in limitType array:

- if 'CCAT' present and 'CCRD' not present then field limitCategoryCode must be present
- if 'CCRD' present then field cardKey must be present
 - if CCAT is requested as well then limitCategoryCode will be retrieved from Card data!
- if 'AACN' present then fields accountNumber and accountCurrCode must be present

7.11.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Rest r	Description
-------	--------	--------	-----	-----------	-------------

limits	Object[]		M	R	Array of LimitGet Object
--------	----------	--	---	---	--------------------------

7.11.2.1 LimitGet Object

Field	Format	Length	M/O	Restr	Description
limitCode	string	32	M	R	Unique limit definition code.
limitType	string	4	M	R	Type of limit: 'CCAT' - Card limit for Limit Category 'CCRD' - Individual Card Limit 'AACN' - Individual Account limit
priority	numeric	5	M	R	Limit priority inside one limit type.
objectID	string	64	M	R	Linked Object ID. Value depends on limitType. Based on limit type objectID will contain: 'CCAT' - Limit Category Code 'CCRD' - Card LKEYID 'AACN' - Account Number
actFrom	string	14	O	R	Limit activity start time (YYYYMMDDHHMMSS)
actTo	string	14	O	R	Limit activity end time (YYYYMMDDHHMMSS)
currCode	string	3	M	R	Limit currency. ISO 4217:2001 numeric codes
txnAmtMin	numeric	10,6	O	R	Minimum amount of 1 transaction. Field can contain value -1 that means statistical parameter is not defined.
txnAmtMax	numeric	10,6	O	R	Maximum amount of 1 transaction. Field can contain value -1 that means statistical parameter is not defined.
cycAmtMax	numeric	10,6	O	R	Maximum accumulated amount in cycle. Field can contain value -1 that means statistical parameter is not defined.
cycNumMax	numeric	10,0	O	R	Maximum number of transactions in cycle. Field can contain value -1 that means statistical parameter is not defined.
cycCntAmt	numeric	10,6	O	R	Amount already spent in current cycle. Field can contain value -1 that means statistical parameter is not defined.
cycCntNum	numeric	10,0	O	R	Number of transactions already made in current cycle. Field can contain value -1 that means statistical parameter is not defined.
companyRegNumber	string	64	O	R	Company registration number

7.12 xmiss/setLimits

Set Individual Card or Account Limits configuration. If limit configuration already exists then it is updated, if does not exist then it is added.

LIMITATION: Service will not allow setting Limit Category limits!

7.12.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		C ¹	CardKey Object
accountNumber	string	64	C ¹	Account Number
accountCurrCode	string	3	C ¹	Limit currency. ISO 4217:2001 numeric codes
limits	Object[]			Array of LimitSet
limitCode	string	32	M	Unique limit definition code.
limitType	string	4	M	Type of limit: 'CCRD' - Individual Card Limit 'AACN' - Individual Account Limit
priority	numeric	5	M	Limit priority.
actFrom	string	14	M	Limit activity start time (YYYYMMDDHHMMSS)
actTo	string	14	M	Limit activity end time (YYYYMMDDHHMMSS)
currCode	string	3	M	Limit currency. ISO 4217:2001 numeric codes
txnAmtMin	numeric	10,6	O	Minimum amount of 1 transaction.
txnAmtMax	numeric	10,6	O	Maximum amount of 1 transaction.
cycAmtMax	numeric	10,6	O	Maximum accumulated amount in cycle.
cycNumMax	numeric	10,0	O	Maximum number of transactions in cycle.
replace	boolean	-	O	"If replace" flag set to true then all existing limits/restrictions for current object will be deleted, before inserting new records.

¹ Request parameters depend on values present in LimitSet.limitType:

- if 'CCRD' present then field cardKey must be present
- if 'AACN' present then fields accountNumber and accountCurrCode must be present

7.12.2 Response

Standard response, no service specific fields.

7.13 xmiss/removeLimits

Remove Individual Card or Account Limits configuration.

LIMITATION: Service will not allow removing Card Group limits!

7.13.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		C ¹	CardKey Object

accountNumber	string	64	C ¹	Account Number
accountCurrCode	string	3	C ¹	Limit currency. ISO 4217:2001 numeric codes
limits	Object[]			Array of LimitRemove
limitCode	string	32	M	Unique limit definition code.
limitType	string	4	M	Type of limit: 'CCRD' - Individual Card Limit 'AACN' - Individual Account Limit
currCode	string	3	M	Limit currency. ISO 4217:2001 numeric codes
priority	numeric	5	M	Limit priority.
actFrom	string	14	M	Limit activity start time (YYYYMMDDHHMMSS)
actTo	string	14	M	Limit activity end time (YYYYMMDDHHMMSS)

¹ Request parameters depends on values present in LimitRemove.limitType:

- if 'CCRD' present then field cardKey must be present
- if 'AACN' present then fields accountNumber must be present and accountCurrCode

7.13.2 Response

Field	Format	Length	M/O	Restr	Description
deletedRecords	number	5,0	M	R	Number of records deleted

7.14 xmiss/resetCardPINTRIES

Reset Card PIN Tries Counter to 0. Restore original pre-PTE (PIN tries exceeded) status. Generate PIN Tries Reset (Unblock PIN) issuer script.

Function will restore previous card status if current card status is 10 (PIN tries exceeded) and last record of card status change history contains change from any other status to 10 (PIN tries exceeded). If current status is not 10 or status change history conflicts with current status, then card status is left as is and separate card status change function call is required.

Function will generate EMV issuer script if card is EMV and offline PIN is allowed in card product settings.

7.14.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object

7.14.2 Response

Standard response, no service specific fields.

7.15 xmiss/setCardAccountPriority

Change Card-Account linkage Priority.

If Card is not linked to Account then response code D/03 (Card not linked to Account) is returned.

7.15.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
accountNumber	string	64	M	Account number
currency	string	3	M	Currency code of the account ISO 4217:2001 numeric codes
priority	number	5,0	M	Card-Account linkage priority.

7.15.2 Response

Standard response, no service specific fields.

7.16 xmiss/getAuthRestrictions

Retrieve Card Authorisation Restriction configuration.

7.16.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		C ¹	CardKey Object
restrCategoryCode	string	32	C ¹	Authorisation Restriction Code
restrTypes	string[]		M	Array of Authorisation Restriction Category codes to retrieve: 'CCAT' - Restrictions for Authorisation Restriction Category 'CCRD' - Individual Card Restrictions

¹ Request parameters depends on values in restrType array:

- if 'CCAT' present and 'CCRD' not present then field restrCategoryCode must be present
- if 'CCRD' present then field cardKey must be present
 - if CCAT is requested as well then restrCategoryCode will be retrieved from Card data!

7.16.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
restrictions	Object[]		M	R	Array of RestrGet Object
restrCode	string	32	M	R	Unique restriction definition code.
restrType	string	4	M	R	Type of restriction: 'CCAT' - Restriction for Authorisation Restriction Category 'CCRD' - Individual Card Restriction
priority	numeric	5	M	R	Restriction priority inside one restriction type.
enabled	boolean	1	M	R	Indicates status of AuthRestriction:

					1 – restriction will be applied to the card; 2 – restriction will be skipped.
objectID	string	64	M	R	Linked Object ID. Value depends on restrType . Based on limit type objectID will contain: 'CCAT' - Restriction Category Code 'CCRD' - Card LKEYID
actFrom	string	14	O	R	Restriction activity start time (YYYYMMDDHHMMSS)
actTo	string	14	O	R	Restriction activity end time (YYYYMMDDHHMMSS)

7.17 xmiss/addAuthRestrictions

Add individual Card Authorisation Restriction items.

LIMITATION: Service will not allow setting restrictions for Authorisation Restriction Category!

7.17.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
restrictions	Object[]			Array of RestrictionSet
restrCode	string	32	M	Unique Authorisation Restriction definition code.
restrType	string	4	M	Type of restriction: 'CCRD' - Individual Card Limit
priority	numeric	5	M	Restriction priority.
actFrom	string	14	M	Restriction activity start time (YYYYMMDDHHMMSS)
actTo	string	14	M	Restriction activity end time (YYYYMMDDHHMMSS)
enabled	numeric	1	M	Indicates status of AuthRestriction: 1 – restriction will be applied to the card; 2 – restriction will be skipped.
replace	boolean	-	O	"If replace" flag set to true then all existing limits/restrictions for current object will be deleted, before inserting new records.

7.18 xmiss/removeAuthRestrictions

Remove Individual Card Authorisation Restriction items.

LIMITATION: Service will not allow removing restrictions for Authorisation Restriction Category!

7.18.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
restrictions	Object[]			Array of RestrictionRemove
restrCode	string	32	M	Unique Authorisation Restriction code.
restrType	string	4	M	Type of restriction: 'CCRD' - Individual Card Limit
priority	numeric	5	O	Restriction priority.
actFrom	string	14	O	Restriction activity start time (YYYYMMDDHHMMSS)
actTo	string	14	O	Restriction activity end time (YYYYMMDDHHMMSS)

NOTE: If priority, actFrom and actTo are not specified then all restrictions with given restrCode will be removed for Card.

7.18.2 Response

Field	Format	Length	M/O	Restr	Description
deletedRecords	number	5,0	M	R	Number of records deleted

7.19 xmiss/updateAccountBalance

Update single account balance.

7.19.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
accountNumber	string	64	M	Account number to be updated.
balanceType	string	32	M	Fixed value: "ABSOLUTE" - in case when file contains absolute balances "DELTA" - in case when file contains deltas - balance changes after previous file.
lastTransactionLogTimestamp	string	20	C	UTC Timestamp of latest transaction loaded into host system and applied to balances. Must contain highest value loaded from TL export file field "tstamp_insert". If value present then all transaction processed in G2B Processing after this time will be applied to balance value when loading file. If value empty then transactions will not be applied to balances. Value is required for requests with balanceType=ABSOLUTE. No impact on requests with balanceType=DELTA. Timestamp, YYYYMMDDHHmmSSccccc.
currency	string	3	M	Currency code for account - ISO 3 digit.
statCode	string	2	O	Status of account.

				• If empty then status is not updated!
blkAmt	numeric	10,6	O	Blocked amount. This represents the value of authorisations in transit on the account.
avlBal	numeric	10,6	O	Financial balance of account, excluding blocked amount. <i>Note: The real available balance for spending is calculated as avlbal - blkamt.</i>
crLimit	numeric	10,6	O	Credit Limit
balIncr	numeric	10,6	O	Temporary balance increase
balIncrExp	date		O	Expiry date of temporary balance increase
description	string	100	O	Description
externalTxnID	string	100	O	External transaction ID

7.19.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Res tr	Description
oldBalance	Object		M	R	Balance Object with old balance
avlBal	numeric	10,6	M	R	Old available balance. See condition C ₂ .
blkAmt	numeric	10,6	M	R	Old blocked amount. See condition C ₂ .
crlimit	numeric	10,6	M	R	Old credit limit. See condition C ₂ .
balincr	numeric	10,6	M	R	Old temporary balance increase. See condition C ₂ .
balincexp	date		M	R	Old expiry date of temporary balance increase. See condition C ₂ .
newBalance	Object		M	R	Balance Object with new balance
avlBal	numeric	10,6	M	R	New available balance. See condition C ₂ .
blkAmt	numeric	10,6	M	R	New blocked amount. See condition C ₂ .
crlimit	numeric	10,6	M	R	New credit limit. See condition C ₂ .
balincr	numeric	10,6	M	R	New temporary balance increase. See condition C ₂ .
balincexp	date		M	R	New expiry date of temporary balance increase. See condition C ₂ .
processingTimestamp	string	20	M	R	Processing date/time, Timestamp, YYYYMMDDHHmmSSccccc. See condition C ₁ .
accountLogId	numeric	20,0	M	R	Account log record identifier. See condition C ₁ .

Conditions:

- C₁ - Value present if Account log record created. Contains empty value in case when Account logging not enabled or in case when old balance is equal to new balance and no Account Log record is created.
- C₂ - Values present only in case when Account Logging is enabled. If Account Logging is disabled all values will be set to 0.

7.20 xmiss/batchUpdateAccountBalances

Update account balances of multiple accounts in batch mode.

7.20.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
accounts[]	Object[]		M	Array of account balances
accountNumber	string	64	M	Account number to be updated.
balanceType	string	32	M	Fixed value: “ABSOLUTE” - in case when file contains absolute balances “DELTA” - in case when file contains deltas - balance changes after previous file.
lastTransactionLogTimestamp	string	20	C	UTC Timestamp of latest transaction loaded into host system and applied to balances. Must contain highest value loaded from TL export file field "tstamp_insert". If value present then all transaction processed in G2B Processing after this time will be applied to balance value when loading file. If value empty then transactions will not be applied to balances. Value is required for requests with balanceType=ABSOLUTE. No impact on requests with balanceType=DELTA. Timestamp, YYYYMMDDHHmmSScccccc.
currency	string	3	M	Currency code for account - ISO 3 digit.
statCode	string	2	O	Status of account. If empty then status is not updated!
blkAmt	numeric	10,6	O	Blocked amount. This represents the value of authorisations in transit on the account.
avlBal	numeric	10,6	O	Financial balance of account, excluding blocked amount. <i>Note: The real available balance for spending is calculated as avlbal - blkamt.</i>
crLimit	numeric	10,6	O	Credit Limit
balIncr	numeric	10,6	O	Temporary balance increase
balIncrExp	date		O	Expiry date of temporary balance increase
description	string	100	O	Description
externalTxnID	string	100	O	External transaction ID

7.20.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Res tr	Description
accounts[]	Object[]		M	R	Array of account balances responses
accountNumber	string	64	M	R	Account number to be updated.
currency	string	3	M	R	Currency code for account - ISO 3 digit.
oldBalance	Object		M	R	Balance Object with old balance
avlBal	numeric	10,6	M	R	Old available balance. See condition C ₂ .
blkAmt	numeric	10,6	M	R	Old blocked amount. See condition C ₂ .

crlimit	numeric	10,6	M	R	Old credit limit. See condition C ₂ .
balincr	numeric	10,6	M	R	Old temporary balance increase. See condition C ₂ .
balincrexp	date		M	R	Old expiry date of temporary balance increase. See condition C ₂ .
newBalance	Object		M	R	Balance Object with new balance
avlBal	numeric	10,6	M	R	New available balance. See condition C ₂ .
blkAmt	numeric	10,6	M	R	New blocked amount. See condition C ₂ .
crlimit	numeric	10,6	M	R	New credit limit See condition C ₂ .
balincr	numeric	10,6	M	R	New temporary balance increase. See condition C ₂ .
balincrexp	date		M	R	New expiry date of temporary balance increase. See condition C ₂ .
processingTimestamp	string	20	M	R	Processing date/time, Timestamp, YYYYMMDDHHmmSSccccc. See condition C ₁ .
accountLogId	numeric	20,0	M	R	Account log record identifier. Present if Account log record created. May be missing is Account logging not enabled or in case when old balance is equal to new balance and no Log record is created. See condition C ₁ .
code	string	1	M	R	1 character of service response code indicating whether request was executed usefully or failed and type of failure. See: "Appendix A - Response Codes".
rspcode	string	2	M	R	2-3 characters of service response code indicating whether request was executed usefully or failed and type of failure. See: "Appendix A - Response Codes".
message	string	255	O	R	Optional human readable error message.

Conditions:

- C₁ - Value present if Account log record created. Contains empty value if Account logging not enabled or in case when old balance is equal to new balance and no Account Log record is created.
- C₂ - Values present only in case when Account Logging is enabled. If Account Logging is disabled all values will be set to 0.

Service specific response codes in object "status":

Code/Rspcode	Description
0/00	Approved. All balances updated sucessfully.
C/45	Executed with errors. For details see response codes in "accounts[]" for each particular account.

Service specific response codes in "accounts[]":

Code/Rspcode	Description
0/00	Balance updated
D/07	Invalid account
9/13	Duplicate transaction

7.21 xmiss/resetCardCVV2Counter

Reset Card CVV2 Tries Counter to 0.

7.21.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object

7.21.2 Response

Standard response, no service specific fields.

7.22 xmiss/verifyPIN

Verify PIN value. Value of PIN must be provided in encrypted PIN block format together with encrypted DES encryption key.

7.22.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object
pinKeyUnderRSA	string	512	M	DES key used for PIN block. Key value should be encrypted under Public RSA key of processing. PKCS#1 v1.5 padding is used. Cryptogram is HEX encoded.
pinBlock	string	16	M	PIN Block format is defined in field pinBlockType. PIN Block must be encrypted using DES Encryption key specified in field pinKeyUnderRSA. Cryptogram must be provided in HEX encoded format.
pinBlockType	numeric	5,0	O	Pin block type: 0 - ANSI X9.8 (Thales Format 01) 1 - ISO 9564-1 Format 1 (Thales Format 05) Default value 0. Please see format description in Appendix G - PIN block formats.

7.22.2 Response

Service specific response codes:

Code/Rspcode	Description
0/00	PIN verified successfully
1/00	PIN received when no PIN required
1/06	PIN tries exceeds max number of allowed tries
1/12	PIN is missing
1/17	Bad PIN
1/18	DB data selection / Card configuration issue
1/26	Invalid PIN block
9/09	System error

7.23 xmiss/getCustomerBasicInfo

Retrieve basic information about Customer.

7.23.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
customerKey	Object		M	CustomerKey Object

7.23.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
customerBasicInfo	Object		M	R	CustomerBasicInfo Object

7.23.2.1 CustomerBasicInfo Object

Field	Format	Length	M/O	Restr	Description
customerKey	Object		M	R	CustomerKey Object
companyRegNumber	string	64	M	R	Company registration number
typeCode	number	5,0	M	R	Customer type. Is used to distinguish regular customers, employees, corporate client, etc. Possible values: 0: Normal 1: Employee 2: Company
statCode	string	2	M	R	Customer status (configurable value)
title	string	10	O	R	Title

firstName	string	50	M	R	First name
lastName	string	50	M	R	Last name
dateBirth	date		M	R	Date of birth
homeCountry	string	20	M	R	Home country
homeCity	string	40	O	R	Home city
homeAddress1	string	128	O	R	Home address (line 1)
homeAddress2	string	128	O	R	Home address (line 2)
homeAddress3	string	128	O	R	Home address (line 3)
fatherName	string	50	O	R	Father name
maidenName	string	50	O	R	Maiden name
latinFirstName	string	50	O	R	First name of customer in Latin
latinLastName	string	50	O	R	Last name of customer in Latin
homeZip	string	10	O	R	Home ZIP
customerCode2	string	64	O	R	Customer code2
middleName	string	50	O	R	Middle name
passPhrase	string	50	O	R	Customer pass phrase
mobTel	string	50	O	R	Mobile phone
documentId	string	50	O	R	Document (passport, id) number
nationality	string	50	O	R	Nationality
nationalId	string	50	O	R	National identification code
homeState	string	40	O	R	State name
priv1	string	64	O	R	Private data 1 for customer
priv2	string	64	O	R	Private data 2 for customer
priv3	string	64	O	R	Private data 3 for customer
priv4	string	64	O	R	Private data 4 for customer
priv5	string	64	O	R	Private data 5 for customer
email	string	254	O	R	Email Address

7.24 xmiss/getCustomerAccountList

Retrieve list of Customer's Accounts.

7.24.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
customerKey	Object		M	CustomerKey Object
pagingAdvanced	Object		M	Advanced paging object.

7.24.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
accountList	Array		M	R	Array of AccountBasicInfo objects. See "AccountBasicInfo Object".

7.24.2.1 AccountBasicInfo Object

Field	Format	Length	M/O	Rest r	Description
accountKey	Object		M	-	AccountKey object
typeCode	string	2	M	R	Account type code (configurable value)
avlBal	numeric	10,6	M	R	Available balance.
blkAmt	numeric	10,6	M	R	Blocked amount.
openDate	date		M	R	Account open date
lastUsage	date		M	R	Account last usage date
statCode	string	2	M	R	Account status (configurable value)
crLimit	number	10,6	O	R	Credit Limit
balincr	number	10,6	O	R	Temporary balance increase
balincrexp	date		O	R	Expiry date of temporary balance increase
blkAmtCr	number	10,6	M	R	Total credit holds
blkAmtDr	number	10,6	M	R	Total debit holds
tstampBalance	string	20	M	R	UTC timestamp of last balance update. Note! Different values for each account.

7.25 xmiss/getCustomerCardList

Retrieve list of Customer's Cards.

7.25.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
customerKey	Object		M	CustomerKey Object
pagingAdvanced	Object		M	Advanced paging object.

7.25.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
cardList	Array		M	R	Array of CardBasicInfo objects. See: "CardBasicInfo Object".

7.26 xmiss/getCustomerCardAccountLinkageList

Retrieve list of Customer's Card-Account Linkage.

7.26.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
customerKey	Object		M	CustomerKey Object.
pagingAdvanced	Object		M	Advanced paging object.

7.26.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
cardAccountLinkageList	Array		M	R	Array of CardAccountLinkage objects.

7.26.2.1 CardAccountLinkage Object

Field	Format	Length	M/O	Restr	Description
cardKey	Object		M	R	CardKey Object
accountKey	Object		M	R	AccountKey Object
priority	number	5	M	R	Card-Account linkage priority

7.27 xmiss/getAccountBasicInfo

Retrieve basic information about Account.

7.27.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
accountKey	Object		M	AccountKey Object

7.27.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
accountBasicInfo	Object		M	R	AccountBasicInfo Object. See "AccountBasicInfo Object".

7.28 xmiss/getAccountCardList

Retrieve list of Account's Cards.

7.28.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
accountKey	Object		M	AccountKey Object

pagingAdvanced	Object		M	Advanced paging object.
----------------	--------	--	---	-------------------------

7.28.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
cardList	Array		M	R	Array of CardBasicInfo objects. See: "CardBasicInfo Object".

7.29 xmiss/getAccountLimitList

Retrieve list of Account's Limits.

7.29.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
accountKey	Object		M	AccountKey Object
pagingAdvanced	Object		M	Advanced paging object.

7.29.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
accountLimits	Array		M	R	Array of LimitGet Object for limitTypes values 'AACN'. See: "LimitGet Object"

7.30 xmiss/getAccountStatement

Retrieve Account's Statement.

7.30.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
accountKey	Object		M	AccountKey Object
dateFrom	date		M	Statement start date
dateTo	date		M	Statement end date.
pagingAdvanced	Object		M	Advanced paging object.

7.30.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
startState	Object		M	R	Account state at the beginning of statement period. Object AccountState
endState	Object		M	R	Account state at the end of statement period. Object AccountState
accountLog	Array		M	R	Account statement items. Array of AccountLog Objects.

7.30.2.1 AccountState Object

Field	Format	Length	M/O	Restr	Description
typeCode	string	2	M	R	Account type code.
currency	string	3	M	R	Currency code of the account ISO 4217:2001 numeric codes
avlBal	number	10,6	M	R	Available balance. This value excludes the blocked amount.
blkAmt	number	10,6	M	R	Blocked amount. This represents the total blocked amount on the account.
avlBalUnsent	number	10,6	O	R	RFU. Available Balance not yet delivered to other back office systems.
blkAmtUnsent	number	10,6	O	R	RFU. Blocked amount not yet delivered to other back office systems.
openDate	date		M	R	Account open date
lastUsage	date		M	R	Account last usage date
statCode	string	2	M	R	Account status (configurable value).
crLimit	number	10,6	M	R	Credit Limit
balIncr	number	10,6	O	R	Temporary balance increase amount. Present only if balIncrExp is set.
balIncrExp	date		O	R	Expiry date of temporary balance increase amount

7.30.2.2 AccountLog Object

Field	Format	Length	M/O	Restr	Description
id	number	20,0	M	R	Account Log ID
oldState	Object		M	R	Object containing the historical states of modified

					account attributes before the change. Object AccountState. See "AccountState Object".
newState	Object		M	R	Object containing the updated states of modified account attributes after the change. Object AccountState. See "AccountState Object".
tstampInsert	string	20	M	R	UTC Timestamp when Account Log record was inserted. Format: YYYYMMDDHHmmSSccccc
dateLocal	date		M	R	Record insert date using local timezone.
tlId	number	20,0	O	R	Transaction Log ID. Present only in case when Account Log record is related to transaction.
recordType	number	5,0	M	R	Record type, reason of Account Log record: • 1 - Online transaction, • 2 - Clearing transaction, • 3 - balimp, • 4 - Rule chain script, • 5 - Account balance adjustment operation through XAPI, • 6 - Account data update through GUI
description	string	100	O	R	Transaction description
procName	string	100	M	R	Process who changed record
userName	string	100	M	R	Username who changed the account
extTxnId	string	100	O	R	External transaction ID

8 Transaction Services

8.1 Overview

8.1.1 Transaction Key Attributes

There are 2 unique key attributes for transactions:

- **ecTxRefno** - E-commerce Transaction Reference Number. Used for transactions generated by External API. Value is generated before authorisation and is used when querying transaction status or generating reversal. Value is present for transactions generated through e-commerce channels, but may be empty for transactions from other channels, for example from POS, ATM, etc.
- **tlId** - Transaction ID in G2B. Value is generated at the time of authorisation. Value is present for every transaction and is used when querying transaction details.

8.2 xkernel/initiateTransaction

Initiate transaction and generate new E-commerce Transaction Reference Number (ecTxRefno).

Service must be called before xkernel/authorizeTransaction and xkernel/reverseTransaction calls.

8.2.1 Request

Empty.

8.2.2 Response

Field	Format	Length	M/O	Restr	Description
ecTxRefno	string	150	M		E-commerce Transaction Reference Number. Note. For backward compatibility reasons attribute name may be changed to "ectxrefno" at configuration level. Please contact system administrators for more information.

8.3 xkernel/getCardTransactionHistory

Retrieve historical list of Card's Transactions.

Service supports paging and paging is mandatory.

Last retrieved value of cardTransactions.tlId must be used as paging.lastRetrievedId in next request.

8.3.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		M	CardKey Object. expiryDate is optional.
dateInsertUTCFrom	date		O	Transaction from date yyyymmddHHMMSSmmmmmm
dateInsertUTCTo	date		O	Transaction to date yyyymmddHHMMSSmmmmmm
dateInsertLocalFrom	date		O	Transaction local from date yyyymmddHHMMSSmmmm
dateInsertLocalTo	date		O	Transaction local to date yyyymmddHHMMSSmmmm
paging	Object		M	PagingRequest Object

8.3.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
cardTransactions	Object[]		M	R	Array of Transaction objects
tlId	numeric	20,0	M	R	Transaction ID in G2B
ecTxRefno	string	150	O	R	E-commerce Transaction Reference Number.
busDate	date		M	R	System Business Date
stan	numeric	10,0	M	R	System Trace Audit Number
rrn	string	12	M	R	Retrieval reference number
dateExp	string	4	M	R	Card expiration date YYMM
termType	string	1	O	R	Terminal Type
lkey	Object		M	R	Lkey object
crdacptID	string	15	M	R	Card Acceptor ID Code Merchant code. Codes are defined in processing system and value must be provided by Processing
crdacptBus	numeric	5,0	M	R	Card Acceptor Business Code (MCC values according to ISO8385 specification)
termCode	string	16	M	R	Terminal Code Terminal codes are defined in processing system and value must be provided by Processing System.
aiid	string	11	M	R	Acquiring Institution ID Code
msgcls	string	1	M	R	Message Class
msgclsorg	string	1	O	R	Message Class of Original Txn
msgfn	string	1	M	R	Message Function
msgfnorg	string	1	O	R	Message Function of Original Txn
txnsrc	string	1	M	R	Transaction Source
txnsrcorg	string	1	O	R	Transaction Source of Original Txn
fnCode	numeric	5,0	M	R	Function Code
txnCode	numeric	5,0	M	R	Processing Code
txStatus	numeric	5,0		M	Transaction status. (see Appendix B - Transaction Status Codes)
txnAmount	numeric	10,6		M	Txn Amount in Acquirer Currency
txnCurrency	string	3		M	Currency Code of Txn Amount ISO 4217:2001 numeric codes

curbill	string	3		O	Currency Code of Billing Amount ISO 4217:2001 numeric codes
amtbill	numeric	10,6		O	Card Holder Billing Amount
amtbillcb	numeric	10,6		O	Txn Amount of Cashback in Billing Currency
ratebill	numeric	10,6		O	Billing Rate
actionCode	string	1	M	R	Response Action
rspCode	string	2	M	R	Response Code
reasonCode	numeric	5,0		O	Reason Code (see Appendix C - Message Reason Codes)
fiid	string	11	O	R	Forwarding Institution ID Code
riid	string	11	O	R	Receiving Institution ID Code
crdactptlocName	string	40	O	R	Card Acceptor name
crdactptlocCity	string	40	O	R	Card Acceptor city
crdactptlocCountry	string	3	O	R	Card Acceptor country
crdactptlocPostcode	string	10	O	R	Card Acceptor postcode
tstamp_insert	string	20		O	Insert date/time, UTC yyyymmddHHMMSSmmm
when_created	string	17		O	Transaction local processing datetime yyyymmddHHMMSSmmm
accountNumber	string	64	M	R	Account number
avlBal	numeric	10,6	M	R	Available balance.
blkAmt	numeric	10,6	M	R	Blocked amount.
crlimit	number	10,6	O	R	Credit Limit
balincr	number	10,6	O	R	Temporary balance increase
balincexp	date		O	R	Expiry date of temporary balance increase
paging	Object		M		PagingResponse Object

8.4 xkernel/getTransactionDetails

Retrieve full details of one transaction.

8.4.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
tlld	numeric	20,0	C ¹	Transaction ID in G2B
ecTxRefno	String	150	C ¹	E-commerce Transaction Reference Number

C¹ - Either tlld or ecTxRefno must be present. If both fields present then tlld is used.

8.4.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
transaction	Object		M	R	
tlld	numeric	20,0	M	R	Transaction ID in G2B
ecTxRefno	string	150	O	R	E-commerce Transaction Reference Number
stan	numeric	10,0	M	R	System Trace Audit Number
stanorg	numeric	10,0	O	R	System Trace Audit Number of Original Txn
rrn	string	12	M	R	Retrieval reference number
rrnorg	string	12	O	R	Retrieval reference number of Original Txn
termtype	string	1	O	R	Terminal Type
termseq	numeric	10,0	O	R	Transaction Seq Number from Terminal
privdata1	numeric	5,0	O	R	Private data
issrtcode	string	16	M	R	Issuer Route Cod
acqrcode	string	16	M	R	Acquirer Route Code
lkey	Object		M	R	Lkey object
dateExp	string	4	M	R	Card expiration date YYMM
svccode	numeric	5,0	O	R	Card Service Code
crdacptBus	numeric	5,0	M	R	Card Acceptor Business Code (MCC values according to ISO8385 specification)
crdacptID	string	15	M	R	Card Acceptor ID Code Merchant code. Codes are defined in processing system and value must be provided by Processing
termCode	string	16	M	R	Terminal Code Terminal codes are defined in processing system and value must be provided by Processing System.
aiid	string	11	M	R	Acquiring Institution ID Code
acqCountry	string	3	M	R	Acquiring Institution Country Code
issCountry	string	3	M	R	Issuing Institution Country Code
dateLocal	date		M	R	Actual Local Date of Txn
timeLocal	time		M	R	Actual Local Time of Txn
msgcls	string	1	M	R	Message Class
msgclsorg	string	1	O	R	Message Class of Original Txn
msgfn	string	1	M	R	Message Function
msgfnorg	string	1	O	R	Message Function of Original Txn
txnsrc	string	1	M	R	Transaction Source
txnsrcorg	string	1	O	R	Transaction Source of Original Txn
fnCode	numeric	5,0	M	R	Function Code

txnCode	numeric	5,0	M	R	Transaction Code
sub_txnCode	string	2	O	R	Sub-Transaction Code
txStatus	numeric	5,0	M	R	Transaction status. (see Appendix B - Transaction Status Codes)
rspsrc	string	2	O	R	Response Source
stipReason	numeric	5,0	O	R	STIP reason
rejReason	numeric	5,0	O	R	Reject Reason Code
txnAmount	numeric	10,6	M	R	Txn Amount in Acquirer Currency
txnCurrency	string	3	M	R	Currency Code of Txn Amount ISO 4217:2001 numeric codes
amttxnfee	numeric	10,6	O	R	Txn Fee Amount
amtprocfee	numeric	10,6	O	R	Processing Fee Amount
amttxnorg	numeric	10,6	O	R	Original Txn Amount
amttxncb	numeric	10,6	O	R	Txn Amount of Cashback
actionCode	string	1	M	R	Response Action (see Error! Reference source not found.)
rspCode	string	2	M	R	Response Code (see Error! Reference source not found.)
reasonCode	numeric	5,0	O	R	Reason Code (see Appendix C - Message Reason Codes)
aprvcCode	string	9	O	R	Approval Code
fiid	string	11	O	R	Forwarding Institution ID Code
riid	string	11	O	R	Receiving Institution ID Code
dateset	date		O	R	Settlement Date
amtset	numeric	10,6	O	R	Settlement Amount
rateset	numeric	10,6	O	R	Settlement Conversion Rate
curset	string	3	O	R	Settlement Currency ISO 4217:2001 numeric codes
busdate	date		M	R	Business Date
afe	string	8	M	R	Acquiring FE
ife	string	8	M	R	Issuing FE
curbill	string	3	O	R	Currency Code of Billing Amount ISO 4217:2001 numeric codes
amtbill	numeric	10,6	O	R	Card Holder Billing Amount
amtbillcb	numeric	10,6	O	R	Txn Amount of Cashback in Billing Currency
ratebill	numeric	10,6	O	R	Billing Rate
amtbillbal	numeric	10,6	O	R	Customers own funds spent on this transaction in Billing currency (curBill)

amtbillcred	numeric	10,6	O	R	Credit funds spent on this transaction in Billing currency (curBill)
scheme	string	12	M	R	Card Scheme Related to Card Group
trn	string	25	O	R	Network Txn ID (Visa 62.2 or Banknet 63)
arn	string	23	O	R	Acquirer Reference Number
curtxnfee	string	3	O	R	Currency for Txn Fee ISO 4217:2001 numeric codes
curprocfee	string	3	O	R	Currency for Processing Fee ISO 4217:2001 numeric codes
curtxnorg	string	3	O	R	Currency for Original Txn ISO 4217:2001 numeric codes
curtax	string	3	O	R	Currency for Transaction Tax ISO 4217:2001 numeric codes
amtcom	numeric	10,6	O	R	Txn Commission Value in Txn Currency
curcom	string	3	O	R	Currency for Transaction Commission ISO 4217:2001 numeric codes
crdacptlocName	string	40	O	R	Card Acceptor name
crdacptlocStreet	string	35	O	R	Card Acceptor Street
crdacptlocCity	string	40	O	R	Card Acceptor city
crdacptlocPostcode	string	10	O	R	Card Acceptor postcode
crdacptlocRegion	string	3	O	R	Card Acceptor region
crdacptlocCountry	string	3	O	R	Card Acceptor country
bingrp	string	19	O	R	BIN group code
senderLkey	Object		O	R	Sender lkey object
senderlocName	string	30	O	R	MoneySend Sender's name
senderFundsrc	string	2	O	R	Source of Funds
senderlocCity	string	25	O	R	MoneySend Sender's city
senderlocStreet	string	35	O	R	MoneySend Sender' street
senderlocCountry	string	3	O	R	MoneySend Sender's country

recipientkey	Object		O	R	Recipient lkey object
recipientCountry	string	3	O	R	Recipient country code
idorg	numeric	20,0	O	R	Original transaction ID. Filled with original transaction ID for reversals and PGI refunds.
purchaseRefno	string	150	O	R	E-Commerce purchase reference number
transactionGroups	TransactionGroup[]		O	R	Transaction groups related to current transaction.
merchantComment	string	255	O	R	Merchant comment
amtTip	Numeric	10,6	O	R	Tip amount (informative field)
curTip	string	3	O	R	Tip amount currency
mcMac	string	2	O	R	Mastercard Merchant Advice Code (DE47.84). Values: 03 - Do not try again 21 - Payment Cancellation (Mastercard use only) - See other values in Mastercard Customer Interface Specification
trnOrg	string	15	O	R	Original TRN identifier (Mastercard TRACE ID – DE48 S63)
sourceAccountType	string	2	O	R	Source (Sender) Account Type
destinationAccountType	string	2	O	R	Destination (Recipient) Account Type

8.5 xkernel/authorizeTransaction

Generate online transaction and send for processing. Based on Card PAN transaction will be routed to Issuer bank or external network (MC/VISA) for authorisation.

Real-time response will be returned in fields.actionCode and status.rspCode.

IMPORTANT: In case if caller doesn't receive response message it is caller's responsibility to check transaction's status by calling "xkernel/getTransactionStatus" Under certain circumstances it is possible having situation where response message is lost, but Processing System doesn't know about this and Transaction is successful in Processing system and in Card Issuer's system.

Web Service Request contains minimum number of attributes that are required for real online transaction (ISO8583). Missing attributes will be set up by Processing system, attributes will be configurable.

8.5.1 Acquirer commission

There are 2 options how acquirer commission may be calculated and applied to transaction.

- **Merchant calculated commission.** Merchant may calculate acquirer commission by himself and pass commission amount in request field merchantCommission, together with other optional details, like commission code, percentage, min/max amount etc Commission amount also must be included in transaction amount (txnAmt)!
- **G2B Processing calculated commission.** If Merchant chooses to use processing calculated commissions, then:
 - Service xkernel/calculateAcqCommission may be called before xkernel/authorizeTransaction. In response there will be commission amount returned, in case if there are appropriate commission rules defined in G2B Processing.
 - After receiving calculated commission Merchant has 2 choices to choose from:
 - 1. Accept commission. In this case commission amount must be passed in request field merchantCommission. Commission amount also must be included in transaction amount (txnAmt)! Commission amount must be the same as returned in response of service xkernel/calculateAcqCommission. If different amount is passed in field merchantCommission, transaction will be rejected!
 - 2. Skip commission. In this case request field merchantCommission must be skipped and transaction will be executed without Merchant commission.

Table below represent differences between 2 types of Acquirer Commissions and:

Service / field	Merchant calculated commission	G2B Processing calculated commission
xkernel/calculateAcqCommission	Service may not be called!	Service should be called before xkernel/authorizeTransaction!
• acqCommCode	-	Field may be optionally used in order to force using 1 special commission. Field value is passed to Stronghold and is available in commission rules.
• acqCommSource • merchantCommission • acqCommDefAmtFix • acqCommDefPercents • acqCommDefAmtMin • acqCommDefAmtMax	-	Fields are ignored.
xkernel/authorizeTransaction	Service should be called.	Service should be called
• merchantCommission	Field must be set in order to have commission.	Field must be set in order to have commission.
• acqCommSource	Must have value: 3.	May have value 0 or field may be skipped.
• acqCommCode • acqCommDefAmtFix • acqCommDefPercents • acqCommDefAmtMin • acqCommDefAmtMax	Fields may be set optionally. Values will be stored in G2B Processing database.	Fields are ignored.

In all cases it is highly recommended to display commission amount to cardholder and asking for approval before calling xkernel/authorizeTransaction!

Service calls xkernel/calculateAcqCommission and xkernel/authorizeTransaction are linked by ecTxRefno value.

8.5.2 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
cardKey	Object		C ¹	CardKey Object. For txnType REFUND field expiryDate is optional.
ecTxRefno	string	150	M	Transaction Reference Number which was generated by service xkernel/initiateTransaction.
txnType	string	16	M	Transaction Type: • "SALES" -Sales • "CASH" -Cash • "DEPOSIT" - Deposit • "BALANCE" - Balance • "TRANSF_C2C" - TransferCardToCard • "TRANSF_H2C" - TransferCashToCard • "TRANSF_C2A" - TransferCardToAcc • "TRANSF_A2C" - TransferAccToCard • "ACCV" - Account Verification • "REFUND" - Refund • "PREAUTH" - Preauthorisation • "SALES_COM" - Sales completion
txnAmount	numeric	10,6	M	Txn Amount in Acquirer Currency (Transaction amount that should be authorized on cardholder account (includes Acquirer Commission amount)) In case of Account verification transaction amount must be set to 0
txnCurrency	string	3	M	Currency Code of Txn Amount ISO 4217:2001 numeric codes
termCode	string	16	M	Terminal code Terminal codes are defined in processing system and value must be provided by Processing System.
crdactID	string	15	M	Card Acceptor ID Code Merchant code. Codes are defined in processing system and value must be provided by Processing

merchantCommission	numeric	10,6	0	Information value about Acquirer Commission amount, that requester included into txnAmount. If this field is present and xkernel/calculateAcqCommission was called for this transaction then merchantCommission amount will override amount that was previously calculated. Usage of this field may be restricted at configuration level for different XAPI callers!
acqCommSource	numeric	1	0	Acquirer commission source: 0 - G2B Processing 3 - Merchant Default value: 0
acqCommCode	string	32	0	Acquirer commission code. Field may be used in 2 scenarios: - In case of Merchant Calculated Commission, field may be passed in service xkernel/authorizeTransaction request and value will be stored in G2B Processing DB. - In case of G2B Processing Calculated Commission, field may be passed in request of service xkernel/calculateAcqCommission and may be used in commission definition rules.
acqCommDefAmtFix	numeric	10,6	C ⁵	Acquirer Commission definition - fixed amount.
acqCommDefPercents	numeric	10,6	C ⁵	Acquirer Commission definition - percents.
acqCommDefAmtMin	numeric	10,6	C ⁵	Acquirer Commission definition - minimum amount.
acqCommDefAmtMax	numeric	10,6	C ⁵	Acquirer Commission definition - maximum amount.
issCommCode	string	32	0	Issuer commission code. If value provided then it will force G2B Processing to apply commission with given code regardless of default commission rules!
issCommSource	numeric	1	C ⁷	Pre-calculated issuer commission source: 3 - Merchant Only one value "3" accepted now. If value present it will force G2B Processing to apply precalculated commission instead of applying default commission rules. If field present then

				issCommAmount should be present.
issCommAmount	numeric	10,6	C ⁷	Precalculated issuer commission amount. Amount must be exposed in the same currency as Transaction amount and Cardholder Account currency. In case of currency mismatch transaction will be rejected!
issCommDefAmtFix	numeric	10,6	C ⁸	Issuer Pre-calculated Commission definition - fixed amount.
issCommDefPercents	numeric	10,6	C ⁸	Issuer Pre-calculated Commission definition - percents.
issCommDefAmtMin	numeric	10,6	C ⁸	Issuer Pre-calculated Commission definition - minimum amount.
issCommDefAmtMax	numeric	10,6	C ⁸	Issuer Pre-calculated Commission definition - maximum amount.
crdacctBus	numeric	5,0	O	Card Acceptor Business Code (MCC values according to ISO8385 specification)
merchantName*	string	40	O	Merchant name NOTE: If one of "*" fields is filled, then other fields also must be filled up
merchantStreet*	string	35	O	Merchant street NOTE: If one of "*" fields is filled, then other fields also must be filled up
merchantCity*	string	40	O	Merchant city NOTE: If one of "*" fields is filled, then other fields also must be filled up
merchantPostcode*	string	10	O	Merchant postcode NOTE: If one of "*" fields is filled, then other fields also must be filled up
merchantCountry*	string	3	O	Merchant country NOTE: If one of "*" fields is filled, then other fields also must be filled up
merchantRegion	string	3	O	Merchant region
cvv2	string	3	O	Clear value of CVV2. Present only if clear CVV2 is allowed for Client.
messageFunction	numeric	5,0	M	Message type: 0-Request 2-Advice
sourceAccount	string	64	O	Source account number
sourceAccountType	string	2	O	Source (Sender) Account Type (!) Note: If txnType: "TRANSF_C2A" and field is not present, then field to be set '00' by default

destinationAccountType	string	2	C ¹	Destination (Recipient) Account Type (!) Note: 1. Do not use value '03' (Card Account) for TRANSF_C2A 2. If field is used in TRANSF_A2C, TRANSF_C2C or TRANSF_H2C, then send value '03'
recipientCardKey	Object		C ¹	CardKey Object Field expiryDate is optional!
senderfirstName	string	30	C ¹	MoneySend Sender's first name
senderMiddleName	string	30	O	MoneySend Sender's middle name.
senderlastName	string	30	C ¹	MoneySend Sender's last name
senderlocCity	string	25	O	MoneySend Sender's city
senderlocStreet	string	35	O	MoneySend Sender's street
senderlocState	String	2	O	MoneySend Sender's state.
senderlocCountry	string	3	O	MoneySend Sender's country
senderAccount	string	64	C ¹	MoneySend Sender's account (used when destination is Account)
senderlocPostcode	String	10	O	MoneySend Sender's postal code.
senderPhone	String	20	O	MoneySend Sender's phone number.
senderDateOfBirth	String	8	O	MoneySend Sender's date of birth.
senderIdType	String	2	O	MoneySend Sender's ID type. This field is used to determine the type of sender identification document for the Visa payment system. Possible values are: • CP (Company registration number), • TX (tax ID), • PS (passport number for individuals). These values are then converted into Visa tags: CPNY, TXIN, PASN. They are used for AFT (Account Funding Transactions) and OCT (Original Credit Transactions). The same field is used for Mastercard (Sender Identification Type), where the possible values are: • 00 - Passport, • 01 - National Identification Card, • 02 - Driver's License, • 03 - Government Issued, • 04 - Other. Note! The payment initiator must populate the data required by the respective payment system(VISA or MC), and no validation is performed on the processing side.

senderIdNum	String	25	0	MoneySend Sender's ID number.
senderIdCountry	String	3	0	MoneySend Sender's ID country.
senderIdExpdate	String	8	0	MoneySend Sender's ID expiry date.
senderNationality	String	3	0	MoneySend Sender's nationality.
senderCountryOfBirth	String	3	0	MoneySend Sender's country of birth. ISO 3 digit numeric country code
transferAdditionalMessage	String	65	0	MoneySend Sender's transfer additional message.
senderParticipationId	String	30	0	MoneySend Sender's participation ID.
transferPurpose	String	2	0	Transfer purpose. See MC online interface field DE108.03.05.
transferClaimCode	String	20	0	Transfer claim code.
recipientCountry	string	3	0	MoneySend Recipient country code
recipientCity	string	25	0	MoneySend Recipient city
recipientStreet	string	35	0	MoneySend Recipient street
recipientfirstName	string	30	0	MoneySend Recipient's first name
recipientMiddleName	string	30	0	MoneySend Recipient's middle name.
recipientlastName	string	30	0	MoneySend Recipient's last name
recipientAccount	string	64	0	MoneySend Recipient's account (used when destination is Account)
recipientState	String	3	0	MoneySend Recipient's state.
recipientPostcode	String	10	0	MoneySend Recipient's postal code.
recipientPhone	String	20	0	MoneySend Recipient's phone number.
recipientDateOfBirth	String	8	0	MoneySend Recipient's date of birth.
recipientIdType	numeric	2	0	MoneySend Recipient's ID type. This field is used to determine the type of recipient identification document for the Visa payment system. Possible values are: • CP (Company registration number), • TX (tax ID), • PS (passport number for individuals). These values are then converted into Visa tags: CPNY, TXIN, PASN. They are used for AFT (Account Funding Transactions) and OCT (Original Credit Transactions) operations. The same field is used for Mastercard (Recipient Identification Type), where the possible values are: • 00 - Passport, • 01 - National Identification Card, • 02 - Driver's License, • 03 - Government Issued,

				• 04 - Other. Note! The payment initiator must populate the data required by the respective payment system(VISA or MC), and no validation is performed on the processing side.
recipientIdNum	String	25	0	MoneySend Recipient's ID number.
recipientIdCountry	String	3	0	MoneySend Recipient's ID country.
recipientIdExpdate	String	8	0	MoneySend Recipient's ID expiry date.
recipientNationality	String	3	0	MoneySend Recipient's nationality.
recipientCountryOfBirth	String	3	0	MoneySend Recipient's country of birth. ISO 3 digit numeric country code
senderFundsSrc	string	2	0	MoneySend Source of Funds
businessAppId	string	3	C ²	MoneySend Business Application Identifier See valid values in Appendix D.
eci3DS	string	1	0	3DS Electronic Commerce Indicator
securityFlag	numeric	5,0	0	Security flag: • 0-No security • 1-DSRP
ucaf	string	40	0	Universal Cardholder Authentication Field
tavv	string	40	C ⁴	Token cryptogram In this field should be provided VISA TAVV cryptogram or MC DSRP cryptogram if received from payment system.
preAuthExpDate	date		0	Pre-authorization Expiration Date
purchrefno	string	150	0	E-Commerce purchase reference number
originalEcTxRefno	string	150	C ³	Transaction Reference Number of original transaction of refund or sales completion. Field is required for all SALES_COM transactions and for REFUND transactions in case when Refund original check is enabled in system, otherwise field is skipped. Note! Field is required in case when field "recurrInd" is provided and contain values "C" or "M".
3dsProgramProtocol	string	1	0	3-D Program Protocol: • 1 - 3-D Secure Version 1.0 (3DS 1.0) • 2 - EMV 3-D Secure (3DS 2.0)
3dsDsTransactionId	string	36	0	Directory Server Transaction ID.
mcPaymentGatewayId	string	11	0	Merchant Payment Gateway ID

				assigned by MasterCard. MasterCard recommendations: - Value 999998 should be used when the acquirer uses its own MPG (because the acquirer wholly owns the MPG). - Value 999997 should be used when the merchant uses no gateway.
merchantComment	string	255	0	Merchant comment
additionalData	String	255	0	Free text additional data
recurrInd	String	1	0	Recurring indicator: 0 - First transaction (original) - C - Subsequent CIT transaction - M - Subsequent MIT transaction In this case of values "C" and "M" field "originalEcTxRefno" is mandatory and should contain reference number of first transaction! Note! may be used only in case when txnType = "SALES"
recurrType	string	4	C6	Recurring transaction type. See values in Appendix I.
paymentTxnType	string	3	0	Payment transaction type. Used in MasterCard payments DE 48, subelement 77.

C¹ - request parameters depend on values in txnType:

- if "TRANSF_C2A" present then field such fields must be present:
 - cardkey, recipientAccount, destinationAccountType
- if "TRANSF_A2C" present then field such fields must be present:
 - cardkey, senderAccount, recipientCardKey (cardKey should be equal to recipientCardKey)
- if "TRANSF_C2C" present then field such fields must be present:
 - cardkey, recipientCardKey
- if "TRANSF_H2C" present then field such fields must be present:
 - cardkey, senderfirstName, senderlastName, senderAccount, recipientCardKey (cardKey should be equal to recipientCardKey)

NOTE: Other sender/ recipient fields are validated based on payment network requirements

C² – field is mandatory if txnType is TRANSF_C2A, TRANSF_A2C, TRANSF_C2C, TRANSF_H2C or Gambling transaction (MCC = 7995).

C³ – field may be present only in case when txnType is REFUND or SALES_COM.

C⁴ – field should be filled if for token transaction was returned cryptogram.

C⁵ – Fields may be used only in case of Merchant Calculated Commission. In case of G2B Processing Calculated Commission fields will be ignored!

C⁶ – Fields Field "recurrType" is mandatory if field "recurrInd" provided.

C⁷ – Fields issCommSource and issCommAmount may be used only together. If one present then other should be present as well.

C⁸ – Fields are optional, but may be used only in case if issCommSource is present.

8.5.3 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
transactionResponse	Object		M	R	TransactionResponse Object
tlld	numeric	20,0	M	R	Transaction ID in G2B
ecTxRefno	string	150	M	R	E-commerce Transaction Reference Number
stan	numeric	10,0	M	R	System Trace Audit Number
rrn	string	12	M	R	Retrieval reference number
actionCode	string	1	M	R	Response Action (the same value also present in status.code)
rspCode	string	2	M	R	Response Code (the same value also present in status.rspcode)
txStatus	numeric	5,0	M	R	Transaction status. (see Appendix B - Transaction Status Codes)
acqCommAmt	numeric	10,6	O	R	Acquirer commission amount
acqCommCur	string	3	O	R	Currency code ISO 4217:2001 numeric codes
issCommAmt	numeric	10,6	O	R	Issuer commission amount
issCommCur	string	3	O	R	Currency code ISO 4217:2001 numeric codes
rejLimitCode	string	32	O	R	Limit code which rejected transaction. Present only in case when transaction is rejected because of exceeded limit.
lkey	Object		M	R	Lkey object
recipientLkey	Object		O	R	Recipient Lkey object
transactionGroups	TransactionGroup[]		O	R	Transaction groups related to current transaction
aprvcCode	string	9	O	R	Approval Code

8.5.4 Preauthorization flow

Preauthorization transaction processing functionality works with 3 transaction types: preauthorization (PREAUTH), reversal (REVERSAL), completion (SALES_COM).

1. Is sent preauthorization transaction to `xkernel/authorizeTransaction` with preauthorised amount to be blocked.
Fields related to preauthorization flow:
 - 1.1. `txntype` contains type `PREAUTH`
 - 1.2. `txnAmount` contains preauthorization amount
2. If needed, calling `xkernel/reverseTransaction` can be returned full amount (on full preauthorization amount) or partial amount reversal. Partial reversal can be performed until all preauthorization amount is returned. For each reversal should be added linkage to original preauthorization transaction.
Fields related to preauthorization flow:
 - 2.1. `originalEcTxRefno` contains `ecTxRefno` from preauthorization transaction.
 - 2.2. `reversalAmount` contains amount less or equal to remaining preauthorization amount.
3. Finally, when final authorization amount is clear, to `xkernel/authorizeTransaction` should be sent completion transaction with final amount to be charged from client.
Fields related to preauthorization flow:
 - 3.1. `Txntype` contains type `SALES_COM`.
 - 3.2. `originalEcTxRefno` contains `ecTxRefno` from preauthorization transaction.
 - 3.3. `txnAmount` contains amount less or equal to remaining preauthorization amount after reversal processing (preauthorization amount – all related reversal amount).

8.6 `xkernel/reverseTransaction`

Reverse Transaction that was previously created by calling "`xkernel/authorizeTransaction`".

Service call will generate Reversal in Processing system and will put it into background queue for execution.

NOTE: Service `xkernel/initiateTransaction` must be called before call of `reverseTransaction`. New Reference Number is required for Reversal. Reference number of original transaction must be passed in field `originalEcTxRefno`.

8.6.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
<code>ecTxRefno</code>	string	150	M	Transaction Reference Number which was generated by service <code>xkernel/initiateTransaction</code> .
<code>originalEcTxRefno</code>	string	150	M	Transaction Reference Number of original transaction which is going to be reversed.
<code>reasonCode</code>	numeric	5,0	M	Reason Code (see Appendix C - Message Reason Codes)
<code>reversalAmount</code>	numeric	10,6	M	Reversal Amount in Acquirer Currency
<code>txnCurrency</code>	string	3	O	Currency Code of Txn Amount ISO 4217:2001 numeric codes. Must match original transaction currency. Attribute is optional for backward compatibility. It is recommended to

				set value for this attribute!
--	--	--	--	-------------------------------

8.6.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
transactionResponse	Object		M	R	TransactionResponse Object
tlld	numeric	20,0	M	R	Transaction ID in G2B
ecTxRefno	string	150	M	R	E-commerce Transaction Reference Number
stan	numeric	10,0	M	R	System Trace Audit Number
rrn	string	12	M	R	Retrieval reference number
actionCode	string	1	M	R	Response Action (the same value also present in status.code)
rspCode	string	2	M	R	Response Code (the same value also present in status.rspcode)
txStatus	numeric	5,0	M	R	Transaction status. (see Appendix B - Transaction Status Codes)

8.7 xkernel/getTransactionStatus

Check Transaction status in Processing system.

8.7.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
ecTxRefno	string	150	C ¹	Transaction Reference Number.
tlld	numeric	20,0	C ¹	Transaction ID in G2B.

C¹ – mandatory usage of one of the parameters. They are mutually exclusive and can't be used together.

8.7.2 Response

Request fields in Data object:

Field	Format	Length	M/O	Restr	Description
transactionStatus	Object		M	R	transactionStatus Object
tlld	numeric	20,0	M	R	Transaction ID in G2B
ecTxRefno	string	150	M	R	E-commerce Transaction Reference Number
rrn	string	12	M	R	Retrieval reference number
txStatus	numeric	5,0	M	R	Transaction status. (see Appendix B - Transaction Status Codes)
actionCode	string	1	M	R	Response Action (see Error! Reference source not found.)
rspCode	string	2	M	R	Response Code (see Error!)

					Reference source not found.)
aprvtCode	string	9	0	R	Approval Code

Field "txStatus" is the main source for transaction status identification. Take into account that some statuses are transient and may change. For more information see Appendix B - Transaction Status Codes.

8.8 xkernel/calculateAcqCommission

Service calculate Acquirer commission.

8.8.1 Request

The same fields as in Request of "xkernel/authorizeTransaction".

8.8.2 Response

Request fields in Data object:

Field	Format	Length	M/O	Filter	Description
acqCommissionItems	Object[]		M	R	Array of AcqCommissionItem Objects
amtComm	numeric	10, 6	M	R	Commission amount
amtSource	numeric	10, 6	M	R	Source amount - amount on which commission is calculated.
defCommCode	string	32	M	R	Commission definition: Unique commission definition code.
totalCommAmount	numeric	10, 6	M		Total commission amount (sum of all "amtComm" values)
commCurrency	string	3	M	R	Currency code ISO 4217:2001 numeric codes
ecTxRefno	string	15 0	M	R	E-commerce Transaction Reference Number. Note. For backward compatibility reasons attribute name may be changed to "ectxrefno" at configuration level. Please contact system administrators for more information.

9 POS services

9.1 xpos/getPosTypeList

Retrieve POS terminal types list based on search criteria.

9.1.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
pagingAdvanced	Object		M	Advanced paging object.
ordering	Array		O	Array of ordering objects. Supported ordering

				columns: - "code"
filters	Array		O	Array of filtering objects. Supported filtering columns: - "code"
data	Object		M	Empty object.

9.1.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
posTypeList	Array		M	R	Array of PosTypeList objects

PosTypeList object:

Field	Format	Length	M/O	Restr	Description
code	String	32	M		POS type code
description	String	64	M		POS type description

9.2 xpos/getPos

Retrieve POS terminal information.

9.2.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
terminalId	String	16	M	POS terminal identification code.

9.2.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
pos	Object		M		POS terminal object

Merchant object:

Field	Format	Length	M/O	Restr	Description
terminalId	String	16	M		Terminal ID code
merchantId	String	16	M		Merchant ID code
currencyCode	String	3	O		Terminal default currency
posType	String	32	M		POS type code
statusCode	Numeric		M		POS terminal status
geolp4	String	16	O		IP address
timezone	String	64	O		Time zone
dukptCntr	Numeric		M		DUKPT transaction counter
tstampInsert	String	20	O		Inserting timestamp
tstampClose	String	20	O		Closing timestamp

addresses	Array		M		Array of Address objects
-----------	-------	--	---	--	--------------------------

POS terminal address object:

Field	Format	Length	M/O	Restr	Description
addressType	Numeric		M		Address type: 3 - Physical address
locationName	String	25	O		Location name
address1	String	128	O		Address line 1
address2	String	128	O		Address line 2
city	String	40	O		City name
region	String	3	O		Region code
postalCode	String	40	O		ZIP/postal Code
country	String	3	O		Country code (numeric)
contactPerson1	String	128	O		Contact person 1
contactPerson2	String	128	O		Contact person 2
contactPerson3	String	128	O		Contact person 3
contactRole1	String	64	O		Contact person 1 role
contactRole2	String	64	O		Contact person 2 role
contactRole3	String	64	O		Contact person 3 role
contactPhone1	String	64	O		Contact person 1 phone
contactPhone2	String	64	O		Contact person 2 phone
contactPhone3	String	64	O		Contact person 3 phone
contactEmail1	String	254	O		Contact person 1 email
contactEmail2	String	254	O		Contact person 2 email
contactEmail3	String	254	O		Contact person 3 email
url	String	255	O		URL address
additionalInfo	String	255	O		Additional info

9.3 xpos/getPosList

Retrieve POS terminal list based on search criteria.

9.3.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Description
pagingAdvanced	Object		M	Advanced paging object.
ordering	Array		O	Array of ordering objects. Supported ordering columns: - "id"
filters	Array		O	Array of filtering objects. Supported filtering columns: - "terminalId" - "merchantId" - "statusCode" - "posType"
data	Object		M	Empty object.

9.3.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Restr	Description
posList	Array		M	R	Array of PosListItem objects

PosListItem object:

Field	Format	Length	M/O	Restr	Description
terminalId	String	16	M		Terminal ID code
merchantId	String	16	M		Merchant ID code
statusCode	Numeric		M		POS status
posType	String	32	M		POS type code

10 Data import services

10.1 xkernel/mdi

Service process MDI records. Structure of MDI records is defined in MDI specification. MDI header and footer is not used in service request. In Response service doesn't return MDI Response trailer.

Supported records:

- Add Currency rate
- Delete Currency rate
- Add Merchant
- Update Merchant
- Delete Merchant
- Set merchant address
- Delete merchant address
- Set merchant scheme
- Delete merchant scheme

10.1.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Filter	Description
mdiRecords	Object[]		M	R	Array of MDI records

10.1.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Filter	Description
mdiResponseHeader	Object[]		M	R	MDI response header object (13.1. Response header)
mdiResponseDetails	Object[]		O	R	Array of MDI record level response objects (13.2. Record level)

					response)
--	--	--	--	--	-----------

10.2 xmiss/mdi

Service process MDI records. Structure of MDI records is defined in MDI specification. MDI header and footer is used in service request. In Response service doesn't return MDI Response trailer.

Supported records:

- Add/Update/Delete customer
- Add/Update/Delete account
- Add/Update/Delete card

10.2.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Filter	Description
mdiRecords	Object[]		M	R	Array of MDI records

10.2.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Filter	Description
mdiResponseHeader	Object[]		M	R	MDI response header object (13.1. Response header)
mdiResponseDetails	Object[]		O	R	Array of MDI record level response objects (13.2. Record level response)

10.3 xpos/mdi

Service process MDI records. Structure of MDI records is defined in MDI specification. MDI header and footer is used in service request. In Response service doesn't return MDI Response trailer.

Supported records:

- Add POS terminal
- Update POS terminal
- Delete POS terminal
- Set POS terminal address
- Delete POS terminal address

10.3.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Filter	Description
-------	--------	--------	-----	--------	-------------

mdiRecords	Object[]		M	R	Array of MDI records
------------	----------	--	---	---	----------------------

10.3.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Filter	Description
mdiResponseHeader	Object[]		M	R	MDI response header object (13.1. Response header)
mdiResponseDetails	Object[]		O	R	Array of MDI record level response objects (13.2. Record level response)

11 Proxy services

11.1 xmiss/g2bHceProxy

Proxy service for G2B HCE module web services. Service should do security checks, retrieve user information and pass nested data object to HCE endpoint.

Because of security considerations only configured list of HCE services are forwarded to HCE service.

11.1.1 Request

Request fields in Root object:

Field	Format	Length	M/O	Filter	Description
hceService	String		M	R	HCE service name.
data	Object[]		O	R	HCE service request data. Contains request fields specified in D8 HCE Payment MAP Interface specification.

11.1.2 Response

Response fields in Data object:

Field	Format	Length	M/O	Filter	Description
data	Object		M	R	HCE service response data. Contains response fields specified in D8 HCE Payment MAP Interface specification.

12 Examples

NOTE: Examples in this version of document are outdated and not compatible to latest specification version.

Here are some truncated examples of Request and Response objects:

12.1.1 xmiss/getCardInfo

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyId": 55964,
    "expiryDate": "2701"
  },
  "reqCardBasicInfo": true,
  "reqCardAccounts": true,
  "reqCardLimits": true,
  "reqCardAccountLimits": true,
  "reqCardAuthRestrictions": true,
  "reqCardTransactions": true,
  "cardTransactionCount" : 2
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan": "5375939999999993",
    "expiryDate": "2701"
  },
  "reqCardBasicInfo": true,
  "reqCardAccounts": true,
  "reqCardLimits": true,
  "reqCardAccountLimits": true,
  "reqCardAuthRestrictions": true,
  "reqCardTransactions": true,
  "cardTransactionCount" : 2
}
```

RSP

```
{
  "data": {
    "lkey": {
      "maskedPan": "537593_9993",
      "pan": "5375939999999993",

```

```

    "sequence": 0,
    "keyId": 55964
  },
  "expiryDate": "2701",
  "title": "MR",
  "firstName": "Oleksander",
  "lastName": "Bondarenko",
  "embossName": "MASTERCARD",
  "deliveryPoint": "Kiev",
  "customerCode": "GC000000000000121634",
  "statCode": "00",
  "statReason": "XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  20221010162647,
  "effDate": "20170801",
  "svccode": "201",
  "pvv": "6576",
  "invalidPinTries": 1,
  "commissionCategory": "X1",
  "feeCategory": "X1",
  "limitCategory": "TEST3"
},
"cardAccounts": [],
"cardLimits": [],
"cardAccountLimits": [],
"cardAuthRestrictions": [],
  "tId": 32919,
  "ecTxRefno": "3HDA6VyKiyFbLhTth2Pb6hb1eYfb1api",
  "stan": 19113,
  "rrn": "301213019113",
  "dateExp": "",
  "keyId": 55964,
  "crdactID": "AC010009",
  "crdactBus": 6536,
  "termCode": "AC010009",
  "aiid": "025882",
  "fnCode": 200,
  "msgcls": "2",
  "txncode": 26,
  "txStatus": 2,
  "txnAmount": 5.0,
  "txnCurrency": "980",
  "actionCode": "1",
  "rspCode": "14",
  "crdactplocName": "FIN*Merch test",
  "crdactplocCity": "KYIV",
  "crdactplocCountry": "804",
  "crdactplocPostcode": "04053",
  "tstamp_insert": "20230112154625988239",
  "when_created": "20230112154625168"
  "tId": 32905,
  "ecTxRefno": "3Ra54QdnutHMH3NTuFNwXWb8fwjihXbQ",
  "stan": 19099,
  "rrn": "301209019099",
  "dateExp": "",
  "keyId": 55964,
  "crdactID": "AC010009",
  "crdactBus": 6536,
  "termCode": "10066133",
  "aiid": "025882",
  "fnCode": 200,
  "msgcls": "2",
  "txncode": 26,
  "txStatus": 2,
  "txnAmount": 50.0,
  "txnCurrency": "980",
  "actionCode": "1",
  "rspCode": "14",

```

```

        "crdactplocName": "FIN*Merch test",
        "crdactplocCity": "rthtrht",
        "crdactplocCountry": "112",
        "crdactplocPostcode": "trhth",
        "tstamp_insert": "20230112115024427079",
        "when_created": "20230112115023521"
    }
}
},
"status": {
    "code": "0",
    "rspcode": "00"
}
}
}

```

12.1.2 xmiss/getCardBasicInfo

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```

{
  "cardKey":
  {
    "lkeyId": 55964,
    "expiryDate": "2701"
  }
}

```

OR

REQ

```

{
  "cardKey":
  {
    "pan": "5375939999999993",
    "expiryDate": "2701"
  }
}

```

RSP

```

{
  "data": {
    "cardBasicInfo": {
      "lkey": {
        "maskedPan": "537593_9993",
        "pan": "5375939999999993",
        "sequence": 0,
        "lkeyId": 55964
      },
      "expiryDate": "2701",
      "title": "MR",
      "firstName": "Oleksander",
      "lastName": "Bondarenko",
      "embossName": "MASTERCARD",
      "deliveryPoint": "Kiev",
      "customerCode": "GC00000000000121634",
      "currcode": "980",
      "companyRegNumber": "1234",
      "statCode": "00",
      "statReason": "????????????????",
      "statExternalUser": "AP",
    }
  }
}

```

```

    "statRequester": "XAPI",
    "statChangeTime": "20221010162647",
    "effDate": "20170801",
    "cdproduct": "X001",
    "svccode": "201",
    "pvki": "1",
    "pvv": "6576",
    "dcec": 0,
    "invalidPinTries": 1,
    "cvv2Type": 0,
    "commissionCategory": "X1",
    "feeCategory": "X1",
    "limitCategory": "TEST3"
  }
},
"status": {
  "code": "0",
  "rspcode": "00"
}
}

```

12.1.3 xmiss/getCardAccounts

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```

{
  "cardKey":
  {
    "lkeyid": 69261,
    "expiryDate": "2011"
  }
}

```

OR

REQ

```

{
  "cardKey":
  {
    "pan": "9998880000000018",
    "expiryDate": "2011"
  }
}

```

RSP

```

{
  "data": {
    "cardAccounts": [
      {
        "accountNumber": "TE2ACC000000000000999",
        "alvBal": 100.0,
        "blkAmt": 0.0,
        "openDate": "20200202",
        "lastUsage": "20210505",
        "priority": 0,
        "crlimit": 0.0,
        "balincr": 0.0,
      }
    ]
  }
}

```

```

},
"status": {
  "code": "0",
  "rspcode": "00"
}
}

```

12.1.4 xkernel/getCardNotifications

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```

{
  "cardKey":
  {
    "pan": "5285290000000010",
    "expiryDate": "2504"
  }
}

```

OR

```

{
  "cardKey":
  {
    "lkeyId": 824,
    "expiryDate": "2504"
  }
}

```

RSP

```

{
  "data": {
    "cardNotifications": [
      {
        "notificationTarget": "+29999999",
        "serviceType": "SMSGEN"
      },
      {
        "notificationTarget": "+28888888",
        "serviceType": "SMSTXN"
      }
    ]
  },
  "paging": {
    "nextPagePresent": true
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.5 xkernel/getCardTransactionHistory

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyId": 21,
    "expiryDate": "2504"
  },
  "paging":
  {
    "size": 1,
    "lastRetrievedId": -1
  }
}
```

OR

```
{
  "cardKey":
  {
    "pan": "5204740000001002",
    "expiryDate": "2504"
  },
  "paging":
  {
    "size": 1,
    "lastRetrievedId": -1
  }
}
```

RSP

```
{
  "data": {
    "cardTransactions": [
      {
        "tId": 22479,
        "ecTxRefno": "ECOM_XAPI/00005Fdetu6qmtqvcNiMbVviKeiS19PM",
        "busDate": "20200629",
        "stan": 16792,
        "rrn": "212612016790",
        "dateExp": "2504",
        "lkey": {
          "maskedPan": "520474_1002",
          "pan": "5204740000001002",
          "sequence": 0,
          "lkeyId": 21,
          "lkeyAlias": "TE2C10LKEYALIAS0017"
        },
        "crdacptID": "AC010009",
        "crdacptBus": 5999,
        "termCode": "AC010009",
        "aiid": "025882",
        "fnCode": 400,
        "msgcls": "4",
        "msgclsorg": "2",
        "msgfn": "2",
        "msgfnorg": "0",
        "txnsrcc": "0",
        "txnsrccorg": "0",
        "txncode": 47,

```

```

        "txStatus": 4,
        "txnAmount": 600.63,
        "txnCurrency": "980",
        "actionCode": "0",
        "rspCode": "00",
        "reasonCode": 4000,
        "crdactplocName": "FIN*Merch test",
        "crdactplocCity": "KYIV",
        "crdactplocCountry": "804",
        "crdactplocPostcode": "04053",
        "tstamp_insert": "20220506151343028531",
        "when_created": "20220506151343027"
    }
}
},
"paging": {
    "nextPagePresent": true
},
"status": {
    "code": "0",
    "rspcode": "00"
}
}
}

```

12.1.6 xkernel/getTransactionDetails

REQ

```

{
    "tllid": 27931
}

```

RSP

```

{
    "data": {
        "transaction": {
            "tllid": 27931,
            "ecTxRefno": "XAPI/00005FcjvqsZuzGZcXj83G8gSyeWBT9x",
            "stan": 18083,
            "stanorg": 0,
            "rrn": "232711018083",
            "issrtcode": "TRNSFACQ",
            "acqrtcode": "NULLROUT",
            "lkey": {
                "maskedPan": "520474_1002",
                "pan": "5204740000001002",
                "sequence": 0,
                "lkeyld": 21,
                "lkeyAlias": "TE2C10LKEYALIAS0017"
            },
            "dateExp": "2504",
            "crdacptBus": 5999,
            "crdacptID": "AC010009",
            "termCode": "AC010009",
            "aiid": "025882",
            "acqCountry": "804",
            "dateLocal": "20221123",
            "timeLocal": "135447",
            "msgcls": "2",
            "msgfn": "0",
            "txnsrsc": "0",
            "fnCode": 200,
            "txnCode": 47,
            "sub_txnCode": "03",
            "txStatus": 3,
            "rspsrc": "C",
            "txnAmount": 50.30,
            "txnCurrency": "980",

```

```

    "amttxnfee": 1.50,
    "curtxnfee": "980",
    "actionCode": "0",
    "rspCode": "00",
    "dateset": "22630831",
    "busdate": "20200629",
    "afe": "88484848",
    "ife": "88484848",
    "scheme": "2:1:MASTER",
    "crdacptlocName": "FIN*Merch test",
    "crdacptlocStreet": "Sichovyh Striltsiv 50",
    "crdacptlocCity": "KYIV",
    "crdacptlocPostcode": "04053",
    "crdacptlocCountry": "804",
    "senderFundsrc": "CC",
    "senderlocCity": "Kyiv",
    "senderlocStreet": "Sichovyh Striltsiv 50",
    "senderlocCountry": "804",
    "recipientLkey": {
      "maskedPan": "-",
      "pan": "00000999889",
      "sequence": 0,
      "lkeyId": 14489
    },
    "recipientCountry": "804",
    "purchrefno": "internalid89769"
  }
},
"status": {
  "code": "0",
  "rspcode": "00"
}
}

```

12.1.7 xkernel/getTransactionStatus

REQ

```

{
  "ecTxRefno": "XAPI/00005FcjvqsZuzGZcXj83G8gSyeWBT9x"
}

```

RSP

```

{
  "data": {
    "transactionStatus": {
      "rrn": "232711018083",
      "actionCode": "0",
      "rspCode": "00",
      "txStatus": 3
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.8 xmiss/setCardStatus

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyId": 55964,
    "expiryDate": "2701"
  },
  "newStatCode": "01",
  "reason": "test03",
  "externalUser": "AP"
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan": "5375939999999993",
    "expiryDate": "2701"
  },
  "newStatCode": "01",
  "reason": "test03",
  "externalUser": "AP"
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00",
  }
}
```

12.1.9 xmiss/generatePIN

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyId": 55964,
    "expiryDate": "2701"
  }
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan": "5375939999999993",
    "expiryDate": "2701"
  }
}
```

RSP

```
{
```

```
"data": {
  "pan": {
    "maskedPan": "537593_9993",
    "sequence": 0,
    "lkeyId": 55964
  }
},
"status": {
  "code": "0",
  "rspcode": "00"
}
}
```

12.1.10 xmiss/setPIN

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

NOTE: This fields must be encrypted by client side HSM

- pinKeyUnderRSA - Key value should be encrypted under Public RSA key of processing. Cryptogram is HEX encoded
- pinBlock - Encrypted PIN Block. PIN block is HEX encoded

Also see chapter Appendix G - PIN block formats.

REQ

```
{
  "cardKey":
  {
    "lkeyId": 55964,
    "expiryDate": "2701"
  },
  "pinKeyUnderRSA": "481BCB9D75B4B4F692C17E377C47272AD1CB475F4082517A8C46E1D6D46DA2ECBB3C4B6815DB5C89296
0342F73BABF9C989BFABCE99A1B154A7D67D1AB647AAF3690884AA0F1986D4B39A59BC47390F574195D2E895EC3D83277082C
ADB0CB0A41CF543822010A82172F956A88A9493922EEDFEA49EF38E5F64EB5007EB63EBF1882E9E73BC10D626FD9082EAF658F
AA39334703DFE96465D48A0493AA2E8F16533219E9A2297413EA288400F9A558B8688ADD09DAF0222CBF97683C0DAA2C5FD83
501B708ACE063CEB3F169BEFE0ECA487D74158233D50678130658BCE062CD2387183BCEC2DB6321F2093C7B2A5CC8D86052B0
F247ECC159DD1218E5B87889",
  "pinBlock": "C788490714FA254A"
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan": "5375939999999993",
    "expiryDate": "2701"
  },
  "pinKeyUnderRSA": "481BCB9D75B4B4F692C17E377C47272AD1CB475F4082517A8C46E1D6D46DA2ECBB3C4B6815DB5C89296
0342F73BABF9C989BFABCE99A1B154A7D67D1AB647AAF3690884AA0F1986D4B39A59BC47390F574195D2E895EC3D83277082C
ADB0CB0A41CF543822010A82172F956A88A9493922EEDFEA49EF38E5F64EB5007EB63EBF1882E9E73BC10D626FD9082EAF658F
AA39334703DFE96465D48A0493AA2E8F16533219E9A2297413EA288400F9A558B8688ADD09DAF0222CBF97683C0DAA2C5FD83
501B708ACE063CEB3F169BEFE0ECA487D74158233D50678130658BCE062CD2387183BCEC2DB6321F2093C7B2A5CC8D86052B0
F247ECC159DD1218E5B87889",
  "pinBlock": "C788490714FA254A"
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00",
  }
}
```

12.1.11 xmiss/getCVV2

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey": {
    {
      "lkeyId": 69265,
      "expiryDate": "2601"
    }
  }
}
```

OR

REQ

```
{
  "cardKey": {
    {
      "pan": "5245234010000049",
      "expiryDate": "2601"
    }
  }
}
```

RSP

```
{
  "data": {
    "lkey": {
      "maskedPan": "524523_0049",
      "pan": "5245234010000049",
      "sequence": 0,
      "lkeyId": 69265
    },
    "cvv2": "313",
    "cvv2Type": 0,
    "cvv2Expiry": "20260131235959"
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.12 xmiss/setCardAccountPriority

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyid":69261,
    "expiryDate":"2011"
  },
  "accountNumber": "TE2ACC0000000000000999",
  "currency": "840",
  "priority":3
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan":"99988800000000018",
    "expiryDate":"2011"
  },
  "accountNumber": "TE2ACC0000000000000999",
  "currency": "840",
  "priority":3
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.13 xkernel/getLkey

REQ

```
{
  "cardKey":
  {
    "lkeyid":69261,
    "expiryDate":"2011"
  }
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan":"99988800000000018",
    "expiryDate":"2011"
  }
}
```

RSP

```
{
  "data": {
```

```

    "lkey": {
      "maskedPan": "999888_0018",
      "pan": "9998880000000018",
      "sequence": 0,
      "lkeyId": 69261,
      "lkeyAlias": "TE2LKEYALIAS0018"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.14 xmiss/ resetCardCVV2Counter

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```

{
  "cardKey":
  {
    "lkeyid":69261,
    "expiryDate":"2011"
  }
}

```

OR

REQ

```

{
  "cardKey":
  {
    "pan":"9998880000000018",
    "expiryDate":"2011"
  }
}

```

RSP

```

{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.15 xmiss/resetCardPINtries

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```

{
  "cardKey":

```

```
{
  "lkeyid":69261,
  "expiryDate":"2011"
}
}
OR
```

REQ

```
{
  "cardKey":
  {
    "pan":"9998880000000018",
    "expiryDate":"2011"
  }
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.16 xmiss/activateCard

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyid":69261,
    "expiryDate":"2011"
  },
  "reason":"test03",
  "externalUser":"AP"
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan":"9998880000000018",
    "expiryDate":"2011"
  },
  "reason":"test03",
  "externalUser":"AP"
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

```
}
}
```

12.1.17 xmiss/addAuthRestrictions

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyId":784,
    "expiryDate":"2504"
  },
  "restrictions":
  [
    {
      "restrCode":"TEST8",
      "restrType":"CCRD",
      "priority":9,
      "actFrom":"20220204102030",
      "actTo":"20220504102030"
    },
    {
      "restrCode":"TEST9",
      "restrType":"CCRD",
      "priority":6,
      "actFrom":"20220204102030",
      "actTo":"20220504102030"
    }
  ],
  "replace":false
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan":"5285290000000010",
    "expiryDate":"2504"
  },
  "restrictions":
  [
    {
      "restrCode":"TEST8",
      "restrType":"CCRD",
      "priority":9,
      "actFrom":"20220204102030",
      "actTo":"20220504102030"
    },
    {
      "restrCode":"TEST9",
      "restrType":"CCRD",
      "priority":6,
      "actFrom":"20220204102030",
      "actTo":"20220504102030"
    }
  ],
  "replace":false
}
```

```
}
]
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.18 xmiss/removeAuthRestrictions

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```
{
  "cardKey":
  {
    "lkeyId": "784",
    "expiryDate": "2504"
  },
  "restrictions":
  [
    {
      "restrCode": "TEST8",
      "restrType": "CCRD",
      "priority": 9,
      "actFrom": "20220204102030",
      "actTo": "20220504102030"
    },
    {
      "restrCode": "TEST9",
      "restrType": "CCRD",
      "priority": 6,
      "actFrom": "20220204102030",
      "actTo": "20220504102030"
    }
  ]
}
```

OR

REQ

```
{
  "cardKey":
  {
    "pan": "5285290000000010",
    "expiryDate": "2504"
  },
  "restrictions":
  [
    {
```

```

    "restrCode": "TEST8",
    "restrType": "CCRD",
    "priority": 9,
    "actFrom": "20220204102030",
    "actTo": "20220504102030"
  },
  {
    "restrCode": "TEST9",
    "restrType": "CCRD",
    "priority": 6,
    "actFrom": "20220204102030",
    "actTo": "20220504102030"
  }
]
}

```

RSP

```

{
  "data": {
    "deletedRecords": 1
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.19 xmiss/getAuthRestrictions

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

Exist 3 algorithms to find data about restrictions:

- CCAT - return 1 category restrictions, category code select from restrCategoryCode
- CCRD - return only card restrictions
- CCAT, CCRD - return card restrictions + category restrictions, category code select from card CARDA.authchk_cat

REQ 1

```

{
  "restrCategoryCode": "TEST",
  "restrTypes": [
    "CCAT"
  ]
}

```

RSP

```

{
  "data": {
    "restrictions": [
      {
        "restrCategoryCode": "TEST",
        "restrType": "CCAT",
        "priority": "1",
        "objectID": "TEST",
        "actFrom": "20220204102030",

```

```

        "actTo": "20220504102030"
      }
    ]
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

REQ 2

```

{  "cardKey":
  {
    "pan": "5285290000000010",
    "expiryDate": "2504"
  },
  "restrTypes":
  [
    "CCRD"
  ]
}

```

RSP

```

{
  "data": {
    "restrictions": [
      {
        "restrCategoryCode": "TEST9",
        "restrType": "CCRD",
        "priority": "6",
        "objectID": "784",
        "actFrom": "20220204102030",
        "actTo": "20220504102030"
      },
      {
        "restrCategoryCode": "TEST8",
        "restrType": "CCRD",
        "priority": "9",
        "objectID": "784",
        "actFrom": "20220204102030",
        "actTo": "20220504102030"
      }
    ]
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

REQ 3

```

{  "cardKey":
  {
    "pan": "5285290000000010",
    "expiryDate": "2504"
  },
  "restrTypes":
  [
    "CCRD",

```

```
    "CCAT"
  ]
}
```

RSP

```
{
  "data": {
    "restrictions": [
      {
        "restrCategoryCode": "TEST",
        "restrType": "CCAT",
        "priority": "1",
        "objectID": "TEST",
        "actFrom": "20220204102030",
        "actTo": "20220504102030"
      },
      {
        "restrCategoryCode": "TEST9",
        "restrType": "CCRD",
        "priority": "6",
        "objectID": "784",
        "actFrom": "20220204102030",
        "actTo": "20220504102030"
      },
      {
        "restrCategoryCode": "TEST8",
        "restrType": "CCRD",
        "priority": "9",
        "objectID": "784",
        "actFrom": "20220204102030",
        "actTo": "20220504102030"
      }
    ]
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.20 xmiss/setLimits

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

Request parameters depends on values present in LimitSet.limitType:

- if 'CCRD' present then field cardKey must be present
- if 'AACN' present then field accountNumber must be present

REQ 1

```
{
  "cardKey":
  {
    "pan": "5285290000000010",
    "expiryDate": "2404"
  },
  "limits":
  [
```

```
{
  "limitCode":"TEST8",
  "limitType":"CCRD",
  "priority":9,
  "actFrom":"20220204102030",
  "actTo":"20220504102030",
  "currCode":"980",
  "txnAmtMin":0.05,
  "txnAmtMax":5.05,
  "cycAmtMax":10.02,
  "cycNumMax":5
}
],
"replace":false
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

REQ 2

```
{
  "accountNumber":"27203086532002",
  "accountCurrCode":"980",
  "limits":
  [
    {
      "limitCode":"TEST8",
      "limitType":"AACN",
      "priority":9,
      "actFrom":"20220204102030",
      "actTo":"20220504102030",
      "currCode":"980",
      "txnAmtMin":0.05,
      "txnAmtMax":5.05,
      "cycAmtMax":10.02,
      "cycNumMax":5
    }
  ]
}
```

RSP

```
{
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.21 xmiss/removeLimits

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

Request parameters depends on values present in LimitSet.limitType:

- if 'CCRD' present then field cardKey must be present
- if 'AACN' present then field accountNumber must be present

REQ 1

```
{
  "cardKey":
  {
    "pan":"5285290000000010",
    "expiryDate":"2404"
  },
  "limits":
  [
    {
      "limitCode":"TEST8",
      "limitType":"CCRD",
      "priority":9,
      "actFrom":"20220204102030",
      "actTo":"20220504102030"
    }
  ]
}
```

RSP

```
{
  "data": {
    "deletedRecords": 1
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "reasonClass": 102,
    "reason": 3
  }
}
```

REQ 2

```
{
  "accountNumber":"27203086532002",
  "accountCurrCode":"980",
  "limits":
  [
    {
      "limitCode":"TEST8",
      "limitType":"AACN",
      "priority":9,
      "actFrom":"20220204102030",
      "actTo":"20220504102030"
    }
  ]
}
```

RSP

```
{
  "data": {
```

```

    "deletedRecords": 1
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "reasonClass": 102,
    "reason": 3
  }
}

```

12.1.22 xmiss/getLimits

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

Exist 3 algorithms to find data about restrictions:

- CCAT - return 1 category limits, limit code select from limitCategoryCode
- CCRD - return only card limits
- CCAT, CCRD - return card limits + category limits, limit code select from card CARDA.limit_cat
- AACN - return only account limits

REQ 1

```

{
  "limitCategoryCode": "ALIS_0001",
  "limitTypes":
  [
    "CCAT"
  ]
}

```

RSP

```

{
  "data": {
    "limits": [
      {
        "limitCode": "CCAT",
        "limitType": "CCAT",
        "priority": 1,
        "objectId": "limit_cat_test",
        "actFrom": "20220209111111",
        "actTo": "20220209222222",
        "currCode": "840",
        "txnAmtMin": 9.01,
        "txnAmtMax": 8.12,
        "cycAmtMax": 7.23,
        "cycNumMax": 0,
        "cycCntAmt": 0,
        "cycCntNum": 0
      }
    ]
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

}

REQ 2

```
{  "cardKey":
  {
    "pan": "5573670000000303",
    "expiryDate": "2501"
  },
  "limitTypes":
  [
    "CCRD",
    "CCAT"
  ]
}
```

RSP

```
{
  "data": {
    "limits": [
      {
        "limitCode": "TEST3",
        "limitType": "CCRD",
        "priority": 9,
        "objectid": "824",
        "actFrom": "20220204102030",
        "actTo": "20220504102030",
        "currCode": "",
        "txnAmtMin": 0,
        "txnAmtMax": 0,
        "cycAmtMax": 0,
        "cycNumMax": 0,
        "cycCntAmt": 0,
        "cycCntNum": 0
      },
      {
        "limitCode": "CCAT",
        "limitType": "CCAT",
        "priority": 1,
        "objectid": "limit_cat_test",
        "actFrom": "20220209111111",
        "actTo": "20220209222222",
        "currCode": "840",
        "txnAmtMin": 9.01,
        "txnAmtMax": 8.12,
        "cycAmtMax": 7.23,
        "cycNumMax": 0,
        "cycCntAmt": 0,
        "cycCntNum": 0
      },
      {
        "limitCode": "1",
        "limitType": "CCRD",
        "priority": 2,
        "objectid": "824",
        "actFrom": "20220209111111",
        "actTo": "20220209222222",
        "currCode": "840",
        "txnAmtMin": 9.01,
        "txnAmtMax": 8.12,
        "cycAmtMax": 7.23,
        "cycNumMax": 0,
        "cycCntAmt": 0,
        "cycCntNum": 0
      }
    ]
  }
}
```

```

    }
  ]
},
"status": {
  "code": "0",
  "rspcode": "00"
}
}

```

REQ 3

```

{
  "accountNumber": "26203086532002",
  "accountCurrCode": "980",
  "limitTypes":
  [
    "AACN"
  ]
}

```

RSP

REQ 4

```

{ "cardKey":
  {
    "pan": "5573670000000303",
    "expiryDate": "2501"
  },
  "limitTypes":
  [
    "CCRD"
  ]
}

```

RSP

```

{
  "data": {
    "limits": [
      {
        "limitCode": "TEST3",
        "limitType": "CCRD",
        "priority": 9,
        "objectid": "824",
        "actFrom": "20220204102030",
        "actTo": "20220504102030",
        "currCode": "",
        "txnAmtMin": 0,
        "txnAmtMax": 0,
        "cycAmtMax": 0,
        "cycNumMax": 0,
        "cycCntAmt": 0,
        "cycCntNum": 0
      },
      {
        "limitCode": "1",
        "limitType": "CCRD",
        "priority": 2,
        "objectid": "824",
        "actFrom": "20220209111111",

```

```

    "actTo": "20220209222222",
    "currCode": "840",
    "txnAmtMin": 9.01,
    "txnAmtMax": 8.12,
    "cycAmtMax": 7.23,
    "cycNumMax": 0,
    "cycCntAmt": 0,
    "cycCntNum": 0
  }
]
},
"status": {
  "code": "0",
  "rspcode": "00"
}
}

```

12.1.23 xmiss/setCVV2Type

There are two options how we can find necessary information:

- pan + expiryDate
- lkeyId + expiryDate

REQ

```

{
  "cardKey":
  {
    "lkeyid": "69265",
    "expiryDate": "2601"
  },
  "cvv2Type": 0
}

```

OR

REQ

```

{
  "data": {
    "lkey": {
      "maskedPan": "524523_0049",
      "pan": "5245234010000049",
      "sequence": 0,
      "lkeyId": 69265
    },
    "cvv2": "313",
    "cvv2Type": 0,
    "cvv2Expiry": "20260131235959"
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.24 xmiss/updateAccountBalance

Example with balance type ABSOLUTE.

REQ

```
{
  "data": {
    {
      "accountNumber": "97838936939",
      "balanceType": "ABSOLUTE",
      "currency": "643",
      "lastTransactionLogTimestamp": "20230717150400000000",
      "avlBal": 10.5,
      "externalTxnID": "TEST1"
    }
  }
}
```

RSP

```
{
  "data": {
    "oldBalance": {
      "avlBal": 21.1,
      "blkAmt": 0.1,
      "crLimit": 0.5,
      "balIncr": 0.6,
      "balIncrExp": "20220131"
    },
    "newBalance": {
      "avlBal": 10.55,
      "blkAmt": 0.1,
      "crLimit": 0.5,
      "balIncr": 0.6,
      "balIncrExp": "20220131"
    },
    "processingTimestamp": "20230718080601845383",
    "accountLogId": 9959
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "reasonClass": 102,
    "reason": 2
  }
}
```

Example with balance type DELTA (please note that 0 values in this case is the same as if field would be skipped).

REQ

```
{
  "data": {
    {
      "accountNumber": "987654321978654321398765432",

```

```

    "balanceType": "DELTA",
    "currency": "978",
    "avlBal": 550,
    "blkAmt": 20.0,
    "crLimit": 0.0,
    "balIncr": 0.0,
    "externalTxnID": "13533"
  }
}
RSP

```

```

{
  "data": {
    "oldBalance": {
      "avlBal": 1118.23,
      "blkAmt": 140.0,
      "crLimit": 1001.0,
      "balIncr": 110.0,
      "balIncrExp": "22630831"
    },
    "newBalance": {
      "avlBal": 1668.23,
      "blkAmt": 160.0,
      "crLimit": 1001.0,
      "balIncr": 110.0,
      "balIncrExp": "22630831"
    },
    "processingTimestamp": "20230908104613413072",
    "accountLogId": 30217
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "reasonClass": 102,
    "reason": 2
  }
}

```

12.1.25 xkernel/initiateTransaction

NOTE: Empty Request must be sended

REQ

```

{
}

```

RSP

```

{
  "ecTxRefno": "XAPI/00005J2nhvkv2rEZ5iAzZ9SzDfqG2aia"
}

```

12.1.26 xkernel/authorizeTransaction

Before transaction must be executed xkernel/initiateTransaction WS call

There are provided some examples for different transaction types:

- SALES
- TRANSF_C2A
- TRANSF_A2C
- TRANSF_C2C

Sales transaction example

REQ

```
{
  "cardKey":
  {
    "pan": "5204740000001002",
    "expiryDate": "2512"
  },
  "ecTxRefno": "XAPI/00005FdUXTHqSxRC3NYxVcBfadvScVsq",
  "txnType": "SALES",
  "txnAmount": "10.30",
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdacptID": "AC010009",
  "messageFunction": "0",
  "purchrefno": "internalid0507"
}
```

RSP

```
{
  "data": {
    "transactionResponse": {
      "tlld": "22469",
      "ecTxRefno": "ECOM_XAPI/00005FdUXTHqSxRC3NYxVcBfadvScVsq",
      "stan": "16782",
      "rrn": "212612016782",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": "0",
      "lkeyId": "21",
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

Card to Account transaction example

REQ

```
{
  "cardKey":
  {
    "pan": "5204740000001002",
    "expiryDate": "2504"
  },
  "ecTxRefno": "XAPI/00005FdUZqCAAXPCLzYEYGY44Qb3Who",
  "txnType": "TRANSF_C2A",
  "txnAmount": 500.63,
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdacptID": "AC010009",
  "cvv2": "100",
  "messageFunction": 0,
  "senderfirstName": "Oleksander",
  "senderlastName": "Bondarenko",
  "senderlocCity": "Kyiv",
  "senderlocStreet": "Sichovyh Striltsiv 50",
  "senderlocCountry": "804",
  "recipientCountry": "804",
  "recipientCity": "Kyiv",
  "recipientStreet": "Sichovyh Striltsiv 51",
  "recipientfirstName": "Yura",
  "recipientlastName": "Bura",
  "recipientAccount": "00000999889",
  "senderFundsSrc": "CC",
  "businessAppld": "TPP",
  "purchrefno": "internalid89769"
}
```

RSP

```
{
  "data": {
    "transactionResponse": {
      "tllid": 22470,
      "ecTxRefno": "ECOM_XAPI/00005FdUZqCAAXPCLzYEYGY44Qb3Who",
      "stan": 16783,
      "rrn": "212612016783",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": 0,
      "lkeyId": 21,
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

Account to Card transaction example

REQ

```
{
  "cardKey":
  {
    "pan": "5204740000001002",
```

```

    "expiryDate": "2504"
  },
  "ecTxRefno": "XAPI/00005FdUcDcz9888tCHoGWe4HdHZ6EaR",
  "txnType": "TRANSF_A2C",
  "txnAmount": 500.63,
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdacptID": "AC010009",
  "cvv2": "100",
  "messageFunction": 0,
  "senderfirstName": "Oleksander",
  "senderlastName": "Bondarenko",
  "senderlocCity": "Kyiv",
  "senderlocStreet": "Sichovyyh Striltsiv 50",
  "senderlocCountry": "804",
  "senderAccount": "00000999890",
  "recipientCardKey": {
    {
      "pan": "5204740000001002"
    }
  },
  "recipientCountry": "804",
  "recipientCity": "Kyiv",
  "recipientStreet": "Sichovyyh Striltsiv 51",
  "recipientfirstName": "Yura",
  "recipientlastName": "Bura",
  "senderFundsSrc": "CC",
  "businessAppId": "TPP",
  "eci3DS": "2",
  "securityFlag": 0,
  "ucaf": "VUwgQUNTIFNpbXVsYXRvcia1NDg=",
  "purchrefno": "internalid89769"
}

```

RSP

```

{
  "data": {
    "transactionResponse": {
      "tllid": 22472,
      "ecTxRefno": "ECOM_XAPI/00005FdUcDcz9888tCHoGWe4HdHZ6EaR",
      "stan": 16785,
      "rrn": "212612016785",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": 0,
      "lkeyId": 21,
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    },
    "recipientLkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": 0,
      "lkeyId": 21,
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

Card to Card transaction example

REQ

```
{
  "cardKey":
  {
    "pan": "5204740000001002",
    "expiryDate": "2512"
  },
  "ecTxRefno": "XAPI/00005FdUcHSGS4LZrLGaZ8W2bv4FAMYB",
  "txnType": "TRANSF_C2C",
  "txnAmount": 60.63,
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdaptID": "AC010009",
  "cvv2": "100",
  "messageFunction": 0,
  "senderfirstName": "Oleksander",
  "senderlastName": "Bondarenko",
  "senderlocCity": "Kyiv",
  "senderlocStreet": "Sichovyyh Striltsiv 50",
  "senderlocCountry": "804",
  "recipientCardKey":
  {
    "pan": "5346937100108112"
  },
  "recipientCountry": "804",
  "recipientCity": "Kyiv",
  "recipientStreet": "Sichovyyh Striltsiv 51",
  "recipientfirstName": "Yura",
  "recipientlastName": "Bura",
  "senderFundsSrc": "CC",
  "businessAppId": "TPP",
  "purchrefno": "internalid0507"
}
```

RSP

```
{
  "data": {
    "transactionResponse": {
      "tId": 22474,
      "ecTxRefno": "ECOM_XAPI/00005FdUcHSGS4LZrLGaZ8W2bv4FAMYB",
      "stan": 16787,
      "rrn": "212612016787",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": 0,
      "lkeyId": 21,
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    },
    "recipientLkey": {
      "maskedPan": "534693_8112",
      "pan": "5346937100108112",
      "sequence": 0,
      "lkeyId": 425
    }
  },
  "status": {
    "code": "0",
  }
}
```

```
    "rspcode": "00"
  }
}
```

Cash to Card transaction example

REQ

```
{
  "cardKey":
  {
    "pan": "5204740000001002",
    "expiryDate": "2504"
  },
  "ecTxRefno": "XAPI/00005FdetpCq12mputnreeeM2KVttHgD",
  "txnType": "TRANSF_H2C",
  "txnAmount": 600.63,
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdaptID": "AC010009",
  "cvv2": "100",
  "messageFunction": 0,
  "senderfirstName": "Oleksander",
  "senderlastName": "Bondarenko",
  "senderlocCity": "Kyiv",
  "senderlocStreet": "Sichovyh Striltsiv 50",
  "senderlocCountry": "804",
  "senderAccount": "00000999890",
  "recipientCardKey":
  {
    "pan": "5204740000001002"
  },
  "recipientCountry": "804",
  "recipientCity": "Kyiv",
  "recipientStreet": "Sichovyh Striltsiv 51",
  "recipientfirstName": "Yura",
  "recipientlastName": "Bura",
  "senderFundsrc": "CH",
  "businessAppId": "TPP",
  "issCommCode": "X1",
  "acqCommCode": "X1",
  "purchrefno": "internalid89769"
}
```

RSP

```
{
  "data": {
    "transactionResponse": {
      "tllid": 22477,
      "ecTxRefno": "ECOM_XAPI/00005FdetpCq12mputnreeeM2KVttHgD",
      "stan": 16790,
      "rrn": "212612016790",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": 0,
      "lkeyId": 21,
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    },
    "recipientLkey": {
```

```

    "maskedPan": "520474_1002",
    "pan": "5204740000001002",
    "sequence": 0,
    "lkeyId": 21,
    "lkeyAlias": "TE2C10LKEYALIAS0017"
  }
},
"status": {
  "code": "0",
  "rspcode": "00"
}
}

```

Preauthorization transaction example

REQ

```

{
  "cardKey":
  {
    "lkeyAlias": "Alias72",
    "expiryDate": "3001"
  },
  "ecTxRefno": "XAPI/00005FdUGyPnouSk3Skp2qze9Qr7hEbZ",
  "txnType": "PREAUTH",
  "txnAmount": 10.00,
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdactptID": "AC010009",
  "cvv2": "301",
  "messageFunction": 0,
  "issCommCode": "A1",
  "acqCommCode": "A1",
  "purchrefno": "internalid89767"
}

```

RSP

```

{
  "data": {
    "transactionResponse": {
      "tllid": 82001,
      "ecTxRefno": "XAPI/00005FdUGyPnouSk3Skp2qze9Qr7hEbZ",
      "stan": 39364,
      "rrn": "334010039364",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "524523_0072",
      "pan": "5245234010000072",
      "sequence": 0,
      "lkeyId": 71670,
      "lkeyAlias": "Alias72"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

Completion transaction example

REQ

```
{
  "cardKey": {
    "lkeyAlias": "Alias72",
    "expiryDate": "3001"
  },
  "ecTxRefno": "XAPI/00005FdUKR79YudkWsGkZHB8Mw3uSjk9",
  "originalEcTxRefno": "XAPI/00005FdUGyPnouSk3Sskp2qze9Qr7hEbZ",
  "txnType": "SALES_COM",
  "txnAmount": 10.00,
  "txnCurrency": "980",
  "termCode": "AC010009",
  "crdactplID": "AC010009",
  "cvv2": "301",
  "messageFunction": 0,
  "issCommCode": "A1",
  "acqCommCode": "A1",
  "purchrefno": "internalid89767"
}
```

RSP

```
{
  "data": {
    "transactionResponse": {
      "tllid": 82002,
      "ecTxRefno": "XAPI/00005FdUKR79YudkWsGkZHB8Mw3uSjk9",
      "stan": 39364,
      "rrn": "334010039364",
      "actionCode": "0",
      "rspCode": "00"
    },
    "lkey": {
      "maskedPan": "524523_0072",
      "pan": "5245234010000072",
      "sequence": 0,
      "lkeyld": 71670,
      "lkeyAlias": "Alias72"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.27 xkernel/reverseTransaction

Before transaction must be executed xkernel/initiateTransaction WS call

REQ

```
{
  "ecTxRefno": "XAPI/00005J3LeDWBww8ADwUhyfG9fhFVDzNj",
  "originalEcTxRefno": "XAPI/00005J2yaCjT3tpwpqfPp7xqe9WX7Ay5",
  "reasonCode": 4000,
  "reversalAmount": 850.30
}
```

RSP

```
{
  "data": {
```

```

    "transactionResponse": {
      "tllid": 22479,
      "ecTxRefno": "ECOM_XAPI/00005Fdetu6qmtqvcNiMbVviKeiS19PM",
      "stan": 16792,
      "rrn": "212612016790",
      "actionCode": "0",
      "rspCode": "00"
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}

```

12.1.28 xmiss/mis

REQ

```

{
  "mdiRecords":
  [
    {
      "ISS_RECTYPE": "CUSTOMER",
      "ISS_RECACTION": "ADD",
      "ISS_RECNUM": 1,
      ...
    },
    {
      "ISS_RECTYPE": "ACCOUNT",
      "ISS_RECACTION": "ADD",
      "ISS_RECNUM": 2,
      ...
    }
  ]
}

```

RSP

```

{
  "mdiResponseHeader":
  {
    "ISS_RECTYPE": "HEADER",
    "ISS_RECACTION": "RESPONSE",
    "C_FILENAME": "xapi/mdi/00005FdHhbwgvfNgR5AnCMwmKw2qbsnA",
    "ISS_SOURCESYS": "mobbank",
    "ISS_COMPANY_REGNR": "CUST123456",
    "ISS_TIMESTAMP": "20200916163525123",
    "ISS_CDBATCH_ID": 1234,
    "C_ACTIONCODE": 0,
    "C_RSPCODE": 00
  },
  "mdiResponseDetails":
  [
    {
      "ISS_RECTYPE": "CUSTOMER",
      "ISS_RECACTION": "ADD",
      "ISS_RECNUM": 1,
      "C_ACTIONCODE": 3,
      "C_RSPCODE": 08,
      "I_REJMSG": "Duplicate record",
      "ISS_COMPANY_REGNR": "CUST123456",
      "DB_CUSTOMER_TYPECODE": 0,
      "DB_CUSTOMER_CUSTCODE": "123456",
    }
  ]
}

```

```

        "DB_CUSTOMER_FIRSTNAME":"JIM",
        "DB_CUSTOMER_LASTNAME":"MORRISON",
        "DB_CUSTOMER_FATHERNAME":"GEORGE",
        "DB_CUSTOMER_CUSTCODE2":"654321",
        "DB_CUSTOMER_DATE_BIRTH":"19841226",
        "DB_CUSTOMER_HOME_COUNTRY":643,
        "DB_CUSTOMER_HOME_CITY":"MOSCOW",
        "DB_CUSTOMER_HOME_ADDRESS1":"Address 1",
        "DB_CUSTOMER_HOME_ADDRESS2":"Address 2",
        "DB_CUSTOMER_HOME_ADDRESS3":"Address 2",
        "DB_CUSTOMER_HOME_ZIP":"01234567"
    },
    {
        "ISS_RECTYPE":"CUSTOMER",
        "ISS_RECACTION":"ADD",
        "ISS_RECNUM":1,
        "C_ACTIONCODE":3,
        "C_RSPCODE":08,
        ...
    },
]
}

```

12.1.29 xkernel/calculateAcqCommission

Before transaction must be executed xkernel/initiateTransaction WS call

Cash to Card transaction example

REQ

```

{
  "cardKey":
  {
    "pan":"5375931040000103",
    "expiryDate":"2511"
  },
  "ecTxRefno":"XAPI/00005J3XLuXmgYFPhgq5kVVYftm5PVd9",
  "txnType":"TRANSF_C2C",
  "txnAmount":100.00,
  "txnCurrency":"980",
  "termCode":"AE001001",
  "crdactptID":"AE001001",
  "cvv2":"627",
  "messageFunction":0,
  "senderfirstName":"Oleksander",
  "senderlastName":"Bondarenko",
  "senderlocCity":"Kyiv",
  "senderlocStreet":"Sichovyh Striltsiv 50",
  "senderlocCountry":"804",
  "recipientCardKey":
  {
    "pan":"5375939999999993"
  },
  "recipientCountry":"804",
  "recipientCity":"Kyiv",
  "recipientStreet":"Sichovyh Striltsiv 51",
  "recipientfirstName":"Yura",
  "recipientlastName":"Bura",
  "senderFundsrc":"CC",
  "businessAppId":"TPP",
  "purchrefno":"internalid0601"
}

```

RSP

```
{
  "data": {
    "acqCommissionItems": [
      {
        "amtComm": 6.0,
        "defCommCode": "ACQ_TRANSFER",
        "amtSource": 100.0
      }
    ],
    "totalCommAmount": 6.0,
    "commCurrency": "UAH",
    "ecTxRefno": "XAPI/0000513XLuXmgYFPhgq5kVYftm5PVD9"
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

12.1.30 xmiss/ g2bHceProxy

REQ

```
{
  "hceService": "cardDataUpdateEvent",
  "data": {
    {
      "requestId": "1234",
      "oldCardRef": "uidPMrnHw6O2bAxZlGrjphBPUjxcV",
      "cardRef": "uidPMrnHw6O2bAxZlGrjphBPUjxcV",
      "oldPan": "4785300000002227",
      "newPan": "4785300000002227",
      "oldExpiryDate": "0526",
      "newExpiryDate": "0530",
      "scheme": "VISA"
    }
  }
}
```

RSP

```
{
  "data": {
    "responseld": "1234"
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "message": "Success"
  }
}
```

12.1.31 xkernel/getCompanyList

REQ:

```
{
  "data": {},
  "ordering": [
    { "column": "name", "order_function": 0 },
    { "column": "parentRegNumber", "order_function": 1 }
  ],
  "filters": [
```

```
{
  {
    "column": "regNumber",
    "values": [
      "3077701"
    ],
    "negation_operator": 0,
    "logical_operator": 0,
    "comparison_operator": 3
  },
  {
    "column": "parentRegNumber",
    "negation_operator": 0,
    "logical_operator": 0,
    "comparison_operator": 6
  }
],
"pagingAdvanced": {
  "page": 1,
  "size": 10
}
}
```

RSP:

```
{
  "pagingAdvanced": {
    "page": 1,
    "size": 10,
    "totalRecordCount": 1
  },
  "data": {
    "companyList": [
      {
        "regNumber": "3077701",
        "name": "A-MERCHANT",
        "parentRegNumber": "307770",
        "parentName": "A-PARENT"
      }
    ]
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "sessionId": "00005FdUSc4ZA4SbJtuMrcQ9jUuNeSD6"
  }
}
```

12.1.32 xkernel/getMerchant

REQ:

```
{
  "merchantId": "TEST_ME"
}
```

RSP:

```
{
  "data": {
    "merchant": {
      "merchantId": "TEST_ME",
      "merchantName": "Test Me",
      "currencyCode": "980",
      "mcc": 5555,
      "statusCode": 0,
      "parentCompany": "307770",
      "merchantType": "DFLT",
      "tstampInsert": "20251211135123917469",
      "govControl": "0",
      "addresses": [],
    }
  }
}
```

```

    },
    "status": {
      "code": "0",
      "rspcode": "00",
      "sessionId": "00005FcBugG42yPfbzLrgzBYES8CHXNg"
    }
  }
}

```

12.1.33 xkernel/getMerchantList

REQ:

```

{
  "pagingAdvanced": {
    "page": 1,
    "size": 5
  },
  "data": {},
  "ordering": [
    {
      "column": "id",
      "order_function": 1
    }
  ],
  "filters": [
    {
      "column": "merchantId",
      "values": [
        "A%"
      ],
      "negation_operator": 0,
      "logical_operator": 0,
      "comparison_operator": 3
    },
    {
      "column": "statusCode",
      "values": [
        "0"
      ],
      "negation_operator": 0,
      "logical_operator": 0,
      "comparison_operator": 0
    }
  ]
}

```

RSP:

```

{
  "pagingAdvanced": {
    "page": 1,
    "size": 5,
    "totalRecordCount": 3
  },
  "data": {
    "merchantList": [
      {
        "merchantId": "ABRET0000013264",
        "merchantName": "MERCHANT 13264",
        "statusCode": 0
      },
      {
        "merchantId": "ABRET0000004013",
        "merchantName": "ABRET",
        "statusCode": 0
      },
      {
        "merchantId": "A433",
        "merchantName": "D8 BRANCH",
        "statusCode": 0
      }
    ]
  },
  "status": {

```

```

        "code": "0",
        "rspcode": "00",
        "sessionId": "00005FcC7hMWroZ4YdXDxxwcXzVsASxd"
    }
}

```

12.1.34 xkernel/getPosTypeList

REQ:

```

{
    "data": {},
    "ordering": [
        { "column": "code", "order_function": 0 }
    ],
    "filters": [
        {
            "column": "code",
            "values": [
                "M%"
            ],
            "negation_operator": 0,
            "logical_operator": 0,
            "comparison_operator": 3
        }
    ],
    "pagingAdvanced": {
        "page": 1,
        "size": 10
    }
}

```

RSP:

```

{
    "pagingAdvanced": {
        "page": 1,
        "size": 10,
        "totalRecordCount": 2
    },
    "data": {
        "posTypeList": [
            {
                "code": "MRCH_POS01",
                "description": "Ingenico"
            },
            {
                "code": "MRCH_POS02",
                "description": "Verifone"
            }
        ]
    },
    "status": {
        "code": "0",
        "rspcode": "00",
        "message": "",
        "sessionId": "00005FdUKSjVadKomxwtoldQMEgrV4K7"
    }
}

```

12.1.35 xkernel/getPos

REQ:

```

{
    "terminalId": "TEST_ME_POS"
}

```

RSP:

```
{
  "data": {
    "pos": {
      "terminalId": "TEST_ME_POS",
      "merchantId": "TEST_ME",
      "posType": "MRCH_POS01",
      "statusCode": 0,
      "currencyCode": "980",
      "dukptCntr": 15,
      "tstampInsert": "20251216145141639138",
      "addresses": [
        {
          "addressType": "3",
          "locationName": "LOCASHOP",
          "address1": "Street str. 1",
          "city": "Rio",
          "postalCode": "99999",
          "country": "804",
          "contactEmail1": "spam@email.com"
        }
      ]
    }
  },
  "status": {
    "code": "0",
    "rspcode": "00",
    "message": "",
    "sessionId": "00005Fck43v5TaZpLWoKtEvsmjAh6G6Z"
  }
}
```

12.1.36 xkernel/getPosList

REQ:

```
{
  "page": "1",
  "pageSize": "10",
  "data": {},
  "ordering": [],
  "filters": [
    {
      "column": "merchantId",
      "values": [
        "TEST_ME"
      ],
      "negation_operator": 0,
      "logical_operator": 0,
      "comparison_operator": 0
    },
    {
      "column": "posType",
      "values": [
        "MRCH_POS*"
      ],
      "negation_operator": 0,
      "logical_operator": 0,
      "comparison_operator": 3
    }
  ]
}
```

RSP:

```
{
  "pagingAdvanced": {
    "page": 1,
    "size": 5,
    "totalRecordCount": 1
  },
}
```

```

    "data": {
      "posList": [
        {
          "terminalId": "TEST_ME_POS",
          "merchantId": "TEST_ME",
          "posType": "MRCH_POS01",
          "statusCode": 0
        }
      ]
    },
    "status": {
      "code": "0",
      "rspcode": "00",
      "message": "",
      "sessionId": "00005FdHcmnPisBaoSUGuGU4d2PqDcFJ"
    }
  }
}

```

Appendix A - Response Codes

This chapter contains list of possible values for fields status.code and status.rspcode.

There are 2 types of response codes:

- Transaction Codes - used only in response of "xkernel/getTransactionDetails" and "xkernel/getTransactionStatus".
- API Codes - may be used in response of any service.

Fields reasonClass and reason are not used in this case.

code	rspcode	Description
Approval Response codes		
0	00	approved
0	01	Honour with ID
0	02	Approved for a partial amount
0	80	ICC offline approved
0	81	ICC unable to go online, approved
0	82	ICC approved after card-initiated referral (AAR)
0	85	Not declined. Valid for all zero amount transactions
Decline Response Codes		
1	00	Do not honour
1	01	Card expired
1	02	Suspected fraud
1	03	Acceptor contact acquirer
1	04	Restricted card
1	05	Acceptor contact security
1	06	PIN tries exceeded
1	07	Acceptor contact issuer
1	08	Refer to issuer special conditions
1	09	Invalid merchant
1	10	Invalid amount
1	11	Invalid card
1	12	PIN required
1	13	Unacceptable fee
1	14	No account of specified type
1	15	Function not supported

1	16	Insufficient funds
1	17	Bad PIN
1	18	No card record
1	19	Transaction not allowed to cardholder
1	20	Transaction not allowed to terminal
1	21	Exceeds amount limit
1	22	Security violation
1	23	Exceeds frequency limit
1	24	Violation of law
1	25	Card not effective
1	26	Invalid PIN block
1	27	PIN length error
1	28	PIN key sync error
1	29	Suspect counterfeit
1	65	Transaction does not fulfill AML requirement
1	70	Cardholder contact issuer
1	71	PIN not changed
1	75	Exceeds limit: PIN required
1	76	Exceeds limit: Additional customer authentication required
1	77	Invalid currency code
1	78	ICC decline, after card-initiated referral (AAR)
1	79	PIN Change declined (New PIN unsafe)
1	80	Amount not divisible
1	81	Amount too big for ATM
1	82	ATM out of cash
1	83	No chequing account
1	84	New PINs entered differ
1	85	CVV validation error
1	86	pre-auth time limit exceeded
1	87	Card has been destroyed
1	88	Restricted card. No funds.
1	89	Foreign exchange not available
1	90	Transaction count limit exceeded
1	91	System overload – retry later
1	92	Statement data not available
1	93	Pin tries exceeded
1	94	No balance data available
1	95	ICC offline declined
1	96	ICC unable to go online, declined
1	97	ICC card authentication failed
1	98	No receipt available on ATM
1	99	No TUC number available
		Pickup Response codes
2	00	Decline, Pickup card
2	01	Not supported by receiver
2	02	Suspected fraud
2	03	Acceptor contact acquirer
2	04	Restricted card
2	05	Acceptor contact security
2	06	PIN tries exceeded
2	07	Special conditions

2	08	Lost card
2	09	Stolen card
2	10	Suspect counterfeit
2	85	CVV validation error
		Error responses
9	02	Invalid transaction
9	03	Re-enter transaction
9	04	Format error
9	05	Acquirer not supported
9	06	Cutover in progress
9	07	issuer/switch inoperative
9	08	Destination not found
9	09	System malfunction
9	10	Issuer signed off
9	11	Issuer timed out
9	12	Issuer unavailable
9	13	Duplicate transaction
9	14	Unable to trace original transaction
9	15	Reconciliation/checkpoint error
9	16	MAC incorrect
9	17	MAC key sync error
9	18	No comms keys available
9	19	Encryption key sync error
9	20	Security error, try again
9	21	Security error, no action
9	22	Message out of sequence
9	23	Request in progress
9	40	Invalid transaction date. (Correct date and re-send transaction.)
9	50	disagreement
9	51	Card expired
9	52	Suspected fraud
9	53	Restricted card
9	54	Invalid merchant
9	55	Invalid amount
9	56	Invalid card number
9	57	Unacceptable fee
9	58	Insufficient funds
9	59	Bad PIN
9	60	Trans not allowed to cardholder
9	61	Trans not allowed to terminal
9	62	Exceeds amount limit
9	63	Security violation
9	64	Exceeds frequency limit
9	65	Violation of law
9	66	reconciliation/chkpnt error
9	67	MAC incorrect
9	68	MAC key sync error
9	69	No comms key available
9	70	Encryption key sync error
9	71	Security error, try again

9	72	Security error, no action
9	73	Message out of sequence

API Codes

These codes are used in responses of any API service.

Fields reasonClass and reason are not used in this case.

code	rspcode	Description	reasonClass	reason
		Approval Response codes		
0	00	Approved		
		Error responses		
9	04	Format error		
9	09	System malfunction		
		Service level error codes		
D	00	Card not found		
D	01	Card is in invalid status.		
D	02	Status change transition not allowed.		
D	03	Card not linked to Account		
D	04	Transaction not found		
D	05	Lkey not found		
D	06	Invalid ecTxRefno		
D	07	Invalid account		
C	32	Record doesn't exist or already modified by other user		
C	33	Record already exists		
C	34	Failed to insert record		
C	35	Failed to update record		
C	36	Failed to delete record		
C	37	Failed to delete record, child records exist		
C	38	Invalid session		
C	39	Permission violation		
C	40	Failed to insert record		
C	41	Failed to insert record, such identified already exist		
C	42	Clear PAN not allowed in request		
C	43	CVV2 not allowed in request		
C	44	Executed with errors.		

Appendix B - Transaction Status Codes

code	Description	In progress	Approved	Rejected / Reversed	Skipped
1	In progress (received in online)	X			
2	Transaction complete, rejected			X	
3	Transaction complete, approved		X ¹		
4	Transaction was approved, but full reversal received			X	
5	Transaction was approved, but partial		X ²	X ²	

	reversal received				
6	Advice logged, but not processed	X			
7	Advice logged, processed and rejected			X	
8	Advice logged, processed and approved		X		
9	Transaction logged for offline	X			
10	Offline - transaction applied on account		X		
11	Offline - transaction failed to apply on account			X	
12	Completed, rejected, got rev			X	
13	Completed, rejected, got part rev			X	
14	In progress, got rev	X			
15	In progress, got part rev	X			
16	First attempt continue...	X			
17	Advice logged, no proceed				X

Transaction status changes:

- In progress - non final status, may change.
- Approved, Rejected/Reversed, Skipped - final statuses, will not change:

Notes:

1. Status 3 is final status for single transaction, but it may be updated after a while in case when reversal received for this transaction!
2. In case of partial reversal part of transaction amount is still approved and part of amount is reversed. It is also possible that multiple Partial Reversals are received on single original transaction until full amount is reversed. Approved and Reversed amounts may be calculated as:
 - a. $\text{ApprovedAmount} = \text{OriginalAmount} - \text{Sum of ReversalAmounts}$
 - b. $\text{ReversedAmount} = \text{Sum of ReversalAmounts}$

Following statuses may be present for transactions initiated by XAPI services, depending on transaction type (txnType):

Service	txnType	Status codes
xkernel/ authorizeTransaction	SALES, CASH, DEPOSIT, BALANCE, TRANSF_C2C, TRANSF_H2C, TRANSF_C2A, TRANSF_A2C, ACCVER, REFUND, PREAUTH	1 - In progress (received in online) 2 - Transaction complete, rejected 3 - Transaction complete, approved 4 - Transaction was approved, but full reversal received 5 - Transaction was approved, but partial reversal received 12 - Completed, rejected, got rev 13 - Completed, rejected, got part rev 14 - In progress, got rev 15 - In progress, got part rev
xkernel/ authorizeTransaction	SALES_COM	6 - Advice logged, but not processed 7 - Advice logged, processed and rejected 8 - Advice logged, processed and approved 16 - First attempt continue...

xkernel/ reverseTransaction	any	6 - Advice logged, but not processed 7 - Advice logged, processed and rejected 8 - Advice logged, processed and approved 16 - First attempt continue... 17 - Advice logged, no proceed <i>Note: Status 17 is used in case when reversal is requested for transaction which do not need to be reversed. For example - reversal of rejected transaction.</i>
--------------------------------	-----	---

Appendix C - Message Reason Codes

code	Description
1000-1499 reason for advice	
1000	Stand-in req. by issuer
1001	Issuer signed off
1002	Issuer timed out on request
1003	Issuer unavailable
1004	Terminal processed
1005	ICC processed
1006	Below floor limit
1007	Stand-in req by acq
1377	Prev revd trn did complete
1378	Clearing authorised trn
1379	Clearing un-authorised trn
1380	Prev revd adj did complete
1381	Unsolicited cashin deposit
1382	Original amount incorrect
1383	Duplicate transmission
1384	Reconcile error
1385	System malfunction
1386	Network down
1387	Issuer sent invalid response
1388	Issuer sent system malf rsp
1389	Access Point (VAP/MIP) error at issuer
1391	PAN mapping failed
1392	PAN mapping at Network failed
1393	Invalid merchant for txn
1394	Txn exceeds merchant limit
1395	Txn exceeds MCC limit
1396	Some network txn check failed
1399	Authentication Advice to Issuer
1400	Chip Offline Advice to Issuer
1401	POS Notification Installments Info
40xx used in Reversal	
4000	Customer cancellation
4001	No action taken
4002	Suspected malfunction

4003	Format error, no action taken
4004	Completed partially
4005	Original amount incorrect
4006	Response received too late
4007	Device unable to complete
4008	Deposit out of balance
4009	No check in envelope
4010	Payment out of balance
4011	Dep out of bal/content applid
4012	Payment out/bal, content applid
4013	Message not delivered to POS
4014	Malfunction, card captured
4015	Malfunction, card returned
4016	Malfunction, T3 not updated
4017	Malfunction, no cash dispense
4018	Cash take timeout, no cash
4019	Card take timeout, no cash
4020	Invalid response, no action
4021	Timeout waiting for response
4351	ADJ: customer cancelled
4352	ADJ: partial disp/wrong amt
4353	ADJ: partial return
4354	ADJ: acquirer error
4355	No confirm from POS
4356	VISA - possible duplicate
4357	VISA - duplicate txn
4358	VISA - BII dup txn not representment
4359	ISO93Host - Auth reversal
4360	Reversal of Expired Auth
4361	Suspect reversal
4362	Reversal of suspected fraud
4363	AFT Rev because OCT was rejected
4364	AFT DR or CR Adjustment
4365	OCT DR or CR Adjustment
4366	Auth reversal, for Deferred

Appendix D Money Transfer Data - Business Application Identifier

businessAppld Value	VISA Value (Field 104, Dataset Hex 57 - Business Application Identifier)	MC Value (DE 48, Subelement 77 - Transaction Type Identifier)
OGG – Online Gambling	OG - Online gambling payout	C04 - Gaming Re-pay
GPO – Gambling payout	GP - Gambling payout (other than online gambling)	C04 - Gaming Re-pay
TPP - Person-to-Person	PP - Person-to-Person	C07 – General Person-to-Person Transfer
TAA - Account-to-Account Transfers	AA – Account-to-Account	C52 - General Transfer to Own Account
TAC - Agent Cash Out	PP - Person-to-Person	C53 - Agent Cash Out

businessApplId Value	VISA Value (Field 104, Dataset Hex 57 - Business Application Identifier)	MC Value (DE 48, Subelement 77 - Transaction Type Identifier)
TCB – Consumer Bill Payment	CP - Card bill payment	C54 - Payment of Own Credit Card Bill
TCP – Card Bill Payment	CP - Card bill payment	C54 - Payment of Own Credit Card Bill
TBD - Business Disbursement	FD - Funds disbursement (general)	C55 - Business Disbursement
TGD - Government/Non-profit Disbursement	GD - Government disbursement	C56 - Government/Non-profit Disbursement
TMS - Acquirer Merchant Settlement	MD - Merchant disbursement	C57 - Rapid Merchant Settlement
TCA - Cash2ATM	CD - Cash deposit	C58 - Cash in at ATM
TCC - Cash2Card	CD - Cash deposit	C59 - Cash in at POS
TBB - Business to Business Transfer	BB - Business-to-Business	C65 - General Business-to-Business Transfer
TBI - Financial Institution offered Bank-Initiated P2P Money Transfer	BI - Money transfer-bank-initiated	C07 - General Person-to-Person Transfer
TFT - Funds Transfer	FT - Funds Transfer	C07 - General Person-to-Person Transfer
TFD - Funds disbursement (general)	FD - Funds disbursement (general)	
TWT - Staged Digital Wallet	WT - Staged Digital Wallet	C07 - General Person-to-Person Transfer
TLA – Liquid Assets	LA – Liquid Assets	C65 - General Business-to-Business Transfer
TMI – Merchant Initiated OCT for Faster Refund	MI – Merchant Initiated OCT for Faster Refund	C65 - General Business-to-Business Transfer
TCR – Cryptocurrency Transaction	FD - Funds disbursement (general)	P70 – Cryptocurrency
TVA – Visa Accept	VA – Visa Accept	
TCS -Speciality merchant([Mastercard use only])	FD - Funds disbursement (general)	P72 - Speciality merchant
TCD - Direct purchase of digital currencies	LA - Liquid Assets	P76 - Direct purchase of fully fiat-backed stablecoins and central bank digital currencies (CBDCs)

Appendix E - Money Transfer Data - Funding source

fundingSource Value	VISA Values (Field 104, Dataset Hex 5F, Tag 08 -Source of Funds)	MC Values (DE108, subelement 3, Subfield 03 - Funding Source)
CC – Card	01 – Credit / 02 – Debit / 03 - Prepaid depending of ARDEFdata for sender card product	01 – Credit / 02 - Debit / 03 - Prepaid
CH – Cash	04 – Cash	06 – Cash
DA - Debit/deposit access accounts	05 - Debit/deposit access accounts	04 - Debit/deposit access accounts
CA - Credit accounts	06 - Credit accounts	01 - Credit
MC - Mobile Money Account	05 - Debit/deposit access accounts	05 - Mobile Money Account

Appendix F – Sub-transaction Code

txnCode	sub_txnCode	Description
Any code	00 or empty	Default logic for this txnCode
00	02	Incremental authorisation
00	03	Purchase of Cryptocurrency
00	10	Network Instalment Purchase
00	99	Unknown
10	01	Account Funding transaction
26	01	OCT/Fast Funds
47	01	Acquirer Card-to-Card MT
47	02	Acquirer Cash-to-Card MT
47	03	Acquirer Card-to-Account MT
47	04	Acquirer Account-to-Card MT

Appendix G - PIN block formats

ANSI X9.8

Format 01 is the ISO 9564-1 Format 0, equivalent to the ANSI X9.8 Format 0 PIN Block, and can only be encrypted using a DES/3DES key.

The format combines the customer PIN and account number as follows:

- A 16-digit block is made from the digit 0, the length of the PIN, the PIN, and a pad character (hexadecimal F). For example, for the 5-digit PIN 92389, the block is:

0592 389F FFFF FFFF

- Another 16-digit block is made from four zeros and the 12 right-most digits of the account number, excluding the check digit. For example, for the 13-digit account number 4000 0012 3456 2, where the check digit is 2, the block is:

0000 4000 0012 3456

- The two blocks are exclusive-OR added:

	05	92	38	9F	FF	FF	FF	FF
	00	00	40	00	00	12	34	56
PIN block:	05	92	78	9F	FF	ED	CB	A9

ISO 9564-1 Format 1

Format 05 is the ISO 9564-1 Format 1 PIN Block, and can only be encrypted using a DES/3DES key.

The PIN block is represented by the following 16 hexadecimal values:

1NPP..P R . . R

Where:

N is the PIN length (4 - C),

PP..P is the N-digit PIN,

R . . R is random padding.

The following validity checks are carried out on incoming Format 05 PIN blocks:

- The first character of the PIN block has value 1.
- The PIN digits (in positions 3 - (N+2)) are in the range 0 to 9.
- The second character (N) is in the hexadecimal range 4 - C.

13 Appendix H – Encryption Examples

Following chapter contains examples for encryption related data preparation. Examples where prepared in Linux environment using openssl utility. Openssl is well known and widely used utility, but G2B XAPI does not force using openssl, other encryption utilities and libraries may be used.

In these examples there are few steps (like conversion between hex and binary, conversion of key formats etc.) which are necessary because of functionality of openssl. It might be different when using other utilities!

13.1 List of example files

All files used in these examples may be found in separate file:

"D8_G2B_Processing_External_API_Encryption_Examples_v1.0.zip"

File	Description
getLKey/	
SIG_priv_key.pem	SIG-priv, generated by client.
SIG_pub_key_pkcs1.der	SIG-pub, generated by client.
SESS_pub_key_pkcs1.der	SESS-priv, generated by client.
SESS_priv_key.pem	SESS-pub, generated by client.
SESS_pub_digest.bin	Digest of SESS-pub retrieved using SIG-priv, generated by client.
SESS_des3_key_clear.bin	SESS-des, encrypted under SESS-pub by G2B, received in WS response.

SESS_des3_key_under_RSA.bin	SESS-des, clear value, decrypted by SESS-priv.
encryptedData.data.bin	encryptedData.data converted from HEX to binary.
setPIN/	
xapi_global_pubkey.der	XAPI-pub RSA key in HEX format, generated by G2B.
xapi_global_rsa_hex.txt	XAPI-pub RSA key, in der format. This file will be delivered to client by G2B.
xapi_global_rsa_pkcs1.der	XAPI-pub RSA key converted to Public key format, required for openssl operations.
setPin_deskey_clear.bin	SESS-des, HEX, generated by client.
setPin_deskey_clear.txt	SESS-des, converted into binary format.
setPin_deskey_under_RSA.bin	SESS-des, encrypted under XAPI-pub.
PIN_block_clear.bin	PIN-block, converted into binary format.
PIN_block_under_DES.bin	PIN-block, encrypted under SESS-des, binary format.

13.2 Example of encryptedData in response (getLKey)

```
//----- configuration client side -----

// gen SIG-priv and SIG-pub
$ openssl genrsa -out SIG_priv_key.pem 2048
$ openssl rsa -in SIG_priv_key.pem -RSAPublicKey_out -outform DER -out SIG_pub_key_pkcs1.der

//----- WS request -----

// generate SESS-priv and SESS-pub
$ openssl genrsa -out SESS_priv_key.pem 2048
$ openssl rsa -in SESS_priv_key.pem -RSAPublicKey_out -outform DER -out SESS_pub_key_pkcs1.der

// generate digest of SESS-pub using SIG-priv
$ openssl dgst -sha256 -sign SIG_priv_key.pem -out SESS_pub_digest.bin SESS_pub_key_pkcs1.der

// Convert SESS-pub into HEX
$ cat SESS_pub_key_pkcs1.der | hexdump -ve '1/1 "%.2x"'
3082010a0282010100a3ab6b6d7c7d90af9646fc59b0d15ad2ea1af30266629558e2f3166bab21c5b953b947f07889
bf98f540bae73c58878ec9e22c917cac2c546032ec7c846e556e7fdd29c47fa77ee65968b3844605bd5a0ae8e37aa0
f4a153e5aa0f9b998cf055b8efdec07daf1803dcd083d486347957591562ca9861bd9536d9a424076b790b55c5c211
8ef7967be6869e5c6c8295d2daac17aa5edbc0e35d816b5de8c8dbf08eaaebddacdb829064ad9ab8119faf150a5eda
190a5d044e6868d04a32e0d791437fe308af1ad9967ea0282b04f23d446a1115defca6d47376248a72dc602558aa92
b8cdd0b346b3e5bc2205e4c6ff271c9fd813a48392b9a04ba5f434c732110203010001

// Convert digest of SESS-pub into HEX
$ cat SESS_pub_digest.bin | hexdump -ve '1/1 "%.2x"'
d45b635fed9f3aa0053b31f86fffb903b50863441f5c601ca5e186021cc82aec34f753d72a8a0f968fd35c36f1216a9
f879119a3733d23ced3be8a615160faf7913b74996fd2069fa14f9733632b81df95398427c9f491756c48b91f43a45
f74d56619b4b4bd93eafe3b060494ed7d81bf688491897ca3b3e98b10c6bbf9fc982a1ea9a832bb49bb766ff815c9b
f14222bacf3584ce40918f5a886006fe72b7d46fe6934cb31bceb2c2da9127a358be9b80994edb3279b61ee229c3db
f8184310d811307744057aea0bbb93573dd4322bd360a0386849db01ec56bd3b8efbca53c8695f91095e1a6c5042de
41efcc99ccafd78f844d2ddb695902bdd5e861f86f

// Create WS request and call WS
REQUEST:
{
  "cardKey":
  {
    "lkeyId":21
  },
  "rsaKeyBlock":
  {
    "publicKey":"3082010A0282010100A3AB6B6D7C7D90AF9646FC59B0D15AD2EA1AF30266629558E2F3166BAB21C5B
953B947F07889BF98F540BAE73C58878EC9E22C917CAC2C546032EC7C846E556E7FDD29C47FA77EE65968B3844605B
D5A0AE8E37AA0F4A153E5AA0F9B998CF055B8EFDEC07DAF1803DCD083D486347957591562CA9861BD9536D9A424076
B790B55C5C2118EF7967BE6869E5C6C8295D2DAAC17AA5EDBC0E35D816B5DE8C8DBF08EAAEBDDACDB829064AD9AB81
19FAF150A5EDA190A5D044E6868D04A32E0D791437FE308AF1AD9967EA0282B04F23D446A1115DEFC6D47376248A7
2DC602558AA92B8CDD0B346B3E5BC2205E4C6FF271C9FD813A48392B9A04BA5F434C732110203010001",
    "publicKeyDigest":"D45B635FED9F3AA0053B31F86FFFB903B50863441F5C601CA5E186021CC82AEC34F753D72A8A
0F968FD35C36F1216A9F879119A3733D23CED3BE8A615160FAF7913B74996FD2069FA14F9733632B81DF95398427C9
F491756C48B91F43A45F74D56619B4B4BD93EAFE3B060494ED7D81BF688491897CA3B3E98B10C6BBF9FC982A1EA9A8
32BB49BB766FF815C9BF14222BACF3584CE40918F5A886006FE72B7D46FE6934CB31BCEB2C2DA9127A358BE9B80994
```

```
EDB3279B61EE229C3DBF8184310D811307744057AEA0BBB93573DD4322BD360A0386849DB01EC56BD3B8EFBCA53C86
95F91095E1A6C5042DE41EFCC99CCAFD78F844D2DDB695902BDD5E861F86F"
```

```
    }
}

RESPONSE:
{
  "data": {
    "lkey": {
      "maskedPan": "520474_1002",
      "pan": "5204740000001002",
      "sequence": 0,
      "lkeyId": 21,
      "lkeyAlias": "TE2C10LKEYALIAS0017"
    },
    "encryptedData": {
      "data": "44C7494E0135FAC7469DA15F2F24039683DCE8AC11C4E8F2A6F970C87D6C43BD",
      "sessionDESKey":
"4FFC5038051077A614D3ED48199FEB890B2CFDC4FB71FED6005BC4605CC636740A47B40DDD2ABC7E8E3FF49425633
F2BE3D424CC31A181844E0B9D19685B3518561A90B76249F193C806E91A10DBB28FFA06DA26A4C4A141AAE48D6045A
2E56DC303259B3E462306D9A507733B50268B3551D30DB92B486B75D1900288B4EA80F659C530C4F459971507B2C45
42D70AD191D789AE1FAE2908E4E31A02397D8F42D7D810EB7EC4E106B0BF0794F7E7522642F11C26A5E5E499069BC0
435ACED869E89D5C5E19522D2683E01F2B913F68FBB39800FF872F1332E50CEE0866A8599717D7CEE2B1911BD8FAC8
59E11741004772285BEE3B922D4168AA5DD27BAC738"
    },
  },
  "status": {
    "code": "0",
    "rspcode": "00"
  }
}
```

```
// Save "encryptedData.sessionDESKey" in binary file
$ echo
"4FFC5038051077A614D3ED48199FEB890B2CFDC4FB71FED6005BC4605CC636740A47B40DDD2ABC7E8E3FF49425633
F2BE3D424CC31A181844E0B9D19685B3518561A90B76249F193C806E91A10DBB28FFA06DA26A4C4A141AAE48D6045A
2E56DC303259B3E462306D9A507733B50268B3551D30DB92B486B75D1900288B4EA80F659C530C4F459971507B2C45
42D70AD191D789AE1FAE2908E4E31A02397D8F42D7D810EB7EC4E106B0BF0794F7E7522642F11C26A5E5E499069BC0
435ACED869E89D5C5E19522D2683E01F2B913F68FBB39800FF872F1332E50CEE0866A8599717D7CEE2B1911BD8FAC8
59E11741004772285BEE3B922D4168AA5DD27BAC738" | xxd -p -r > SESS_des3_key_under_RSA.bin

// Decrypt "encryptedData.sessionDESKey"
$ openssl rsautl -decrypt -inkey SESS_priv_key.pem -in SESS_des3_key_under_RSA.bin -out
SESS_des3_key_clear.bin

// Convert "encryptedData.sessionDESKey" to HEX
$ xxd -p SESS_des3_key_clear.bin
ec7a8f85b52513e502588f92fe389ef7

// Convert "encryptedData.data" into binary format
$ echo "44C7494E0135FAC7469DA15F2F24039683DCE8AC11C4E8F2A6F970C87D6C43BD" | xxd -p -r >
encryptedData.data.bin

// Decrypt "encryptedData.data"
$ openssl enc -d -des-ede -in encryptedData.data.bin -nosalt -nopad -K
ec7a8f85b52513e502588f92fe389ef7
{"pan": "5204740000001002"}
```

13.3 Example of PIN data in request (setPIN)

```
//----- configuration processing side side -----

// Save XAPI-pub into file and convert into binary format
$ echo -n
"3082010802820101008BF5150FDDF828A2F2710DDEB92A5583286BE1589910A6F4E94E34D0FE9A4CA1802654326A5
BDD363DC2AA1FA4F88BF87BD87464A3C3173B7A50EFB39C1A7248536881E5C3E0DA0F280659F52AB12C786191ADBF
B008B29D709C7BC6183EA1322415FDC3574923EA5C5C9018F245E26FB024240B4DCEA31405AD50DC54C13313093FA7
2EFD792BAB2BB1CAFA31687E260C284A30B7F36A899D61CA59A65C318CE2D70626D5918F885472ACA932EB133C3738
7E4A92EA7A0F67199AFC2D56B67268F9D9B8DE863CF8E43B94D3CB8488CE7878B5194B1C24F7D12D98F4C9CE7CACAD
2001F6E70E2C104B64312AC8D10B6DFAB9F410C0836EA057170FAB88A8B19020111" > xapi_global_rsa_hex.txt
$ xxd -p -r xapi_global_rsa_hex.txt xapi_global_rsa_pkcs1.der

//----- client side -----
```

```
// Generate SESS-des
// For testing purposes we will use well known 3DES Double length key with Odd parity:
0123456789ABCDEF0123456789ABCDEF
$ echo -n "0123456789ABCDEF0123456789ABCDEF" > setPin_deskey_clear.txt

// Convert SESS-des key into binary format
$ xxd -p -r setPin_deskey_clear.txt setPin_deskey_clear.bin

// Convert XAPI-pub DER file from RSA Public Key format into Public Key format (because this
is format required by openssl)
$ openssl rsa -in xapi_global_rsa_pkcs1.der -inform DER -pubin -out xapi_global_pubkey.der -
outform DER -RSAPublicKey_in

// Encrypt SESS-des key under XAPI-pub and convert into hex
$ openssl rsautl -encrypt -inkey xapi_global_pubkey.der -pubin -keyform DER -in
setPin_deskey_clear.bin -out setPin_deskey_under_RSA.bin
$ xxd -p -c 10000 < setPin_deskey_under_RSA.bin
3d6cd13b76fab2fd4aa5564e6d5baf141f597076e5a988d360758f16bde311d10d69c0cae8e5829d3d5311c423c549
72058ca64fceeec0c25d9903e52b1eedb43e0cc536e5fc752e528e93765282f9da52cdf20c846fa8e411b882db9b32
6735e490d98d98b1f0ffdedb9575852075e33b3c5de3aa6f1088a12377518ac38cffc7aa8c28e88fe7712aae419efd
b82bcf1bdc4e83772ad3c3cc765c8475a981b6641c024ee9ab85a67c099a7a5c08446ba8222f1be3972a1d079f75b7
266e3363ee6f01eed2131a753f847705b1ef30ff8632b0b757cb33ac5be4f746cc0e48a29b49c1459fca86dfb39039
d691e71187979e8f6b6aa760d8923d7744c1d851e

// Prepare PIN-block
PAN: 5375939999999993
PIN: 1234

Calculate PIN block:
-----
041234FFFFFFFFF
xor 000059399999999
-----
04126DC666666666

// Convert PIN-block into binary format
$ echo "04126DC666666666" | xxd -p -r > PIN_block_clear.bin

// Encrypt PIN-block with SESS-des and convert into hex
$ openssl enc -e -des-ede -in PIN_block_clear.bin -nosalt -nopad -K
0123456789ABCDEF0123456789ABCDEF -out PIN_block_under_DES.bin
$ xxd -p -c 10000 < PIN_block_under_DES.bin
5a8bf4651adda2d4

// Create WS request and call WS
REQUEST:

RESPONSE:
{
  "cardKey":
  {
    "pan": "5375939999999993",
    "expiryDate": "2701"
  },
  "pinKeyUnderRSA": "3D6CD13B76FAB2FD4AA5564E6D5BAF141F597076E5A988D360758F16BDE311D10D69C0CAE8E5
829D3D5311C423C54972058CA64FCEECC0C25D9903E52B1EEDB43E0CC536E5FC752E528E93765282F9DA52CDF20C84
6FA8E411B882DB9B326735E490D98D98B1F0FFDEDB9575852075E33B3C5DE3AA6F1088A12377518AC38CFFC7AA8C28
E88FE7712AAE419EFDB82BCF1BDC4E83772AD3C3CC765C8475A981B6641C024EE9AB85A67C099A7A5C08446BA8222F
1BE3972A1D079F75B7266E3363EE6F01EED2131A753F847705B1EF30FF8632B0B757CB33AC5BE4F746CC0E48A29B49
C1459FCA86DFB39039D691E71187979E8F6B6BAA760D8923D7744C1D851E",
  "pinBlock": "5A8BF4651ADDA2D4"
}
```

13.4 Example of PIN data in request (setPIN) with OAEP encryption (April 2025)

```
// Generate RSA pair on HSM
$ genkeyrsa -r "XAPI/SETPIN_GEL" -f XAPI_pub_key.der -l 2048 -t ED -c 0 -i 0 -s 0

$ xxd -p -u -c 10000 < XAPI_pub_key.der
3082010A0282010100B6CA3A669D3B6999A9871297483BA50FD07849D5A6787E1324A3480301B63171DC466432D6B0
EA610DE4B260A1BDA6E448E9ACF277F576EDC112A87D2D9B3AC66E49DAD4F7A2442D5E8B4EC502B70CD4D742E98E0A
A4FA33533BC56B3757C824FBF06925FCF934FEB699547B9E0FF56C4653BE25CBE06BD0D07F1059FF7498EA6BB59869
```

```
34C6CBF72E18BDA73D3637193F1FED833BAD45F7F49EDBC1036E69B4B46CD34FE8E486196C4616D553283DD26EC8C2
53EEA24AE98B20D3EA9C49327CAAFD473974AA82B35A08D8515B46EAE82A95513A8DCDB64619645841A204BC694FF5
51C0BF892D702FA218F0C2F74E128C3EB3FA5780D34D7109F55E6BC200050203010001
```

```
//----- client side -----

// Generate SESS-des
// For testing purposes we will use well known 3DES Double length key with Odd parity:
0123456789ABCDEF0123456789ABCDEF
$ echo -n "0123456789ABCDEF0123456789ABCDEF" > setPin_deskey_clear.txt

// Convert SESS-des key into binary format
$ xxd -p -r setPin_deskey_clear.txt setPin_deskey_clear.bin

// Convert Public key from RSA Public key format to the one used in Openssl
$ openssl rsa -in XAPI_pub_key.der -inform DER -pubin -out XAPI_pub_key_OPENSSL.der -outform
DER -RSAPublicKey_in

$ xxd -p -u -c 10000 < XAPI_pub_key_OPENSSL.der
30820122300D06092A864886F70D010105000382010F003082010A0282010100B6CA3A669D3B6999A9871297483B
A50FD07849D5A6787E1324A3480301B63171DC466432D6B0EA610DE4B260A1BDA6E448E9ACF277F576EDC112A87D2D
9B3AC66E49DAD4F7A2442D5E8B4EC502B70CD4D742E98E0AA4FA33533BC56B3757C824FBF06925FCF934FEB699547B
9E0FF56C4653BE25CBE06BD0D07F1059FF7498EA6BB5986934C6CBF72E18BDA73D3637193F1FED833BAD45F7F49EDB
C1036E69B4B46CD34FE8E486196C4616D553283DD26EC8C253EEA24AE98B20D3EA9C49327CAAFD473974AA82B35A08
D8515B46EAE82A95513A8DCDB64619645841A204BC694FF551C0BF892D702FA218F0C2F74E128C3EB3FA5780D34D71
09F55E6BC200050203010001

// Encrypt SESS-des key under XAPI-pub using OAEP and convert into hex
$ openssl pkeyutl -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256 -encrypt -inkey
XAPI_pub_key_OPENSSL.der -pubin -keyform DER -in setPin_deskey_clear.bin -out
setPin_deskey_under_RSA_OAEP.bin

$ xxd -p -u -c 10000 < setPin_deskey_under_RSA_OAEP.bin
60F9A76DD03D63497DFDC796591E94526E20455005B5E6FA5CB8D84F3AD0A76A915EF609F0350AB7FA4CBED9E34AD7
0A9BFF5DFD7C8F37B590DFFA56BC61F222DAB5F8A10D763FE4B5CDA5AC61B32F7875F0A712B2A558B8E317AFAE8314
B79EBBDFFC0534789940F75601885AB321F2052A18A652D0BDA8EA6973F91C32354DFDE409BCDA09235511ECB425BE
652EF3CDB5A802C851236325D197BE3BF4D6283CDD56E6FC41D2F2B268FB7D3064DD59CC5BD5ECF6B956ECDE62A8D6
15C743AD75EF84688EE9A5CC34B477A492B5B518BFAA0C60D13CB92FC3669C5BB1943A0AB1A5E0922543A0F70BF566
76CAD3D9B04A910CDB34B32C27671E9DDCEB4D0FFA

// Prepare PIN-block
PAN: 99988800000000018
PIN: 7007

Calculate PIN block:
-----
047007FFFFFFFFF
xor 0000888000000001
-----
04708F7FFFFFFFFFE

// Convert PIN-block into binary format
$ echo "04708F7FFFFFFFFFE" | xxd -p -r > PIN_block_clear.bin

// Encrypt PIN-block with SESS-des and convert into hex
$ openssl enc -e -des-ede -in PIN_block_clear.bin -nosalt -nopad -K
0123456789ABCDEF0123456789ABCDEF -out PIN_block_under_DES.bin

$ xxd -p -u -c 10000 < PIN_block_under_DES.bin
89DFC049D1769C5B

// Create WS request and call WS
REQUEST:

RESPONSE:
{
  "cardKey":
  {
    "pan": "99988800000000018",
```

```

    "expiryDate": "2011"
  },
  "pinKeyUnderRSA": "60F9A76DD03D63497DFDC796591E94526E20455005B5E6FA5CB8D84F3AD0A76A915EF609F035
0AB7FA4CBED9E34AD70A9BFF5DFD7C8F37B590DFFA56BC61F222DAB5F8A10D763FE4B5CDA5AC61B32F7875F0A712B2
A558B8E317AFAE8314B79EBBDFCC0534789940F75601885AB321F2052A18A652D0BDA8EA6973F91C32354DFDE409BC
DA09235511ECB425BE652EF3CDB5A802C851236325D197BE3BF4D6283CDD56E6FC41D2F2B268FB7D3064DD59CC5BD5
ECF6B956ECDE62A8D615C743AD75EF84688EE9A5CC34B477A492B5B518BFAA0C60D13CB92FC3669C5BB1943A0AB1A5
E0922543A0F70BF56676CAD3D9B04A910CDB34B32C27671E9DDCEB4D0FFA",
  "pinBlock": "89DFC049D1769C5B"
}

```

Appendix I – Recurring transaction types.

Recurring transaction types. Different values may be used depending on field "recurrInd" values.

Allowed recurrInd values	Type	Description	Mapping MC field DE48.22.5	Mapping to VISA field F126.13
O,C	C101	CIT Unscheduled Credential on File	C101	C
O,C	C102	CIT Standing Order (variable amount and fixed frequency)	C102	R
O,C	C103	CIT Subscription (fixed amount and frequency)	C103	R
O,M	M101	MIT Unscheduled Credential on File	M101	C
O,M	M102	MIT Standing Order (variable amount and fixed frequency)	M102	R
O,M	M103	MIT Subscription (fixed amount and frequency)	M103	R

Appendix J – Transaction codes

Code	Description
00	Goods and services
01	Withdrawal
02	Debit Adjustment
09	Goods and services with cash disbursement
10	Non-cash instrument
11	Quasi cash
17	Goods/sale with tip
20	Refund
21	Deposits
22	Credit Adjustment
26	Cardholder funds transfer
28	Cash Deposit (cash in)
30	Available Funds Enquiry
31	Balance Enquiry
47	Money Transfer

50	Bill payment
58	Payment external bank
90	PIN Change
93	Customer Authentication
94	PIN unblock (EMV only)
95	Application unblock (EMV only)
96	Address Verification

Appendix K – Message Class

Code	Description
1	Authorization
2	Financial
3	File update
4	Reversal/ Chargeback
5	Reconciliation
6	Administrative
7	Fee collection
8	Network management

Appendix L – Message Function

Code	Description
0	Request
1	Request response
2	Advice
3	Advice response
4	Notification

Appendix M – Function Code

Code	Description
	1xx used in AUTH msgs
100	orig auth, amount accurate
101	org auth, amount estimated
102	rpl auth, amount accurate
103	rpl auth, amt estimated
104	resubmission, amount accurate
105	resubmission, amount estim
106	supplement, amount accurate
107	supplement, amount estim
108	inquiry
180	acc/addr verification only
181	telecode verification only

182	customer verification only
	2xx used in FIN msgs
200	original financial req/adv
201	previously auth d, amt same
202	prev auth d, amt differs
203	resubmission of prev denied
204	resubmission of prev reversed
205	first representation
206	second representation
207	third or subsequent repres
282	partial repres (ECCF)
	3xx used in File Update
300	unassigned
301	add record
302	change record
303	delete record
304	replace record
305	inquiry
306	replace file
307	add file
308	delete file
309	card admin
	40x used in Reversal
400	full reversal, txn not cmplt
401	part reversal, txn not cmplt
445	full representment rev
446	partial representment rev
	45x used in Chargeback
450	full chargeback, first
451	full chargeback, second
452	full chargeback, 3rd & subs
453	part chargeback, first
454	part chargeback, second
455	part chargeback, 3rd & subs
490	chargeback reversal, 1
491	chargeback reversal, 2
492	chargeback reversal, 3
493	part chargeback reversal, 1
494	part chargeback reversal, 2
495	part chargeback reversal, 3
	5xx used in Recon Totals
500	final reconciliation
501	checkpoint reconciliation
502	final rec, specified curr
503	checkpoint rec, spec curr
504	reconciliation tots request

570	272 Acquirer GIV totals
571	273 Acquirer GIV totals N/A
572	274 Issuer GIV totals
573	275 Issuer GIV totals N/A
574	261 Acq. batch cutoff totals
575	262 Acq. standin cutoff totals
576	263 Iss. batch cutoff totals
577	264 Iss. standin cutoff totals
578	265 Acq. settl cutoff totals
579	266 Acq. standin cutoff totals
580	267 Iss. settl cutoff totals
581	268 Iss. standin cutoff totals
582	281 Acq. batch cutoff tot NA
583	282 Acq. standin cut tots NA
584	283 Iss. batch cutoff tot NA
585	284 Iss. standin cut tots NA
586	285 Acq. settl cutoff tot NA
587	286 Acq. standin cut tots NA
588	287 Iss. settl cutoff tot NA
589	288 Iss. standin cut tots NA
590	289 Notif. settl cutoff
	6xx used in Admin
600	Original rcpt, retrieval rq
601	Original rcpt, repeat rq
602	Original rcpt, fulfilment
603	copy rcpt, retrieval rq
604	copy rcpt, repeat rq
605	copy rcpt, fulfilment
606	vehicle rental
607	hotel charge detail
608	POS photograph
609	proof of delivery
610	imprint
611	PVT 600/610
612	ICC infomation advice 620/630
640	VISA-type fraud advice
641	VISA free text message
642	VISA funds xfer totals
643	VISA copy request status
644	Euro copy request cancel(S44)
645	Request Response Cancel (S44)
646	Network Installment Info
	7xx used in Fee collection
700	fee collection
701	fee collection cnx full/part
760	reversal of 780 (IPM)

761	reversal of 781 (IPM)
762	reversal of 782 (IPM)
780	fee collection return (IPM)
781	fee collection resubmission (IPM)
782	fee collection arbitration return (IPM)
8xx used in Net Management	
801	Sign on to network
802	Sign off network
803	System unavailable
811	security / key change
812	security / alert
813	security / password change
814	security / device authent
821	sysacct / cutover
822	sysacct / checkpoint
831	system audit / echo test
841	Start of batch/file transfer
842	End of batch/file transfer
861	incoming traffic on
862	incoming traffic off
883	Advice on
884	Advice off
887	101 Key change request
888	160 Request new AWK
889	161 Request new IWK
890	162 Deliver new AWK
891	163 Deliver new IWK
892	270 Request current rec tot
893	280 Request previous rec tot
894	Europay reject message
896	Start upload (no sign-on)
897	Start upload after sign-on.